

IMPERIAL

MSC INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

A Semantic Representation Layer for Autonomous Testing Agents

Academic Supervisor:
Dr Robert Chatley

Author:
Timothee Issenmann

Industry Collaborator:
Andrey Breslav
CEO, Codespeak

Second Marker:
Alastair F. Donaldson

September 2026

Abstract

As generative coding agents reduce the cost of implementing a web application, more of the development effort shifts to verifying that it behaves as intended. Autonomous testing agents can perform that verification by exercising a running application through the browser as a user would, but they typically perceive it through screenshots, the DOM and the accessibility tree. These representations were not designed with agents in mind: they omit the purpose of each element and include detail the agent cannot act on, so every observation consumes more tokens than the task requires, and the cost of a test run scales with that excess.

This report shows that a web application annotated for a testing agent can be tested at lower token cost, with no loss of defect-detection ability on capable models. It presents Semtag, a closed vocabulary of semantic hints: **data-semtag-*** attributes that annotate front-end source code with each element’s role, identity, state and target. An extractor assembles these annotations at runtime into a compact semantic snapshot, which the agent observes in place of the accessibility snapshot.

The evaluation compares two agents that share the same harness, model and pre-defined test cases, over 840 runs across three models and fourteen AI-generated front-end applications from the WebTestBench dataset. The baseline agent observes the page through accessibility snapshots; the hinted agent observes it primarily through semantic snapshots, with fallback to the accessibility snapshot. On Claude Sonnet 5 and Claude Opus 5, the hinted agent reduced the cost of a run at API rates by 15.9% and 12.6% while matching the baseline on precision, recall and F1 to within three points, at unchanged latency. On Claude Haiku 4.5, it produced no cost saving but raised F1 by ten points, an effect consistent in direction but not statistically significant across fourteen applications. A semantic snapshot is on average 69% smaller than the accessibility snapshot of the same page; the saving per run is smaller, since observations are only part of the context a cached, long-context agent processes on each turn. The benefit of the semantic representation is therefore bounded by the rest of the agent’s context, and its character depends on the model: on the larger models it lowered cost, and on the smaller one it improved detection.

Acknowledgements

I would like to thank my supervisor, Dr Robert Chatley, and Andrey Breslav of Codespeak, for their close and sustained involvement throughout this project. Our regular conversations shaped the direction of the work at every stage, and their scrutiny raised the standard to which it was carried out. I am also grateful to Codespeak for their interest in this work and for covering the compute costs of the experiments; without their support, an evaluation of this scale would not have been possible.

Contents

1	Introduction	6
2	Related Work	8
2.1	End-to-End Web Testing	8
2.2	LLM Browser Agents	8
2.3	Agent-Oriented Interfaces	9
2.4	Browser Agents as Web Testers	10
2.5	Research Gap and Positioning	11
3	The Semantic Hint System Design and Vocabulary	12
3.1	Motivation and overview	12
3.2	Design goals and principles	14
3.3	The <code>data-semtag-*</code> system and vocabulary	14
3.4	Semantic snapshots and targeted observations	16
3.5	Rejected designs	17
3.6	Limitations of the design	18
4	System Implementation	19
4.1	The Hint Injector skill	19
4.2	The Hint Extractor MCP	21
4.3	Adapting the testing agent	21
4.4	Run harness and instrumentation	22
4.5	Verification tooling	22
5	Experimental Methodology	25
5.1	Benchmark choice	25
5.2	Dataset	25
5.3	Experimental design	27
5.4	Metrics and analysis	27
6	Results	29
6.1	Overview	29
6.2	Efficacy (RQ1)	30
6.2.1	Overall differences	30
6.2.2	Per-application differences	30
6.2.3	By test-case class	31
6.2.4	Fallbacks on accessibility trees	31
6.3	Efficiency (RQ2)	32
6.3.1	Tokens and cost	32
6.3.2	Latency and turns	34
6.3.3	Observation size	34
6.3.4	Effect of observation size reduction on savings	34
6.4	Generalisability (RQ3)	35
6.4.1	Across models	35
6.4.2	Across applications	35
6.4.3	By application category	37
6.5	Break-even against the cost of injection	37

7	Discussion and limitations	39
7.1	Answers to the research questions	39
7.2	Interpretation	39
7.2.1	The effect depends on the model	39
7.2.2	Explaining the token savings	40
7.2.3	What equal efficacy shows	40
7.3	Relation to prior work	40
7.4	Threats to validity	41
7.4.1	Internal validity	41
7.4.2	External validity	41
7.4.3	Construct validity	42
8	Conclusion and future work	43
8.1	Contributions	43
8.2	Implications	43
8.3	Future work	44
8.4	Closing remarks	44
	References	45
	Declarations	48
A	Full semantic snapshot of the home page of WebTestBench application 0001	50
B	Semtag Specification	55
B.1	Purpose	55
B.2	How hints reach the agent	55
B.3	The seven attributes	55
B.4	The ten roles	56
B.5	data-semtag-state	56
B.6	Naming	57
B.7	Repeated collections	57
B.8	Uniqueness	57
B.9	Coverage	58
B.10	Examples	58
C	Hint Injection Validator Checks	59
D	Baseline and Hinted Agent Prompts	61
D.1	Shared prompt core	61
D.2	Baseline agent prompt	62
D.3	Hinted agent prompt	63
E	Test-Case Corrections	66
F	Corrected Test Cases	88
G	Original and Corrected Checklist Comparison	100
H	Supplementary Statistical Tests	101
I	Wall-Clock Time Decomposition	102
J	Application-Level Predictors of Token Savings	103
K	Effects by Application and Category	104

List of Figures

3.1	Product list item on the admin page of application 0001.	17
4.1	System architecture of the Semtag experimental pipeline.	20
4.2	The Hint Overlay tool showing annotated and unhinted elements on a running application.	23
4.3	The semantic snapshot view in the Hint Overlay tool.	24
4.4	The Trace Viewer interface for a completed test run.	24
6.1	Difference in precision, recall and F1 between the hinted and baseline arms for each model. Error bars show 95% confidence intervals across the fourteen applications. .	30
6.2	F1 by application, model and experimental arm.	31
6.3	Relative change in effective-token use from the baseline to the hinted arm, by application and model.	33
6.4	Effective input-token use by cache status, model and experimental arm.	33
6.5	Mean accessibility- and semantic-snapshot sizes by application.	34
6.6	F1 against API-equivalent cost per run for each model and experimental arm. . . .	36
6.7	Accessibility-snapshot fallback rate against effective-token saving, by application. .	36
6.8	Cumulative testing cost with and without hints, including the upfront cost of hint injection with Table 5.	38

List of Tables

3.1	Semtag attribute vocabulary.	15
3.2	Semtag roles.	15
5.1	Applications included in the experimental dataset. Test-case and expected-failure counts reflect the corrected checklists.	26
6.1	Mean results by model and experimental arm.	29
6.2	Defect-detection F1 by test-case class, model and arm.	31
6.3	Observation calls per run in each arm, and the hinted agent’s use of the accessibility-snapshot fallback.	32
6.4	Cost of hint injection by injecting model.	37
C.1	Static checks performed by the hint-injection validator.	60
E.1	Full log of test-case corrections.	67
F.1	Full list of corrected test cases used in the final experiments.	89
G.1	Original and corrected checklist metrics by application.	100
H.1	Pooled and mean per-application effects of hints, with paired t-test and Wilcoxon signed-rank p-values.	101
I.1	Wall-clock time per run by component, model and arm.	102
J.1	Correlations between application-level predictors and effective-token change.	103
K.1	Effect of hints on tokens and F1 by application and category.	104

Chapter 1

Introduction

Recent advances in generative AI have fundamentally changed the way software can be produced. With little to no human intervention, coding agents can now generate applications from natural-language specifications. Where implementation once dominated development effort, the cost of producing a working application now falls increasingly on verification: confirming that what was generated actually satisfies the intended behaviour when executed.

The established way to check that a web application behaves correctly when running is to drive it through a browser, using browser automation frameworks such as Playwright [1], Cypress [2], or Selenium [3]. Executing the application in a real or headless browser exercises the behaviour that static inspection and unit tests cannot reach, such as interaction, client-side navigation, and asynchronous state transitions. The weakness of this approach is that the tests are written against the implementation, not the behaviour: they are coupled to the structure of the UI elements they select. A change to that structure can break the test even when the user-facing behaviour remains correct, so suites require continual maintenance. Generating those tests with AI does not resolve the problem, since a suite derived from the implementation records what the application does rather than what it was meant to do.

This motivates a different form of verification, in which a separate testing agent evaluates the running interface from the perspective of the user, following the original specification, rather than from the generated code. In testing terminology, this moves verification away from a white-box approach, where the tester relies on implementation structure, towards a black-box approach, where the system is assessed through observable behaviour.

Agents that operate a browser, or “browser agents”, offer the foundation for this kind of verification, since they can explore and check an application without a pre-written test script. This pattern is visible in leading commercial AI coding tools such as Claude Code [4], Codex [5] or Cursor [6], which increasingly incorporate browser-based verification after generating or modifying code. However, their practical obstacle is one of legibility. A browser agent perceives an application through the artefacts built to render it—HTML, the DOM, the accessibility tree—which explicitly describe how a page is structured, not what can be done with it or what state it is in. They are also exhaustive, encoding presentation detail, wrapper elements and repeated structure that carry no useful information the agent can act on. Capabilities and state must therefore be inferred from indirect cues at every step from a representation far larger than what the agent requires. Cost follows on both counts: each observation is processed by the model, so larger representations consume more tokens, while misread affordances add further steps and further observations.

This project addresses that obstacle by presenting Semtag, a compact, semantic representation of an application and its elements. Semantic hints are embedded in the front-end source code and extracted at runtime to build that representation, in place of the accessibility snapshot an agent would otherwise consume.

The setting in which this is evaluated is constrained. Our proposed system concerns functional end-to-end testing of generated front-end web applications: no backend is under test, and applications are executed in a controlled environment with synthetic data. Hints are injected into applications after they have been generated, since the dataset consists of applications that already exist; in practice however, they could be produced alongside the code itself. Test cases and their ground-truth oracles are defined in advance, and the agent is responsible for navigating, interacting with and observing the running interface to determine whether each case passes. The evaluation

compares two otherwise identical agents that differ only in what they observe. The baseline perceives the application through accessibility snapshots; the hinted condition perceives it primarily through semantic snapshots derived from the injected hints, with fallbacks to accessibility snapshots where these do not suffice. Both are measured on defect-detection efficacy, and on the cost incurred in terms of tokens and latency. The following research questions follow:

RQ1 — Efficacy

How does the semantic representation affect defect-detection precision, recall and F1?

RQ2 — Efficiency

How does it affect token use, cost and latency?

RQ3 — Generalisability

How do these effects vary across models and applications?

In answering them, the project makes three contributions:

1. A specification comprising a vocabulary of semantic hints for front-end source code and a compact observation format derived from them.
2. A reference implementation of the specification, covering hint injection, runtime extraction, verification, and agent-driven test execution.
3. An empirical comparison of hinted and unhinted agents, reporting defect-detection efficacy alongside token use, cost and latency.

Across three models and fourteen applications, the semantic representation reduced token use by 15.9% on Anthropic’s Claude Sonnet 5 model and 12.6% on Claude Opus 5 model while matching the baseline’s defect-detection precision and recall, at equal latency. The effect was dependent on the model used: on Claude Haiku 4.5 it produced no token saving, but appeared instead to improve defect detection, with hinted runs pursuing the task further before terminating.

Chapter 2 reviews the literature on end-to-end web testing, browser agents and agent-oriented interfaces, and positions this work in the gap between them. Chapter 3 presents the design of Semtag: its goals, the hint vocabulary, the semantic snapshot format, and the alternatives that were rejected. Chapter 4 describes the software built to realise it, from the injection skill and extractor MCP server to the adaptations made to the WebTester harness. Chapter 5 sets out the experimental methodology, including the choice of benchmark, the correction of its test cases and the metrics on which the two arms are compared. Chapter 6 reports the results against each research question in turn, and Chapter 7 interprets them, relates them to prior work and assesses the threats to their validity. Chapter 8 summarises the contributions and their implications and proposes directions for future work.

Chapter 2

Related Work

2.1 End-to-End Web Testing

End-to-end testing validates an application by exercising it through the same browser-facing interface used by end users, simulating realistic scenarios across a full workflow and checking that the resulting behaviour matches expectations [7, 8]. This makes it the standard instrument for front-end systems, where failures often surface only at runtime, once rendering, interaction and asynchronous state are involved.

Its weaknesses are well documented. A survey of Selenium WebDriver practitioners identified asynchronous interactions, brittleness and flakiness among the most significant challenges faced in practice [7]. WEFix similarly describes web E2E tests as prone to inconsistent outcomes even when the application code has not changed, often because the test and the client-side application execute in nondeterministic orders [8]. A separate line of work on robust locators shows that scripts break as interface structure evolves, because tests depend on selectors and DOM paths that are not stable across releases [9]. These are not incidental defects, but consequences of how such tests are written: a test encodes a fixed sequence of selectors, waits and assertions, coupled to the structure of the implementation.

Model-based approaches addressed part of this by deriving application structure. Crawljax crawls Ajax-based applications by dynamically exercising user-interface events and constructing a state-flow graph of observed UI states and transitions [10], and later crawl-based analysis frames web testing as the discovery and analysis of the state space exposed through the browser [11]. The state and action structure of an interface has therefore long been treated as central to automated testing, and as something an automated tester must recover for itself.

Execution is only part of the problem. While browser automation tools like Playwright, Selenium and Crawljax can perform actions and discover transitions, something must still determine which behaviours are meaningful and what outcome counts as correct. This is the test-oracle problem: deciding, given an observed behaviour from the system under test, whether that behaviour is correct [12]. In this work, oracles are defined in advance, so the agent’s task is to determine whether the interface behaves as the test case describes, not to decide what correct behaviour would be. The difficulty that remains is establishing what the interface currently offers and what state it is in.

2.2 LLM Browser Agents

LLM-based browser agents replace the pre-written script with a loop. Given a natural-language goal, the agent observes the current page, selects an action, executes it, and observes again, repeating until the task is complete or abandoned. This loop is usually described as scrape, parse, reason, and act: the agent extracts the page into a representation it can process, reasons over that representation, and issues browser actions accordingly [13]. For testing, the appeal is that the agent is not restricted to selectors chosen in advance, and can determine how to exercise an interface from a description of what should happen rather than from a script of what to click.

Browser agents differ mainly in what they observe, and the options are limited to the artefacts a browser already produces. Screenshots preserve visual layout but require structure and intent to be recovered from pixels; neither condition evaluated in this work receives them, so the comparison

stays confined to textual representations of the interface. Raw HTML and DOM snapshots expose the page exactly as implemented, including presentation detail, wrapper elements and repeated structure that bear on no decision the agent must make. Accessibility trees are the most economical of the three, since they were built to convey interface semantics to assistive technology rather than to describe markup. However, they convey only what native HTML elements and ARIA attributes make available [14], which is a description of what an element *is*, not its role in the wider context of the application. As a result, failures in browser agents often arise not because the model cannot reason about the task, but because it has misidentified what is present on the page, what an element does, or whether the interface has changed after an action. Lu et al. characterise this as a structural mismatch between interfaces built for human use and the requirements of agentic interaction [15].

The consequences are visible in how existing systems compensate. WebVoyager combines screenshots with textual page signals and annotated element boxes, suggesting that screenshots alone do not support reliable action grounding on real websites [16]. SeeClick works from screenshots exclusively and addresses the resulting difficulty by improving GUI grounding, the task of locating the correct element from a language instruction [17]. OmniParser parses screenshots into structured interactable regions with captions, so that a vision-language model reasons over detected elements, not raw pixels [18]. Agent-E works primarily from the DOM, but requires distillation, denoising, hierarchical planning and explicit change observation to make the environment tractable [19]. These approaches point to the same underlying issue: the browser does not expose the task-level semantics agents need, so each system has to reconstruct those semantics from indirect evidence.

The cost of this compensation is measurable, and substantial. In UI-agent settings, interface representation has been found to account for 80-99% of total input token consumption, with optimised transformations reducing token cost by 48.7-55.8% across the evaluated tasks while preserving, and in some cases improving, task performance [20]. Observation is therefore the dominant share of the agent’s work, paid at every step of the loop.

Several systems address this directly by reducing what the agent observes. FocusAgent starts from the accessibility tree, which is roughly ten times smaller than the full DOM but can still exceed tens of thousands of tokens, and retrieves only the portions relevant to the current task, halving observation size while maintaining competitive performance [21]. Prune4Web takes a programmatic route, noting DOM structures in the range of 10,000 to 100,000 tokens and reporting a 25-50x reduction in candidate elements after pruning [22]. LineRetriever treats the problem as retrieval over long observations, selecting lines from a DOM or accessibility tree according to the agent’s current plan, and reducing input tokens by 61-73% [23]. Nor is the appropriate degree of reduction fixed. The best observation format appears to depend on the capability of the model and the reasoning budget available, with stronger models sometimes benefiting from richer context where more constrained agents benefit from aggressive reduction [24]. Whether a single representation can serve models of different sizes is a question our evaluation addresses directly.

These results establish that observation size is reducible without proportional loss of capability, but they share a structural property: the reduction is computed from the full representation, on every observation. The page is produced in full, then filtered, ranked or pruned before the agent reads it, so the cost of deciding what matters recurs at each step of every run. The alternative is to move that work into the application itself, so that it is done once and reused on every observation. That is the direction this project takes, and the next section develops it.

2.3 Agent-Oriented Interfaces

The systems in the previous section improve how agents interpret representations built for other purposes. A different response is to change what the page exposes. Lu et al. argue for building the web *for* agents rather than adapting agents to interfaces designed for humans, proposing design principles for agent-specific interfaces with more efficient observations and constrained action spaces [15]. The premise is that the mismatch identified above is a property of the interface, not a deficiency of the model, and is therefore better fixed at the source.

Accessibility standards are the existing precedent for exposing machine-readable interface semantics. WAI-ARIA defines roles, states and properties that allow browsers and assistive technologies to determine the type, relationships and current state of interface elements [14], and these are what an accessibility snapshot carries. But the vocabulary is fixed to what accessibility requires. It

can express that an element is a button in a pressed state; it cannot express which domain object that button acts on, what the action is expected to trigger, or which parts of the page constitute its result.

Recent proposals extend the idea beyond accessibility and are designed to meet the needs of browser agents. VOIX defines a declarative framework in which a site exposes structured descriptions of page state, available actions and interaction constraints, giving agents an explicit contract [25]. webMCP takes a metadata route: the page ships a static JSON descriptor mapping each interactive element to its selector, role, parameters and endpoint, so the agent reads a compact interaction graph instead of parsing the full document, while still executing the actions itself [13]. CI4A applies the idea at component level, defining semantic component interfaces that expose a component’s state and supported actions as tool primitives [26]. Chrome’s proposed WebMCP standard goes furthest, replacing DOM interaction altogether: the page registers JavaScript functions with JSON Schema signatures through a browser API, and the agent calls those functions while the site’s own code carries out the action [27].

These proposals have in common that they expose a curated subset of the interface’s actions and state, treating the rest of the page as implementation detail. For an agent completing a task as an end user, that is the point: a small, well-described action space serves it better than the full interface. But the compression is lossy by design. Anything the contract omits is invisible to the agent, so a testing agent working from it can only exercise the paths the site’s authors chose to describe, not the full range of what a user can do through the GUI.

These proposals also share an obstacle: they require adoption. An interface contract is only useful to an agent if the sites it visits implement it, which means persuading developers at large to instrument applications for the benefit of third-party agents. Their designs reflect that setting in other ways too. Because the site, the user, the agent and the agent’s provider may be separate parties, they must address disclosure across a trust boundary; webMCP, for instance, uses encrypted payloads to protect private form data from the model [13].

These requirements do not apply to testing. The party running the tests is the party that maintains the source, so instrumentation needs no coordination beyond the codebase and no adoption beyond the project that chooses it. More importantly, the behaviour under test is the human-facing GUI itself, so a contract that abstracts part of it away leaves out exactly what the agent must test. What remains from these proposals is the underlying idea that an application can describe itself to an agent more economically than an agent can infer a description.

2.4 Browser Agents as Web Testers

A browser-based testing agent must derive its own testing plan from high-level human directions, and then determine whether the application meets the requirements. Feldt et al. call such systems Autonomous Testing Agents (ATAs) [28]. Existing ATAs differ in how much of these two tasks, test-case generation and test-case execution, they take on.

Translating natural-language requirements into executable tests is a long-standing research problem, surveyed by Mustafa et al. [29], independently of LLMs. Ferreira et al. generate scenarios from user stories and then produce Cypress tests using the HTML of the pages under test [30]. This automates authoring, but the output is still a script whose selectors and assertions are bound to the implementation they were generated against, and which inherits the fragility described in Section 2.1.

Execution by an agent, rather than by a fixed script, is what separates an ATA from a test generator. Chevrot et al. introduce SeeAct-ATA and PinATA, in which browser agents navigate an application, verify assertions and return pass/fail verdicts on natural-language manual test cases [31]. Their studies show that execution is itself two tasks: the agent must drive the interaction to the right point, and then judge whether what it observes satisfies the case. Exploratory systems such as WebProber, which crawls an application from a URL and reports bugs without a specification to check against [32], perform the first task without the second and are therefore outside the scope of this work.

Some systems take on both tasks. WebTestPilot derives requirements from user-facing documentation, parses them into steps, infers preconditions and postconditions, symbolises the relevant GUI elements, and generates executable assertions over those symbols [33]. WebTestBench similarly derives its own oracles before executing them [34].

Evaluating an ATA requires a benchmark that can expose both whether the agent can complete a test case and whether it can correctly judge the result. Testing benchmarks obtain the second by seeding defects into the applications under test, so that the correct verdict is known in advance and a missed defect can be attributed to the agent. WebTestPilot evaluates against bug-injected variants using requirements derived from user-facing documentation, reporting task completion, trace correctness, and bug-detection precision and recall [33]. WebTestBench evaluates ATAs over a dataset of 100 AI-generated applications containing natural defects from code generation [34]. General-purpose agent benchmarks such as Mind2Web [35], WebArena [36], WebVoyager [16] and WorkArena [37] measure only the first: whether an agent completes a task in a realistic browser environment, with BrowserGym providing a common harness for running them [38].

Salva’s work is the closest published setting to ours [39]. It takes natural-language test cases as input, focuses entirely on their execution by an agent, and evaluates the correctness of the resulting verdicts across repeated runs on six web applications and three models. The evaluation finds that high-capability models rarely report false negatives or false positives, while weaker models degrade sharply on both counts. Additionally, agent behaviour varies substantially across repeated executions, which motivates the guardrails that work introduces. Where that study holds the observation constant and varies the executor, this work does the reverse.

WebTestBench provides the dataset and the agent harness used in this work. Its dataset consists of front-end web applications with defects and per-application checklists of test cases, supplying the fixed oracles the evaluation depends on [34]. Its defect-detection agent, WebTester, gives a baseline implementation that can be held constant while the observation format is varied for the treatment. Section 4.3 describes the modifications made to WebTester, and Section 5.2 covers the corrections that were made to its dataset of test cases.

2.5 Research Gap and Positioning

Work on general browser agents shows that observation is the dominant cost of the agentic loop, and that it can be substantially reduced without proportional loss of capability. But the reduction techniques surveyed in Section 2.2 take the application as given. The agent works from what the browser already exposes, and improves only how much of it is read. The application’s meaning, state, and capabilities still have to be inferred at every step.

Research on agent-oriented interfaces removes that limitation at the source, by having the application declare its own state and capabilities. As Section 2.3 argued, however, these interfaces compress the action space to serve task completion, and have not been evaluated in a setting where the interface must be preserved in full and the agent acts as a tester.

Work on autonomous testing agents, finally, has established that browser agents can execute natural-language test cases and return verdicts, and has found that accuracy varies with model capability and across repeated runs. But the systems surveyed here read the page in the form the browser produces for rendering, not for understanding. The cost of that observation, and its effect on the verdicts, has not been isolated.

This is the gap this work addresses. It brings the premise of agent-oriented interfaces – that an application can describe itself to an agent more concisely than the agent can infer a description – into the setting of autonomous testing. The evaluation is a controlled one: two agents that share the same harness, model, test cases and applications, differing only in whether they observe an accessibility snapshot or a semantic snapshot derived from hints in the source. Because oracles are fixed and the executor is held constant, any difference in efficacy or cost given sufficient repetitions is attributable to the representation. The research questions in Chapter 1 follow directly from this design, asking in turn whether the semantic representation preserves defect-detection ability, what it saves, and whether the answer depends on the model or the application.

Chapter 3

The Semantic Hint System Design and Vocabulary

Chapter 2 established that a browser agent pays for observation on every step of its loop, and that the representations available to it describe how a page is built, not what the elements mean. This chapter presents Semtag, the hint system designed to address both problems for the specific case of testing. Section 3.1 motivates the design with an example, Section 3.2 sets out its goals, Section 3.3 defines the hint vocabulary, and Section 3.4 describes the semantic snapshot an agent receives at runtime. Section 3.5 records the designs that were considered and rejected, and Section 3.6 the limitations of the one chosen.

3.1 Motivation and overview

The accessibility tree is the most compact of the representations a browser produces, and it is what the baseline agent in this work observes. It is still far from minimal. Below is an excerpt from the accessibility tree of the home page of application 0001 in the WebTestBench dataset, for the navigation header of the page:

```
- generic [ref=e2]:
- region "Notifications (F8)":
- list
- region "Notifications alt+T"
- generic [ref=e3]:
- banner [ref=e4]:
- generic [ref=e6]:
- link "CuratedPicks" [ref=e7] [cursor=pointer]:
- /url: /
- generic [ref=e8]: CuratedPicks
- navigation [ref=e9]:
- link "Home" [ref=e10] [cursor=pointer]:
- /url: /
- link "All Products" [ref=e11] [cursor=pointer]:
- /url: /products
- link "Tech" [ref=e12] [cursor=pointer]:
- /url: /category/tech
- link "Home" [ref=e13] [cursor=pointer]:
- /url: /category/home
- link "Fitness" [ref=e14] [cursor=pointer]:
- /url: /category/fitness
- generic [ref=e15]:
- generic [ref=e17]:
- img [ref=e18]
- searchbox "Search products..." [ref=e21]
- link [ref=e22] [cursor=pointer]:
- /url: /admin
- button [ref=e23]:
- img
```

Two properties stand out. The first is that much of it carries no information the agent can act on: nested `generic` nodes with no role of their own, repeated structure, and reference and cursor

annotations on many lines. While each of these costs a few tokens, an agent ingests observations like this hundreds of times over a test run. The second is the absence of application-level meaning. The tree exposes what each element is (e.g. a button, a link or an image), not what it is for. It cannot say that a button leads to the checkout, that an image shows the product on sale, or that a section lists the product's specifications. The agent must reconstruct that meaning through reasoning at each step, spending further tokens on inference.

Semtag addresses both properties. At a high level, it operates in two phases. In the first, during or after code generation, the components of the page are annotated in the source with `data-semtag-*` attributes. These are invisible in the rendered interface and describe each component's role in the application, its identity, the state it carries, and more. They are analogous in mechanism to ARIA attributes, which also annotate markup with information the rendering does not need; however, their vocabulary is chosen for what an agent needs to understand and interact with the interface rather than for what assistive technology needs. In this project, the annotations are placed by an agentic hint-injection skill, described in Chapter 4, but nothing in the design depends on that; a developer or a code generator could place them directly. Here is an example of a navigation component where Semtag hints were injected:

```
<nav className="hidden md:flex items-center gap-1">
  {navLinks.map((link) => (
    <Link
      key={link.href}
      to={link.href}
      data-semtag-id={link.semtagId}
      data-semtag-role="navigation"
      data-semtag-target={link.semtagTarget}
      className={cn(
        'px-3 py-2 text-sm font-medium rounded-md transition-colors',
        location.pathname === link.href
          ? 'text-primary bg-primary/5'
          : 'text-muted-foreground hover:text-foreground hover:bg-muted'
      )}
    >
      {link.label}
    </Link>
  ))}
</nav>
```

In the second phase, at runtime, the annotations are extracted from the live page and assembled into a semantic snapshot: a representation in which only the annotated components appear, described in the terms supplied by the annotations. In this project, the extraction is performed by a semantic-extractor MCP server, also described in Chapter 4. The hinted agent observes the application through that snapshot alone, with fallbacks on the accessibility tree when necessary. It never sees the hinted attributes in the source, nor the DOM from which they were extracted. Here is an excerpt of the semantic snapshot for the same header presented above:

```
"navigation": [
  {
    "id": "nav.logo",
    "name": "CuratedPicks",
    "target": "home.page"
  },
  {
    "id": "nav.home",
    "name": "Home",
    "target": "home.page"
  },
  {
    "id": "nav.products",
    "name": "All Products",
    "target": "products.page"
  },
  {
    "id": "nav.category.tech",
    "name": "Tech",
    "target": "category.tech"
  },
  {
    "id": "nav.category.home",
```

```

    "name": "Home",
    "target": "category.home"
  },
  {
    "id": "nav.category.fitness",
    "name": "Fitness",
    "target": "category.fitness"
  },
  {
    "id": "nav.admin",
    "name": "",
    "target": "admin.page"
  },
}

```

The wrapper nodes (“generic” lines in accessibility tree), reference numbers and cursor annotations are gone; what remains is one entry per element the agent can act on, and the **navigation** group states what kind of element it is. Each entry carries a stable identifier the agent can use directly in an action, and a **target** that names where the link leads (**category.tech**). Two things that were ambiguous in the accessibility tree are now explicit: the two links both labelled “Home” are distinguished by identifier (**nav.home**, **nav.category.home**), and the icon-only admin link, which had no accessible name at all, is identified as **nav.admin** with its destination stated. The notification regions at the top of the tree, which serve no purpose to a tester, do not appear because nothing marked them. The semantic snapshot excerpt is 729 characters, against 1,018 for the same header in the accessibility tree. Appendix A gives the full semantic snapshot of the home page of application 0001, and states where the accessibility snapshot of the same page can be found.

3.2 Design goals and principles

The design of Semtag follows from the position taken in Chapter 2: that a testing agent needs the full human-facing interface made legible, not a narrower interface made convenient. Four goals follow.

First, completeness of coverage. The hints must give the agent enough to observe, understand, navigate and interact with the application to the same extent as a human user could. This requires semantic information on every part of the interface a user can reach, and stable identifiers the agent can use to target specific components in its actions.

Second, compactness of representation. The hints must carry only what the agent strictly needs, so that the resulting snapshot is substantially smaller than an accessibility snapshot of the same page without sacrificing the coverage principle above.

Third, generality across application structure. The system must work in dynamic, component-based applications such as those built with React, where the page is assembled at runtime from reusable components, and where the same component may appear many times with different data. Hints must therefore describe live state, not only static attributes.

Fourth, simplicity of the design. The vocabulary must be small and unambiguous enough that a coding agent can inject it consistently across a codebase, that a straightforward extractor can rebuild a snapshot from it, and that tooling can validate the result.

Beyond these goals, one constraint applies to every hint: it describes the interface, and never encodes intended outcomes, assertions or pass/fail logic. This keeps the oracle in the test cases given to the testing agent.

3.3 The data-semtag-* system and vocabulary

All Semtag attributes use the **data-** prefix, following the HTML mechanism for attaching custom, non-rendering data to elements. They are therefore valid in any document, ignored by the browser, and readable by the extractor through the standard DOM API. Only **data-semtag-*** attributes are read by it; data placed in any other attribute never reaches the agent.

The vocabulary is closed and comprises seven attributes, shown in Table 3.1:

Table 3.1: Semtag attribute vocabulary.

Attribute	Value	Purpose
<code>data-semtag-id</code>	Dotted domain path (<code>checkout.submit</code>)	Stable, page-unique identifier. Required on every hinted element; the handle the agent uses in actions.
<code>data-semtag-role</code>	One of ten roles (below)	The element’s semantic category, and the group under which it appears in the snapshot.
<code>data-semtag-action</code>	Verb-object (<code>submit-order</code>)	The user-level action a control performs.
<code>data-semtag-state</code>	State key(s) (<code>cart.total</code>) or literal	What the element reads or edits.
<code>data-semtag-target</code>	Destination name (<code>product.detail</code>)	Where a navigation leads, named by what it is rather than by URL.
<code>data-semtag-controls</code>	A region’s <code>data-semtag-id</code>	The region a control drives, where there is a distinct one (a search field and its result list).
<code>data-semtag-options</code>	<code>value Label</code> ; <code>value Label</code>	A select trigger’s available options, so the agent can choose without opening the menu.

For the `data-semtag-role` attribute, ten roles were defined, as shown in Table 3.2:

Table 3.2: Semtag roles.

Role	Applies to
<code>navigation</code>	Link or button that moves to another screen or view.
<code>action</code>	Command button: e.g. <code>submit</code> , <code>buy</code> , <code>edit</code> , <code>delete</code> , <code>add</code> .
<code>option</code>	One option within a select or listbox.
<code>input</code>	Text, number or search field, or textarea.
<code>select</code>	Dropdown trigger, paired with <code>data-semtag-options</code> .
<code>toggle</code>	Checkbox, switch, on/off chip, tag filter.
<code>slider</code>	Range slider.
<code>observable</code>	Element that only displays a value to be read.
<code>region</code>	Meaningful grouping: form, filter panel, dialog, banner.
<code>collection</code>	Container of repeated items: e.g. <code>list</code> , <code>grid</code> , <code>table</code> .

Any element with an identifier but no role, or a role outside this list, is filed under an **other** category in the semantic snapshot.

The vocabulary is completed by a few conventions on how the attributes are used. Identifiers (`data-semtag-id`) name domain concepts, not implementation details or positions (e.g. `checkout.submit` rather than `button-3`, or `products.grid.item.sony-xm5` rather than `card-component-7`). They are always a new attribute, never a reuse of an existing id or `data-testid` from the code, since those carry no meaning and are bound to the implementation rather than the domain, which is the coupling we previously identified as the source of selector fragility. They are unique across the page; if one component appears twice, in two collections or in desktop and mobile layouts, the identifier is scoped by its container or suffixed, while its state key stays the same.

The state of a component, in turn, is read at extraction time. For roles that hold a value (`input`, `select`, `toggle`, `slider`, `observable`), the extractor captures the element’s current value at runtime, so `data-semtag-state` names what the value is, such as `cart.total`. A literal is used only where the state is not visible as text, such as `open` or `selected`. A control that edits a value and the element that displays it carry the same `data-semtag-state` key-value pair, so the agent knows which observable that action will change.

Coverage of hints is determined by one rule: an element is annotated if an agent might need to navigate to it, act on it, or read it. Decorative elements, layout wrappers, and icons that duplicate

an adjacent labelled control are left out. Anything the agent might click must be reachable by its identifier alone, so repeated instances, such as items in a list, each get their own.

The following excerpt from the specification shows the attributes in use on representative components:

```
<!-- action -->
<button data-semtag-id="checkout.submit"
        data-semtag-role="action"
        data-semtag-action="submit-order">Place order</button>

<!-- observable: the extractor captures "Total: £42.00" as its value -->
<div data-semtag-id="cart.total"
      data-semtag-role="observable"
      data-semtag-state="cart.total">Total: £42.00</div>

<!-- navigation -->
<a data-semtag-id="nav.orders"
   data-semtag-role="navigation"
   data-semtag-target="orders.page" href="/orders">Orders</a>

<!-- input -->
<input data-semtag-id="shipping.postcode"
        data-semtag-role="input"
        data-semtag-state="shipping.postcode" name="postcode" />

<!-- select trigger listing its options, driving a region -->
<button data-semtag-id="filters.sort"
        data-semtag-role="select"
        data-semtag-action="sort-products"
        data-semtag-controls="products.grid"
        data-semtag-options="newest|Newest;price-asc|Price: Low to High">Sort by</button>

<!-- region -->
<section data-semtag-id="checkout.form" data-semtag-role="region">...</section>
```

The complete specification of the Semtag vocabulary can be found in Appendix B.

3.4 Semantic snapshots and targeted observations

At runtime, the extractor reads every element carrying a `data-semtag-id` and assembles a semantic snapshot of the current page. The snapshot is serialised as JSON, a format that models read and parse reliably. Every snapshot begins with two fields, `url` and `screen`, giving the current URL and the annotated name of the page. The remainder is grouped by role: all navigation together, all actions, all inputs, and so on. Grouping by role saves the role field from being repeated on every entry. It also means all elements of one kind sit together, so the agent looking for a specific element can focus on one section.

The extractor infers nothing that is not in the Semtag hints. It never uses tag names, ARIA roles or input types to guess an element’s meaning, and it never completes a partial hint: an element with an identifier but no role appears under `other`. The snapshot is therefore an exact record of what was annotated, which keeps the hinted agent’s results attributable to the hints alone.

Collections, meaning containers of repeated items such as lists, grids and tables, account for much of an accessibility snapshot’s size. The extractor “folds” collections into JSON blocks, stating the shared structure only once: id, role, state, actions, and fields – and then each item as a single row with field values. Folding is lossless: every item and control remains individually addressable by its full identifier; but it is much shorter than stating each element as its own block in the snapshot, especially for large collections.

Consider the product list item on the admin page of application 0001 (Figure 3.1). That one item occupies 27 lines of the accessibility tree:

```
- generic [ref=e492]:
```

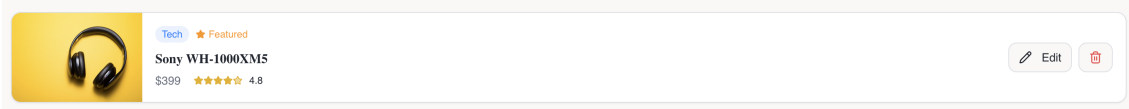


Figure 3.1: Product list item on the admin page of application 0001.

```
- generic [ref=e494]:
- img "Sony WH-1000XM5" [ref=e496]
- generic [ref=e497]:
- generic [ref=e498]:
- generic [ref=e499]:
- generic [ref=e500]: Tech
- generic [ref=e501]:
- img [ref=e502]
- text: Featured
- heading "Sony WH-1000XM5" [level=3] [ref=e504]
- generic [ref=e505]:
- generic [ref=e506]: $399
- generic [ref=e507]:
- generic [ref=e508]:
- img [ref=e509]
- img [ref=e511]
- img [ref=e513]
- img [ref=e515]
- img [ref=e517]
- generic [ref=e519]: "4.8"
- generic [ref=e520]:
- button "Edit" [ref=e521] [cursor=pointer]:
- img
- text: Edit
- button [ref=e522] [cursor=pointer]:
- img
```

In the semantic snapshot the same item is one row,

```
"items": [
  ["1", "Sony WH-1000XM5", "$399", "4.8"]
```

A full snapshot is not always needed by the agent. The extractor should also provide for a targeted observation: a request for a single element or region by identifier, returning only its current state and attributes. After an action whose effect is known in advance, an agent can check the one observable it cares about instead of re-reading the page.

3.5 Rejected designs

Several alternatives were considered before settling on the design above.

A sidecar interaction graph. The first alternative, and the one closest to webMCP [13], was to describe the interface in a separate file: a JSON model of the pages, their elements, actions and transitions, maintained alongside the source code, not embedded within it. It was rejected because a sidecar is a second source of truth. Any change to the components that is not mirrored in the file makes the two implementations divergent. webMCP itself anticipates that drift: its toolchain includes a CI check that fails a pull request when selectors in the sidecar no longer match the page [13].

Hints without extraction. A simpler design would add the semantic attributes to the source and let the agent read them in the DOM or accessibility tree, with no separate snapshot. This would supply the missing meaning, but not the compactness: the agent would read the full representation plus the annotations, so input tokens would rise. Also, Playwright's accessibility snapshot, like any observation derived from the accessibility tree, does not surface `data-*` attributes, so the agent would have to fall back to the raw DOM.

Reusing ARIA. Instead of introducing new attributes, the existing ARIA vocabulary could have been extended or repurposed, since accessibility snapshots already carry it. This was rejected for two reasons. ARIA's vocabulary is fixed to what assistive technology needs, and cannot express application-level roles, targets or state keys [14]. And ARIA attributes have real end users beyond

testing agents: values invented for an agent would reach screen readers and change the application’s accessibility behaviour.

Exposing actions as callable tools. webMCP and CI4A let the agent invoke an application’s capabilities directly, as functions, rather than through the interface [13, 26]. That route was closed for the purpose of the system. A testing agent must exercise the interface a user sees; calling a function that bypasses the button under test says nothing about whether the button works.

An open-ended vocabulary. Letting the injector choose its own roles and attributes would have been more expressive, but would go against the fourth goal: simplicity of the design. If a coding agent invented different labels across files, the testing agent could not be taught a stable format, and the snapshot could not group entries by role as easily, because the set of roles would not be known. Validation of hints would become difficult for the same reason.

Inferring missing semantics. Where a hint was absent or incomplete, the snapshot could have filled the gap from tag names, ARIA roles or input types. This was rejected because it would make the hinted agent’s results depend on the quality of the underlying markup rather than on the hints, confounding the comparison.

Incremental snapshots. A final alternative was to send only what had changed since the previous observation. This would reduce tokens further on pages that change little between steps, but it was rejected for three reasons. It requires the agent to maintain an accurate picture of the page across turns; it offers little token savings on navigation, where most of the page changes; and the common case it serves, checking one element after an action, is already covered more simply by targeted observations. A full snapshot has the further advantage of being self-contained, so each observation can be understood without the ones before it, which matters in long test runs where earlier context may be compacted or lost.

3.6 Limitations of the design

The design carries limitations that follow from its own choices.

Hints go stale. The most serious limitation is maintenance. Hints live in the source code, but they can still diverge from the interface they describe, and nothing detects the drift: they would still be present and extracted. Keeping hints up to date is therefore a maintenance responsibility similar to the one we identified for test scripts, and which agent-based testing was meant to remove. In practice however, the same coding agents that change the implementation can be instructed to update its hints in the same pass, provided they are given the Semtag specification.

Quality depends on the injector. The semantic snapshot is only as good as the hints in the source code. Coverage is left to the injecting agent’s judgement of what a tester might need to reach, and a gap in that judgement is a blind spot for every test that follows. In this work, injection was performed by a highly capable model and reviewed by hand, and the concern should diminish as coding agents continue to improve. However, the design offers no guarantee of completeness.

Feedback outside the annotated DOM is not described. Interface feedback produced outside the annotated tree, such as toasts rendered by a notification library, native validation bubbles, and browser confirmation dialogs, carries no hint and is absent from the snapshot.

Visual behaviour is not described. Hints describe what an element is, does and displays. They do not describe how it looks: layout, colour, alignment, spacing, imagery and any content rendered into a canvas have no representation in the hint vocabulary. Without the ability to capture screenshots, a test whose oracle is visual (e.g. “*Images presented [...] are consistent with the item’s name and description*”) cannot produce an accurate verdict from a semantic snapshot. This was evident in our evaluation: test cases with visual oracles were where both agents performed worst, and where the hinted agent gained nothing over the baseline.

The closed vocabulary limits expressiveness. Ten roles and seven attributes cover basic components well, such as forms, navigation, or lists. They do not cover components that do not decompose into those primitives as well: a calendar picker, a drag-and-drop board, a rich-text editor, a chart, a canvas. The evaluation showed this directly: the applications on which the hinted agent fell back on accessibility snapshots most often were those built around such components, where the hints described the component’s boundary but not the interactions inside it.

Chapter 4

System Implementation

In the previous chapter, we described the design and goals behind the semantic hints and snapshot system; in this chapter, we describe the software that was built to implement these concepts. Figure 4.1 shows how the components fit together, following the path of an application through the experiment.

The Hint Injector skill (Section 4.1) annotates an application’s source with `data-semtag-*` attributes and validates the result. The Hint Extractor MCP (Section 4.2) reads those attributes from the running page and serves semantic snapshots to the agent. The testing agent itself (Section 4.3) is WebTester, the harness published with the WebTestBench benchmark, adapted so that the baseline and hinted arms differ only in what they observe. The run harness (Section 4.4) launches repetitions, scores them and records the token, cost and latency figures for the evaluation. Finally, two tools built for human review (Section 4.5) let the experimenter verify that injected hints are correct and inspect what an agent did during a run.

4.1 The Hint Injector skill

In this work, hints are injected into the applications after the code has been written, since the dataset consists of applications that were already generated; if hints were placed in the code during generation, this component would not be needed. Injection is performed by an agent using a “skill”: a packaged, installable directory containing a prompt and the Semtag specification in markdown format, which a coding agent loads when the skill is invoked. The specification is kept in its own file rather than paraphrased into the prompt, so the two can be revised independently. Turning the prompt into a skill also makes injection self-contained: it can be installed and run unchanged on any repository, so the treatment can be easily reproduced in future experiments.

Given a repository, the skill has the agent proceed in five steps. It first reads the `hint-design.md` specification file. It then inspects the application to identify its screens and the controls, forms, navigation, collections and displayed state a tester would need to reach; this is the point at which coverage is decided and left to the agent’s judgement. Third, it adds `data-semtag-*` attributes to the corresponding elements, leaving visible behaviour and existing test logic unchanged. Fourth, it runs the validator script described below and repairs the reported warning or errors. Finally, it runs the application’s own checks, such as type-checking, linting, build and any existing tests, to confirm nothing was broken, and reports what was annotated and any gaps it could not close. A separate harness was built to launch the agent, invoke the skill, and record the wall-clock time and token use of the whole injection run.

The validator is a static script that scans front-end source files (`.jsx`, `.tsx`, `.vue`, `.svelte`, `.html`) and checks the injected attributes against the specification, without running the application. It verifies the following: that only the seven attributes and ten roles are used; that every hinted element has an id; that ids are unique across files and not keyed by position; that collection items sit inside a collection container; and that no attribute value contains test intent such as `expected` or `assert`. The full list of checks is given in Appendix C.

What the validator cannot check is whether the hints are true: whether the role, state key and target on an element actually correspond to what the element is and does at runtime. That is a judgement about the running interface, not the source, and it is the reason we built the hint overlay tool, described in Section 4.5.

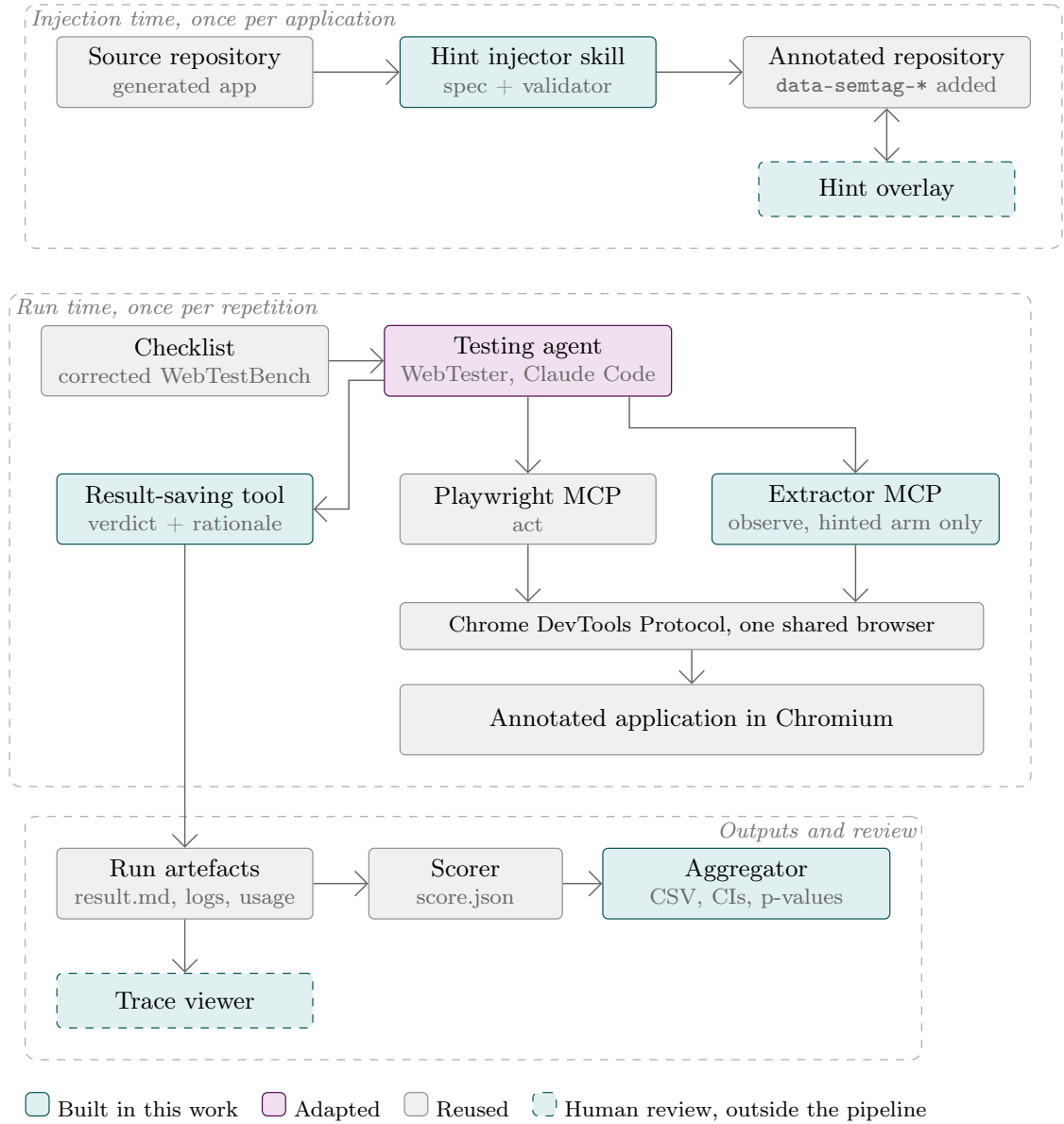


Figure 4.1: System architecture of the Semtag experimental pipeline.

4.2 The Hint Extractor MCP

The extractor is the runtime part of the toolchain: it reads the `data-semtag-*` attributes from the live page and assembles the semantic snapshot. It is packaged as an MCP server, installable alongside any MCP-capable agent, and it is strictly read-only. It exposes two tools, `semantic_snapshot` and `semantic_observe`. All interactions with the page go through a second server, the official Playwright MCP, which the testing agent calls in parallel. We chose this separation over forking Playwright MCP so that the extractor remains reusable alongside other browser-interaction tools.

`semantic_snapshot` observes the whole page or a subtree of it, and `semantic_observe` resolves exactly one hinted element. Listing 4.1 gives the signatures of the two tools.

Listing 4.1: Signatures of the semantic extractor tools.

```
semantic_snapshot({
  url?: string,           // navigate here first
  scope?: string,         // data-semtag-id or CSS selector of a subtree
  includeHidden?: boolean // default: false
});

semantic_observe({
  id: string              // a data-semtag-id
});
```

A call to `semantic_snapshot` proceeds in five steps. The server attaches to the browser session and obtains the current page, navigating first if a `url` was given. It finds every element carrying a `data-semtag-id`, within the `scope` subtree if one was specified, and excluding hidden elements unless asked otherwise. For each, it reads the semantic attributes (`role`, `action`, `state`, `target`, `controls`, `options`) alongside the live states and values that the hints do not carry (accessible name, current value, visibility, enabled state). It does not read the native constraints a browser attaches to form controls, such as required, input type, min/max or the current validation message. It then groups the elements by their exact role, placing any with a missing or unrecognised role under `other`. Finally, it parses select options and folds repeated collections, and returns the semantic snapshot as JSON. `semantic_observe` performs the same read for a single element and returns its current state, reporting an error if zero or several elements match the id.

Every id in the snapshot is directly usable as a Playwright selector, `[data-semtag-id="..."]`, which is how the two servers connect: the agent observes through the extractor and acts through Playwright MCP using the identifiers the extractor returned. For that to work, both servers must interact with the same browser. They therefore attach to a single Chromium instance over the Chrome DevTools Protocol: Playwright MCP owns the browser, and the extractor attaches to it.

4.3 Adapting the testing agent

The testing agent we used is WebTester, the harness proposed alongside the WebTestBench dataset [34]. It runs as a Claude Code agent: the model, the agent loop, tool use, memory and context management are provided by the Claude Agent SDK [40], and the harness supplies the prompt, the tool set and the application under test. This is a change in what building an ATA involves. Chevrot et al.’s PinATA, from 2025, had to build these parts itself, with specialised sub-agents for navigation and assertion [31]; a year later, building on a general coding agent reduced the work to configuring it.

WebTester, originally, has two stages. The first, `checklist_generation`, generates a checklist of test cases for an application from its description; the second, `defect_detection`, takes an application and a checklist, drives the application in a browser through Playwright MCP, and returns a verdict for each test case. Since this work fixes test cases in advance, the first stage was archived, and only the second was reused. The following modifications were made to the `defect_detection` agent.

First, the agent’s prompt was restructured into a single template shared by both arms, with arm-specific details supplied through variables, keeping the instructions otherwise identical. The one substantive difference is a section, present only in the hinted arm, that teaches the agent to observe through `semantic_snapshot` and `semantic_observe`, and to act through Playwright MCP. The hinted agent is instructed to prefer semantic observation and to fall back on Playwright’s `browser_snapshot` tool only when the semantic snapshot does not contain what it needs. The

baseline agent, with the extractor absent from its tool set, observes through `browser_snapshot` alone. The full prompt used in both arms is shown in Appendix D.

Second, while a WebTester agent delivers all its verdicts at once at the end of a run, results are here saved progressively through a tool. In early runs with the published harness, we observed that some agents stopped before writing their results, or wrote them in a format the scorer could not parse, so verdicts were lost. A result-saving tool was added that the agent calls after each test case, recording the verdict (“PASS” or “FAIL”) and a rationale. The harness collects these into a `result.md` file per run in a fixed format. This removed a source of variance that had little to do with the agent’s testing ability, and relieved the agent of holding earlier verdicts in its context until the end of a run.

Third, shell access was removed. Our agent trace reviews showed that many agents, confused by the length of the task or by automatic compaction of their context, were using shell commands to re-read their own earlier reasoning and instructions from the session log – even though they had been explicitly asked not to use shell commands in their instructions. Blocking the shell entirely in the harness meant the agent could neither read the application’s source nor act on it outside the browser, keeping the testing truly “black-box”.

The two arms are therefore identical in model, prompt template, tool restrictions, turn limit, result format and scoring, and differ in one respect: the hinted agent has access to semantic snapshots, and any difference in outcome is attributable to that.

4.4 Run harness and instrumentation

All experiments ran on a single AWS EC2 instance (m7i.large, 2 vCPUs, 8 GB memory, Ubuntu 26.04, us-east-1), running Claude Code 2.1.227, Playwright MCP 1.63.0 on Chromium 149.0.7827.22 and Node.js v22.22.1. Runs lasted hours and had to continue unattended, and a dedicated remote machine removed confounds a laptop would have introduced, such as competing local processes and variable network conditions. The agent was powered through a Claude Code subscription rather than a per-token API key, reducing the cost of experimentation substantially. Cost figures reported in our evaluation are therefore computed, not billed, so that the results reflect what a run would cost at API rates regardless of how it was authenticated.

Runs are launched by a runner script we wrote, configured through environment variables: the model, the condition (baseline or hints), the list of applications, the number of repetitions, the turn limit, and a run identifier under which results are stored.

For each repetition, the script serves a fresh instance of the application on its own port with its seeded data reset, launches a new browser and agent session, and runs the defect-detection agent to completion. Repetitions are therefore fully independent. Because each invocation is self-contained and binds its own port range, several could run concurrently in separate terminal sessions on the same instance.

The Claude Code SDK reports usage per turn, including input tokens split into uncached input, cache reads and cache creation, output tokens, and wall-clock time per run. The division of wall clock time into model generation time and tool execution time is derived from run log timestamps. Tool-call counts by tool, including how often the hinted agent fell back on `browser_snapshot`, also come from the same record. Defect-detection precision, recall and F1 come from WebTester’s scorer.

Each repetition writes its artefacts to its own directory: the `result.md` of verdicts and rationales, the `score.json` from the scorer, the harness’s own run log, and the session log Claude Code writes to the machine’s `.claude` directory. An aggregator script we wrote then parses these directories across all repetitions, applications, models and conditions, and produces a CSV of raw scores and usage figures, per-application and overall means with 95% confidence intervals, and p-values for the baseline-hinted comparison, together with a summary sheet across applications.

4.5 Verification tooling

Two tools were built for human review. The first lets a reviewer confirm that the hints on a running application are correct; the second lets a reviewer inspect the testing agent’s behaviour during a run.

The Hint Overlay tool is a script that, once pasted into the browser console of the running application, draws a floating inspector over the page (Figure 4.2). It outlines every hinted element in the page, coloured by role, with its identifier and, for value-holding roles, its captured state; it highlights elements that look interactive but carry no hint; and it lists warnings such as duplicate identifiers, missing roles, and collections or selects whose annotation is incomplete. The tool also provides a view of the exact semantic snapshot the testing agent would see for the current page (Figure 4.3). This tool allows for checks that the static validator cannot perform: the validator confirms that hints are well-formed, and the overlay lets a human confirm that they are correct. Every hinted application in the evaluation dataset was reviewed this way after injection.

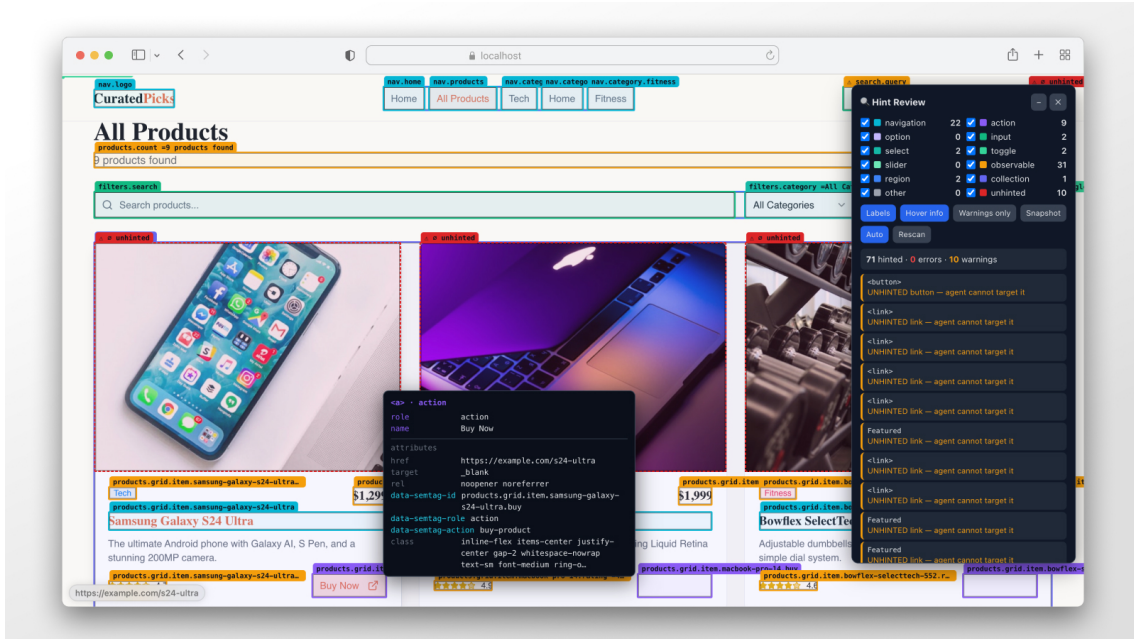


Figure 4.2: The Hint Overlay tool showing annotated and unhinted elements on a running application.

The Trace Viewer tool turns the artefacts of a completed run into a static, browsable interface (Figure 4.4). Each page combines the agent’s reasoning, tool calls and results in sequence, alongside the checklist with the expected and reported verdict for each test case, and the run’s metrics: precision, recall, F1, turns, tokens, wall-clock time and cost. False negatives (missed defects) and false positives (false alarms) are highlighted, so that the turns that produced each one can be traced. The viewer allowed us to find multiple harness issues such as early stopping of the agent, malformed output, and the use of shell commands.

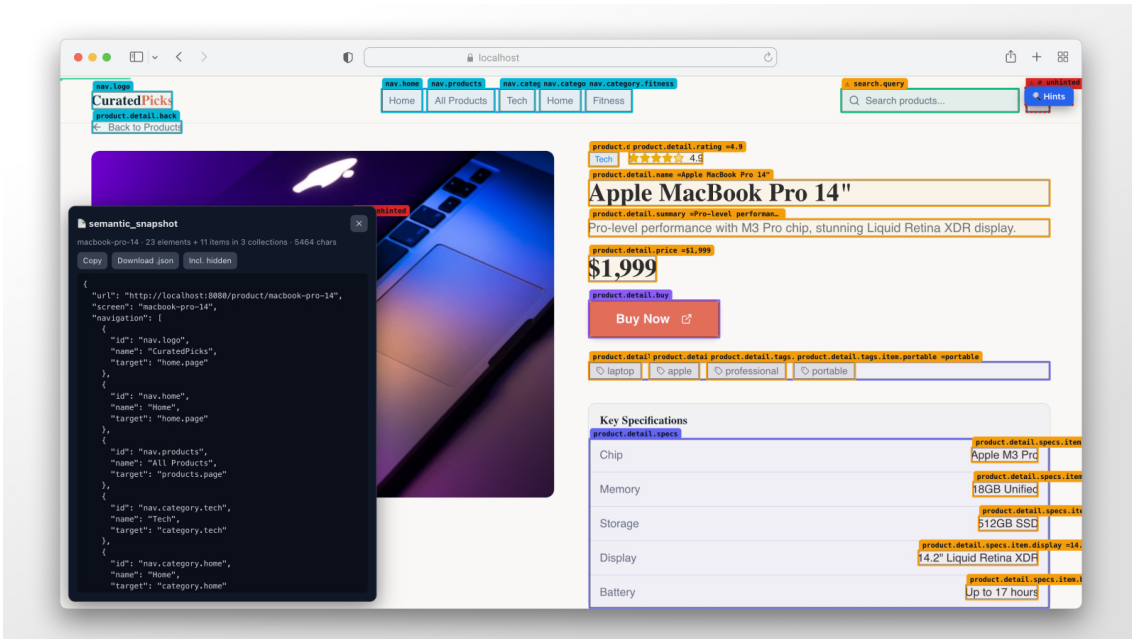


Figure 4.3: The semantic snapshot view in the Hint Overlay tool.

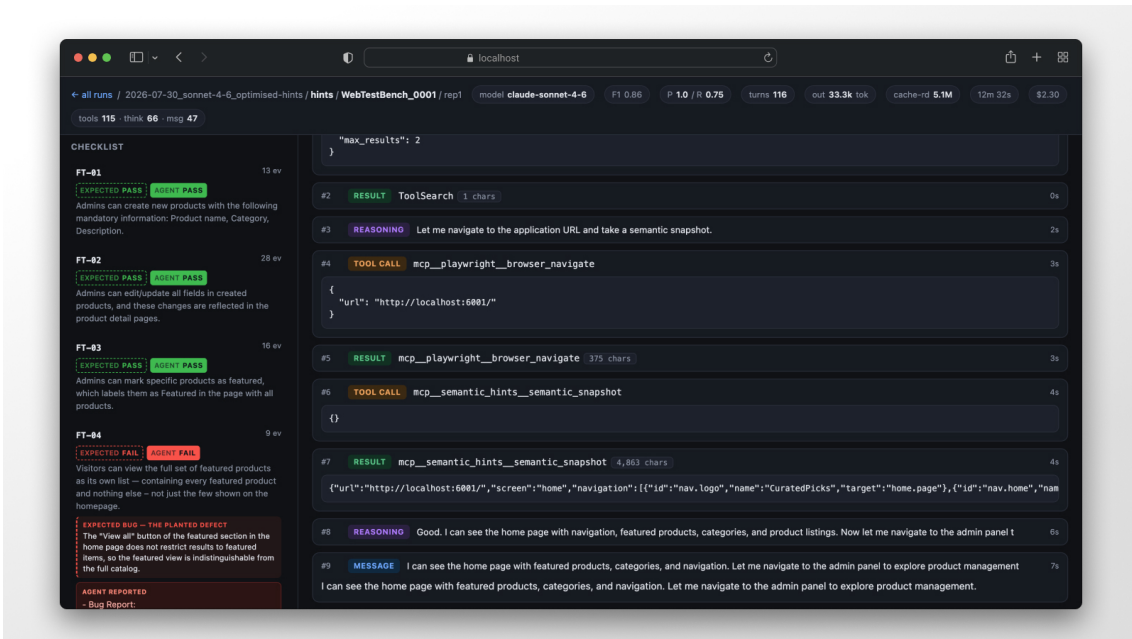


Figure 4.4: The Trace Viewer interface for a completed test run.

Chapter 5

Experimental Methodology

The previous chapter described the implemented Sntag system: how hints are injected, how snapshots are extracted, and how the testing agent was adapted to consume them. This chapter sets out how that implementation was evaluated against a baseline. Section 5.1 explains why WebTestBench was chosen as the benchmark over the alternatives. Section 5.2 describes the dataset drawn from it, the selection of fourteen applications and the correction of their test cases, and the measures taken to keep that correction from biasing the results. Section 5.3 defines the two arms of the experiment, the models, the number of repetitions and the conditions under which the 840 runs were executed. Section 5.4 defines the metrics on which the arms are compared and the statistical treatment applied to them. These choices allow the results in Chapter 6 to be attributed to the observation format alone.

5.1 Benchmark choice

Since our experimental comparison isolates the observation format the agent receives, the harness, test cases and oracles all had to be held constant across the two arms. A benchmark that left the agent to derive its own oracles would have reintroduced a source of variance in results. We also sought a benchmark whose test cases describe behaviour at the level a user would state it (e.g. “Admins can delete job postings from the job board”), rather than as a sequence of steps tied to the implementation (e.g. “Click the admin panel button in the navigation bar”, then “Click the Edit button on the top item”, and so on). As argued in the introduction, test cases bound to an implementation inherit its churn and become expensive to maintain. Beyond that, we also wanted to assess the effect of hints in a context where most of the development process is driven by AI, on the assumption that a coding agent could delegate verification to an ATA without human involvement.

WebTestBench [34] was a close match to these criteria. Its dataset comprises 100 front-end applications generated in a few prompts on Lovable, a platform on which users can build complete applications with LLMs without touching code; their defects arose during code generation, not injected afterwards. Its checklists state test cases at the level of behaviour, and it separates the generation of checklists from their execution, so that the execution stage could be reused with its oracles fixed from the provided “gold checklist”. Its authors frame the benchmark around a setting in which tests are not maintained by hand, which is the setting this work addresses. Its accompanying agent, WebTester, supplied the original harness for this work. As Section 5.2 discusses, however, its test cases contained flaws that had to be corrected before the dataset could be used.

WebTestPilot [33] was also considered, but rejected as it is built around deriving oracles, symbols and assertions from natural-language requirements. Its four real, open-source applications are also further from the generated-code setting than WebTestBench’s.

5.2 Dataset

WebTestBench groups its 100 applications into seven categories: Presentation, Search, Tool, Commerce, Data Management, Workflow and User-Generated Content [34]. The final dataset comprises

14 of them, two per category, so that no category dominates the aggregate results and each is represented well enough for the per-category comparison in Chapter 6.

Four of the fourteen were development applications, used throughout the project to fix harness issues and build the tooling. Neither the agent prompt nor the test cases were tuned to them. The dataset’s applications numbered 1–5 had originally been chosen for convenience, but one was dropped when it emerged that its seeded data was date-dependent and did not appear on the interface as a result. The remaining ten were drawn by rejection sampling: a number from 1 to 100 was picked at random, and the corresponding application was accepted if its category still had an open slot and rejected otherwise, until every category held two applications.

Table 5.1 lists the fourteen applications with their identifier, name, category, a one-line description and the number of pages, counted as distinct routes in the application’s router, with a parameterised route such as `/products/:id` counted once. It also gives the numbers of test cases and of failing test cases after the corrections described below.

Table 5.1: Applications included in the experimental dataset. Test-case and expected-failure counts reflect the corrected checklists.

Number	Name	Category	Description	Pages	Test cases	Expected fail-ures
0001	CuratedPicks	Search	Product recommendation site with reviews and affiliate links	5	22	6
0003	JobBoard	Search	Job listings board with applications	4	20	5
0004	Ever After	Workflow	Wedding planning dashboard with vendor listings	5	19	5
0005	ResumeBuilder	Tool	Template-based resume editor	1	21	5
0006	LedgerFlow	Data Management	Small-business bookkeeping and reports	5	24	4
0010	TableSpot	Commerce	Restaurant table reservations	3	17	5
0019	ThreadHive	User-Generated Content	Community discussion forum	7	22	7
0023	ReVive	Commerce	Second-hand/upcycled marketplace	9	23	6
0045	My Tasks	Workflow	To-do list with task dependencies	1	22	3
0054	TechSpecs	Presentation	Tech docs knowledge base with knowledge graph	1	18	2
0063	Chinese Heritage	Presentation	360° panorama cultural-heritage browser	2	14	4
0074	FlexBase	Data Management	Flexible database, table/card views	1	20	6
0089	Template Library	User-Generated Content	Fill-in text template library	1	29	5
0097	CompetencyHub	Tool	HR competency graph modeling and simulation	1	20	8

The applications themselves were used exactly as published. However, we made important corrections to the “gold” test cases provided by the benchmark. Early experiments on the development applications showed that agents were performing poorly not because they could not test, but because the checklist items could not be tested correctly: many were broad or ambiguous, some referred to features or domains the application did not have, and some carried the wrong verdict. Since the evaluation attributes every difference between the arms to the observation format, test cases that could not be resolved would have added variance that had nothing to do with the representation, and wrong verdicts would have made the absolute efficacy figures meaningless.

We therefore rewrote the test cases for all fourteen applications in a single pass, against the following rules. Every application has at least 13 test cases, at least two of which fail; all four WebTestBench classes are represented in eleven of the fourteen applications (0019 has no interaction case, 0054 and 0074 no content case). Every test case pertains to a feature the application actually has, states one oracle only, and is unambiguous about the expected behaviour. Every verdict, including failing ones, was established by running the application and checking the behaviour by

hand, with WebTestBench’s original checklists serving as a starting point whose items and verdicts were each verified. Every test case is also written at the high level of expected behaviour, and does not describe a step-by-step sequence that the agent must follow.

Measures were taken to keep this correction from contaminating the results. Agent output from previous runs was never used to author or adjust a test case: ground truth came from manual verification of the running application alone. Additionally, every edit was recorded in a change log with the original item and the reason for the change, whether ambiguity, irrelevance to the application, an incorrect verdict or a compound oracle. The complete log of corrections and the corrected checklists are given in Appendices E and F.

As a consistency check, the baseline agent on Claude Sonnet 5 was run ten times per application on the original checklists and ten times on the corrected ones. Overall precision rose from 59.2% to 87.5%, recall from 44.6% to 78.9% and F1 from 47.7% to 81.1%. This is not a validation of the corrections, but it confirms that the corrected checklists behave as they should: an agent that reads the interface correctly now agrees with the ground truth far more often. The full comparison is given in Appendix G.

5.3 Experimental design

Hints were injected into a copy of each of the fourteen applications using the Hint Injector skill (Section 4.1), with Claude Fable 5 as the coding agent, chosen as the most capable model available so that the quality of the injection would not confound the comparison. Injection ran unattended, and by the end of each run the validator reported no errors. Each application was then reviewed in the Hint Overlay tool (Section 4.5); no hinted element was found to be misannotated. The time and tokens consumed by each injection were recorded, and are reported in Chapter 6 as the fixed cost against which the per-run savings are amortised.

The experiment compares two arms. The baseline agent runs against the unmodified application as published by WebTestBench and observes it through Playwright’s accessibility snapshot. The hinted agent runs against the annotated copy and observes it primarily through the semantic snapshots, falling back on the accessibility snapshot where those do not suffice. Since hints are non-rendering attributes, the two arms exercise behaviourally identical applications.

Both arms were run on the latest three models spanning Anthropic’s current range (as of mid-August 2026), excluding Claude Fable 5: Claude Haiku 4.5, Claude Sonnet 5 and Claude Opus 5¹. Reasoning effort was set to “high” throughout for consistency. Each combination of application, arm and model was repeated ten times, giving 280 runs per model and 840 runs in total, enough that per-application means are not dominated by the run-to-run variance that Salva [39] and Chevrot et al. [31] both report on agent-driven test execution.

A run is complete when it returns a verdict for every test case in the checklist within the turn limit. That limit was raised from WebTester’s default of 150 turns to 400 after early runs showed that some applications could not be fully covered within it. Runs that reached the limit without completing the checklist were discarded and repeated until ten complete runs existed for each application, arm and model. All runs were executed between August 24th and August 28th 2026. Baseline and hinted batches for each model were launched concurrently in separate sessions on the same instance, so that both arms experienced the same load and API latency throughout.

5.4 Metrics and analysis

Defect-detection efficacy is measured as a classification problem over test cases. A test case whose ground-truth verdict is FAIL is a defect; a FAIL verdict from the agent is a detection. A true positive is therefore a FAIL verdict on a failing case, a false positive a FAIL verdict on a passing case, and a false negative a PASS verdict on a failing case. Precision is the share of reported defects that are real, recall the share of real defects that were reported, and F1 their harmonic mean. All three come from WebTester’s scorer [34], computed per run and then averaged over the ten runs of each application. Recall is arguably the more important of the two in this setting: a false alarm costs a developer a few minutes of review, whereas a missed defect reaches users. Precision and recall are therefore reported separately.

¹The models were accessed through the Claude Agent SDK under the identifiers `claude-haiku-4-5`, `claude-sonnet-5` and `claude-opus-5`.

Efficiency is measured in tokens, cost and time. The Claude Code SDK reports usage per turn, split into uncached input tokens, cache-creation tokens, cache-read tokens and output tokens, and these are summed over a run. Since the four are priced differently, they are combined into a single figure of *effective tokens*, weighting each by its price relative to a base input token:

$$\text{Effective tokens} = 1 \times \text{input} + 2 \times \text{cache creation} + 0.1 \times \text{cache read} + 5 \times \text{output}.$$

“Effective input tokens” measures the weighted sum of the first three kinds, excluding output tokens. The multipliers follow Anthropic’s published pricing as of mid-August 2026 [41], under which one-hour cache writes cost twice the base input rate, cache reads a tenth of it, and output tokens five times it; effective tokens are thus proportional to cost, and comparable across runs and models regardless of how the tokens were split. Cost in dollars is obtained by multiplying effective tokens by the model’s base input price per million tokens, using \$1 for Haiku 4.5, \$2 for Sonnet 5 and \$5 for Opus 5 [41]. Because the agent was authenticated through a subscription rather than an API key, these figures are computed and represent what a run would cost at API rates.

Latency is the wall-clock time of a run from launch to the final verdict, decomposed into model generation time and tool execution time from the timestamps the harness records around every tool call. The harness’s logs also yield counts per tool for each run, from which the hinted agent’s rate of fallback on the accessibility snapshot is derived.

The unit of analysis throughout is the application. For every metric, the ten runs of an application, arm and model are averaged into a per-application mean. The effect of hints on that application is the difference between its hinted and baseline means for the efficacy metrics, or the relative change for tokens, latency and turns.

The overall effect reported for a model is the mean of its fourteen per-application effects, with a 95% confidence interval across applications, and its significance is assessed with a paired t-test across the fourteen applications; a Wilcoxon signed-rank test on the same fourteen effects is reported in Appendix H for every comparison in Table 6.1, as a check on the normality assumption at this sample size. Because this averages relative changes with equal weight per application, it differs from the change in total token consumption across the dataset, which weights applications by their size. While the pooled mean is the primary figure throughout, the mean per-application change is reported with the test statistics in Appendix H.

Per-application differences are tested with a two-tailed Welch’s t-test on the ten runs of each arm. With seven metrics over fourteen applications, some of these will fall below 0.05 by chance, so per-application significance should be read as indicative. A non-significant overall difference is not taken as evidence that the arms are equivalent: for efficacy in particular, where the claim of interest is that hints do not degrade detection, the confidence interval on the difference is reported so that the reader can judge how large a degradation the data rule out. Comparisons across models are descriptive; each model’s effect is reported with its own interval, and no test of the difference between models is made. Quantitative results are complemented by review of individual runs in the Trace Viewer (Section 4.5), which is the source of the qualitative findings on which test cases the hinted agent got wrong, when it fell back on the accessibility snapshot, and where runs of either arm lost their way.

Chapter 6

Results

6.1 Overview

Table 6.1 summarises the 840 runs by model and arm. Every cell is a mean over the fourteen applications, each of which contributed ten complete runs per arm; the Δ rows give the pooled change from baseline to hinted, computed from the two cells above them, with an asterisk where a two-tailed paired t-test on the per-application means rejects the null hypothesis of no difference at $p < 0.05$. A Wilcoxon signed-rank test on the same per-application effects, reported in Appendix H, agrees with the t-test on every comparison but two borderline ones, neither of which affects the conclusions below. Efficacy differences are in absolute points; token, latency and turn differences are relative; cost is in dollars.

Table 6.1: Mean results by model and experimental arm.

Model	Arm	Precision	Recall	F1	Effective tokens	Effective input	Output tokens	Latency (s)	Cost (\$)	Turns
Haiku 4.5	Baseline	0.62	0.42	0.47	1,065,062	993,541	23,840	370	1.07	114
	Hinted	0.75	0.51	0.57	1,083,481	992,790	30,230	483	1.08	147
	Δ	+0.13	+0.09	+0.10	+1.73%	-0.08%	+26.80%*	+30.38%*	+0.02	+29.59%*
Sonnet 5	Baseline	0.88	0.79	0.81	1,782,586	1,685,531	32,351	563	3.57	172
	Hinted	0.90	0.82	0.84	1,499,492	1,393,681	35,271	598	3.00	180
	Δ	+0.03	+0.03	+0.02	-15.88%*	-17.32%*	+9.02%*	+6.19%	-0.57*	+4.45%
Opus 5	Baseline	0.94	0.89	0.90	1,213,205	1,116,518	32,229	612	6.07	161
	Hinted	0.93	0.91	0.91	1,060,478	961,848	32,877	599	5.30	156
	Δ	-0.01	+0.02*	+0.01	-12.59%*	-13.85%*	+2.01%	-2.22%	-0.76*	-2.93%

Note: An asterisk marks comparisons where a two-tailed paired t-test on the fourteen per-application means rejects the null hypothesis at $p < 0.05$.

On Sonnet 5 and Opus 5, the semantic representation preserved defect-detection efficacy while lowering the cost of each run. Precision and F1 moved by three points or fewer, none of it significant; the only significant efficacy difference on either model is a small rise in recall on Opus (+0.02). Effective token use fell by 15.9% on Sonnet and 12.6% on Opus, corresponding to savings of \$0.57 and \$0.76 per run at API rates. The saving came entirely from input tokens (-17.3% and -13.9%); output tokens rose by 9% on Sonnet and were unchanged on Opus. Wall-clock time did not differ significantly on either model.

On Haiku 4.5, the representation had no significant effect on cost, and appeared instead to change what the agent did. Effective token use was almost unchanged (+1.7%), because a small reduction in input was offset by a 27% increase in output. That increase accompanied 30% more turns per run and 30% longer runs, and a rise of ten points in F1, from 0.47 to 0.57, with precision and recall both higher. None of the efficacy differences on Haiku reaches significance at the 5% level across fourteen applications, but they point in the same direction.

Across both arms, the three models separate clearly on efficacy: F1 rises from 0.47 on Haiku, to 0.81 on Sonnet, to 0.90 on Opus in the baseline arm, and hinting the smallest model does not close the gap to the next one.

The rest of this chapter takes the research questions in turn. Section 6.2 examines the efficacy result per application and per test-case class (RQ1). Section 6.3 examines where the token savings came from (RQ2). Section 6.4 examines how the effect varied across models and applications (RQ3), including the Haiku behaviour and the hinted agent’s use of the accessibility-snapshot

fallback. Section 6.5 sets the per-run savings against the cost of injecting the hints.

6.2 Efficacy (RQ1)

RQ1 asks whether an agent observing through semantic snapshots detects defects as well as one observing through accessibility snapshots. The claim of interest is one of no degradation, so this section reports the confidence interval on each difference alongside its significance.

6.2.1 Overall differences

Figure 6.1 plots the difference in precision, recall and F1 between the hinted and baseline arms for each model, with 95% confidence intervals across the fourteen applications. On Sonnet 5 the differences are +0.03 (precision), +0.03 (recall) and +0.02 (F1), with intervals of ± 0.06 , ± 0.04 and ± 0.05 ; none is significant, and the lower bounds place any degradation at no more than 0.03 in precision and F1 and 0.01 in recall. On Opus 5 the intervals are narrower (± 0.02 on all three metrics); precision and F1 are unchanged (-0.01 and $+0.01$) and recall rises by 0.02, the one efficacy difference on either model that reaches significance ($p = 0.03$). Its lower bound is zero. For both models, then, the data rule out any loss of defect-detection ability larger than three points. On the metric this work treats as more important, recall, they rule out a loss on Opus and larger than one point on Sonnet.

On Haiku 4.5, the picture is different in kind. All three metrics rise, by +0.13 (precision), +0.09 (recall) and +0.10 (F1), but with intervals of ± 0.14 , ± 0.12 and ± 0.12 none reaches significance at the 5% level ($p = 0.06$, 0.12 and 0.10). Under the Wilcoxon signed-rank test, precision reaches $p = 0.04$ while recall and F1 do not (Appendix H); the result is best read as borderline under both tests. The intervals exclude any material degradation and are consistent with an improvement of up to twenty points. The size of the intervals reflects the variance of the underlying runs: Haiku’s per-application F1 in the baseline arm ranges from 0.00 to 0.91, and its run-to-run variation within an application is several times that of the larger models.

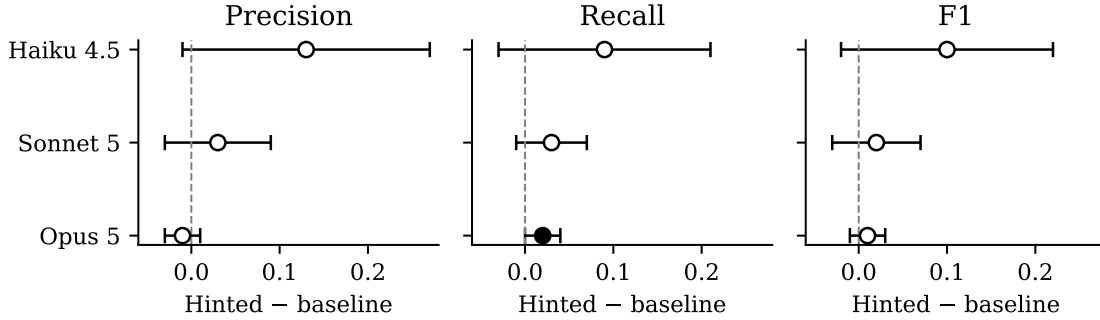


Figure 6.1: Difference in precision, recall and F1 between the hinted and baseline arms for each model. Error bars show 95% confidence intervals across the fourteen applications.

6.2.2 Per-application differences

Figure 6.2 shows F1 per application in each arm. On Sonnet and Opus, the two arms track each other closely: the largest per-application F1 difference on Opus is 0.07 (application 0063), and on Sonnet the differences exceed 0.10 in only two applications (0001 and 0063), both in the hinted arm’s favour. Tested individually, seven of the 42 per-application comparisons on Sonnet and three on Opus are significant at $p < 0.05$. With this many tests, a few significant results are expected by chance alone, and the significant differences do not recur across models, so we read these as scatter rather than application-specific effects. On Haiku, eight comparisons are significant, and they are concentrated in three applications (0006, 0023 and 0097) where the baseline agent’s F1 was 0.07, 0.45 and 0.12 and the hinted agent’s was 0.78, 0.60 and 0.38—so the gains on this model are large where they occur.

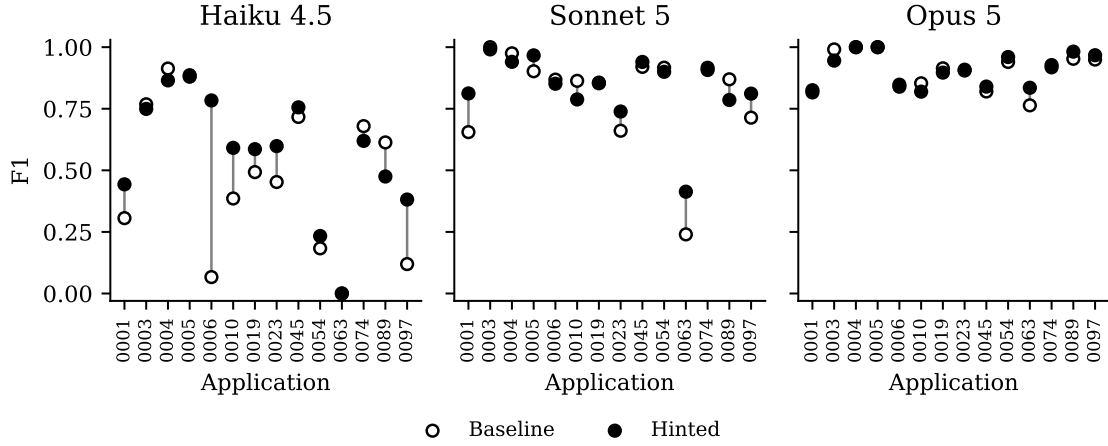


Figure 6.2: F1 by application, model and experimental arm.

6.2.3 By test-case class

Table 6.2 breaks the efficacy results down by WebTestBench’s four classes of test cases (Functionality, Constraint, Interaction, Content), pooling test cases across applications and pairing baseline and hinted runs by model and repetition. A repeated-measures ANOVA finds no evidence that the effect of hints on F1 differs by class overall ($p = 0.24$), nor on Sonnet ($p = 0.13$) or Opus ($p = 1.00$). The one exception is on Haiku, where the class effect is borderline ($p = 0.05$) and the pattern is clear: hints improved F1 on functionality (+0.10), constraint (+0.09) and content (+0.14) test cases, and did not improve it on interaction test cases (−0.06). Interaction test cases are also the hardest class for every model in both arms, with a mean F1 of 0.64 against 0.71–0.84 for the other three. These are the test cases that exercise multi-step dynamics, such as drag-and-drop, modal dialogs and hover states.

Table 6.2: Defect-detection F1 by test-case class, model and arm.

Class	Defects	Haiku 4.5			Sonnet 5			Opus 5		
		Base	Hint	Δ	Base	Hint	Δ	Base	Hint	Δ
Functionality	26	0.52	0.62	+0.10*	0.81	0.86	+0.05	0.90	0.90	+0.00
Constraint	25	0.68	0.77	+0.09*	0.89	0.87	−0.02	0.93	0.93	+0.00
Interaction	11	0.39	0.34	−0.06	0.67	0.75	+0.09	0.85	0.85	+0.00
Content	9	0.25	0.39	+0.14*	0.80	0.86	+0.06	0.98	0.99	+0.01

Note: Mean F1 per class, pooling test cases across the fourteen applications; baseline and hinted runs are paired by repetition (ten pairs per model). Defects is the number of failing test cases in the class. * $p < 0.05$, paired t-test, unadjusted.

6.2.4 Fallbacks on accessibility trees

The efficacy results above can be attributed to the semantic representation only to the extent that the hinted agent actually observed through it. The hinted prompt permitted a fallback to Playwright’s accessibility snapshot where the semantic snapshot did not suffice, and Table 6.3 reports how often that occurred. Opus fell back on 5.4% of its observation calls (1.1 accessibility snapshots per run), Haiku on 12.8% (6.7 per run) and Sonnet on 24.3% (13.0 per run). For Opus, then, the verdicts were almost entirely derived from the semantic representation. For Sonnet, a quarter of observations went through the accessibility tree, and the efficacy result for that model is correspondingly a result about an agent that observed mainly, not exclusively, through hints. The rate varied more with the model than with the application: in 0003 and 0010, where Sonnet fell back on 44% of its observations, Opus fell back on 3% and 5%.

Review of the Sonnet logs shows that the semantic snapshot was valid and well-formed in almost every case, but lacked the detail the test oracle required. Most fallbacks either followed an action to check a result such as a toast, validation message or dialog that the semantic snapshot did not carry, or served to obtain a reference to a control that the hints did not name. A smaller number followed near-empty snapshots on screens the injector had not annotated, or sought content the hints do not represent, such as images, full text and error-page copy. That Opus fell back far less on the same applications indicates that the missing detail was the same for both models, but that Opus was more often able to judge without it.

Table 6.3: Observation calls per run in each arm, and the hinted agent’s use of the accessibility-snapshot fallback.

Model	Observation calls per run		Fallbacks (hinted arm)	
	Baseline	Hinted	Per run	Rate (%)
Haiku 4.5	39.2	52.1	6.7	12.8
Sonnet 5	52.9	53.6	13.0	24.3
Opus 5	14.6	21.1	1.1	5.4

Note: Means over fourteen applications $\times$ ten runs. In the baseline arm an observation call is `browser_snapshot`; in the hinted arm it is `semantic_snapshot`, `semantic_observe` or `browser_snapshot`. A fallback is a hinted-arm call to `browser_snapshot`, the accessibility snapshot; the rate is fallbacks as a share of the hinted arm’s observation calls.

6.3 Efficiency (RQ2)

RQ2 asks how the semantic representation affects the cost of a run: its token use, dollar cost and latency. In this section, we also examine the mechanism: the reduction in observation size that the semantic snapshot achieves, and why the saving at the level of a run is smaller than the saving at the level of an observation.

6.3.1 Tokens and cost

Effective token use fell by 15.9% on Sonnet 5 and 12.6% on Opus 5 (both $p < 0.001$, paired across applications), and did not change on Haiku 4.5 (+1.7%, $p = 0.68$). Figure 6.3 shows the change per application. On Sonnet every application used fewer tokens in the hinted arm, by between 2% (0023) and 44% (0054), and the difference is significant in eight of the fourteen; on Opus, thirteen of fourteen did, by between 2% and 22%, with one application (0074) unchanged, and seven differences are significant. On Haiku the changes are split evenly in sign, from -20% to $+20\%$, and none is significant. The saving on the two larger models is therefore general across applications, but its size varies by an order of magnitude between them.

The saving comes entirely from input tokens. Effective input tokens fell by 17.3% on Sonnet and 13.9% on Opus, while output tokens rose by 9.0% on Sonnet ($p = 0.02$) and were unchanged on Opus ($+2.0\%$, $p = 0.30$). Output tokens are generated at a smaller quantity than input tokens are ingested, but are the more expensive kind: once weighted by their relative price, they account for between a tenth and a sixth of a run’s effective-token total¹. On Haiku, input tokens were unchanged (-0.1%) and output rose by 27%.

Figure 6.4 breaks input down by cache status. Uncached input is negligible, so a run’s input cost is the cost of writing new content into the cache (cache creation) and of re-reading the accumulated context on every turn (cache read). Cache reads dominate, accounting for 72–81% of effective input-token cost after weighting. Dividing by turns gives the mean context the model re-reads each turn: 65k on Haiku, 71k tokens on Sonnet and 57k on Opus in the baseline arm. Hints reduced that per-turn context by 18%, 14% and 13% respectively.

At API rates, the token saving corresponds to \$0.57 per run on Sonnet, from \$3.57 to \$3.00, and \$0.76 on Opus, from \$6.07 to \$5.30. Opus saves the smaller share but the larger amount, because its base price is two and a half times Sonnet’s. Haiku’s cost was unchanged at \$1.07.

¹Computed from Table 6.1 as $5 \times \text{output tokens} \div \text{effective tokens}$. By model and arm: Haiku 4.5, 11% baseline and 14% hinted; Sonnet 5, 9% and 12%; Opus 5, 13% and 16%. The share rises in the hinted arm on every model, since output is unchanged or higher while input falls.

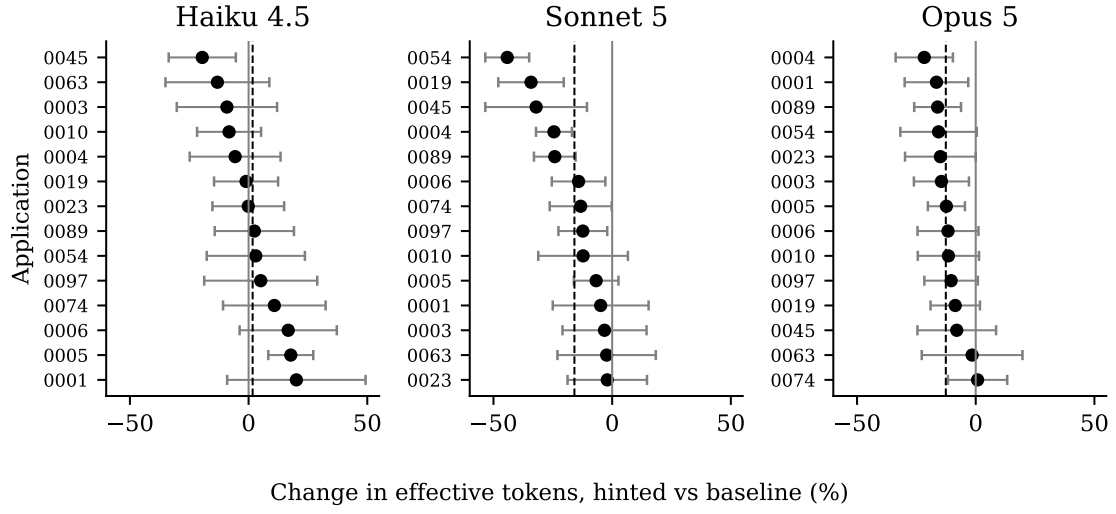


Figure 6.3: Relative change in effective-token use from the baseline to the hinted arm, by application and model.

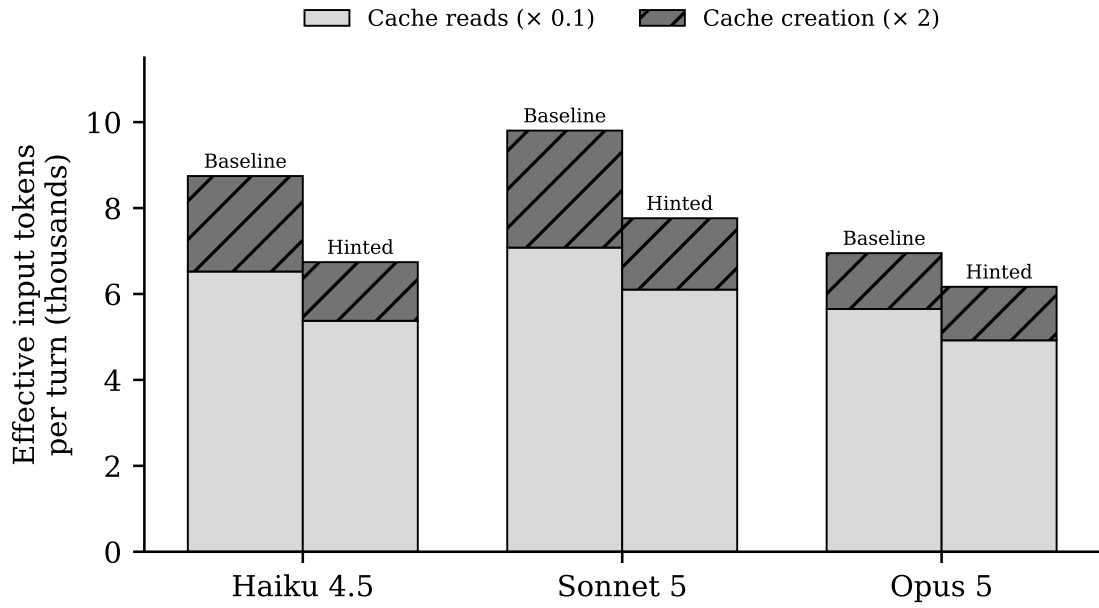


Figure 6.4: Effective input-token use by cache status, model and experimental arm.

6.3.2 Latency and turns

Wall-clock time per run did not differ significantly on Sonnet (+6.2%, $p = 0.12$) or Opus (−2.2%, $p = 0.28$)—so runs were cheaper but not slower. On Haiku, runs took 30% longer ($p < 0.001$), which again can be attributed to the higher turn count. Appendix I decomposes wall-clock time into model generation, tool execution and other. Model generation accounts for 70–77% of it on every model and arm, tool execution for 8–16%, and harness overhead for the remainder. Model time per turn is unaffected by hints (2.4s against 2.6s on Sonnet, 2.9s on Opus in both arms), so a smaller context does not make a turn faster, and latency is governed by the number of turns. Turns rose by 30% on Haiku ($p < 0.001$), did not change significantly on Sonnet (+4.5%, $p = 0.18$), and fell by 2.9% on Opus, a difference that is borderline under both tests ($p = 0.07$; Wilcoxon $p = 0.04$). On Haiku, output tokens, latency and turns all rose by roughly the same proportion.

6.3.3 Observation size

Figure 6.5 plots, for each application, the mean size of an accessibility snapshot in the baseline arm against the mean size of a semantic snapshot in the hinted arm. Across applications, the semantic snapshot is 69% smaller (range 46% to 79%). The proportion saved is not fixed: larger pages compress further. The accessibility snapshot ranges from 3.8k characters on application 0005 to 12.3k on 0097, whereas the semantic snapshot stays between 1.8k and 2.9k on every application, so the largest pages compress the most ($r = 0.85$ between accessibility-snapshot size and reduction). This is the behaviour the design intended. A semantic snapshot’s size is governed by the number of distinct things on the page, not by the markup that renders them, and collection folding means that a list of many items costs little more than a list of a few items.

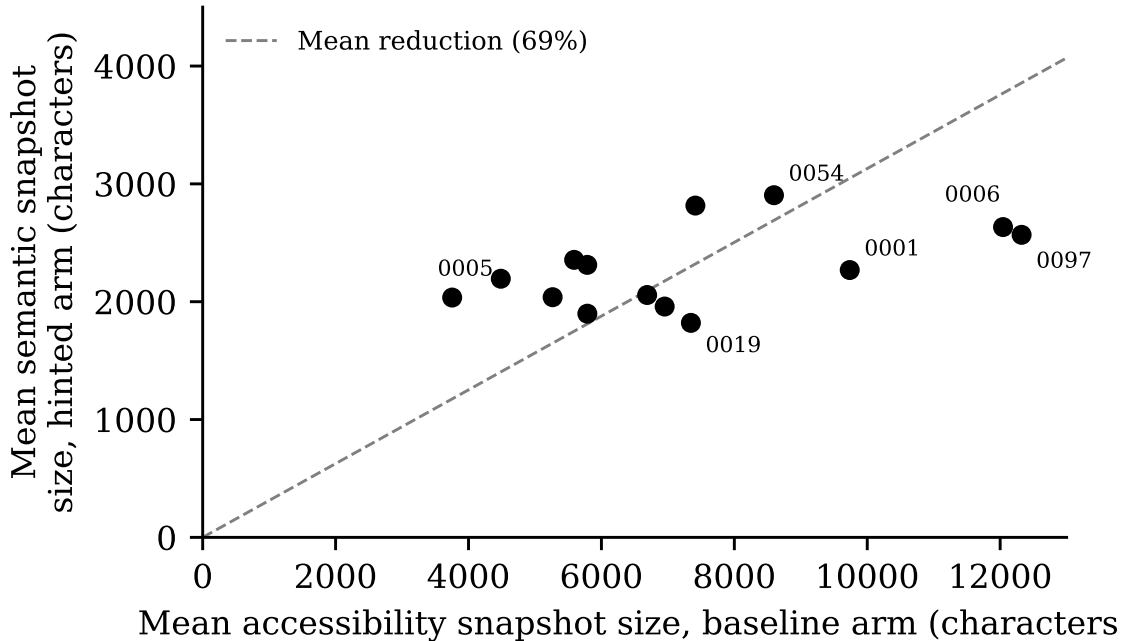


Figure 6.5: Mean accessibility- and semantic-snapshot sizes by application.

6.3.4 Effect of observation size reduction on savings

A 69% reduction in the size of an observation and a 17% reduction in input tokens per run are consistent only if observations are a minority of what the agent reads, or if the hinted agent spends part of the saving on accessibility-snapshot fallbacks and on additional turns. In our experiments, both of these were true. Observations are new cache content, and new content is the smaller part of a run’s total input cache: on Sonnet, 234k tokens are written into the cache per baseline run against 12.2 million re-read. The snapshot reduction lowers what is appended on each observing turn, and that reduction then propagates into every subsequent re-read, but is diluted by everything else the

context holds: the system prompt and tool definitions, the checklist, the results of certain actions and the agent’s own reasoning.

The saving shrinks at each of four levels between a single observation and the context the model reads on a turn. A semantic snapshot is 69% smaller than an accessibility snapshot of the same page. Over a whole Sonnet run, observation content fell by 62% rather than 69%, because the hinted agent fell back on the accessibility snapshot for a quarter of its observations, and those thirteen fallbacks supplied more than half of its observation content. The content a run appends to its context (cache creation) fell by 36%, from 234k to 149k tokens, because observations are only part of what is appended, alongside action results and the agent’s own reasoning; if the whole fall is attributed to observations, they were about 60% of what a baseline run appended. The context re-read on every turn (cache read) fell by 14%, because it also carries a fixed prefix of system prompt, tool definitions and checklist that the representation cannot touch. The 15.9% saving in effective tokens is what remains of the 69% after these three dilutions.

6.4 Generalisability (RQ3)

RQ3 asks whether the effects reported in the previous sections depend on the model or the application. This section examines each in turn: how the effect changes with model capability, what explains the wide variation in savings between applications, and whether application category matters.

6.4.1 Across models

The three models differ in efficacy far more than either representation changes them. In the baseline arm, F1 rises from 0.47 on Haiku to 0.81 on Sonnet and 0.90 on Opus. Figure 6.6 places the six model-arm combinations on those two axes, with an arrow from each model’s baseline to its hinted result. The arrows point in different directions. On Sonnet and Opus they point left: cost falls and F1 barely moves. On Haiku the arrow points up: cost is unchanged and F1 rises. The representation’s effect thus changes in kind, not merely in size, as capability falls.

The Haiku arrow does not reach the next model. Hinted Haiku achieves an F1 of 0.57 at \$1.08 per run, against 0.81 at \$3.57 for baseline Sonnet, so the representation does not let a cheaper model stand in for a more capable one. Nor does the gap close at the top: hinted Sonnet (0.84, \$3.00) remains below baseline Opus (0.90, \$6.07) on efficacy, though at half the cost. Where a testing budget permits it, the more capable model with hints remains the better tester; where it does not, hints make the cheaper model cheaper still, without making it as good.

The Haiku result is the one that the efficiency framing of this work did not anticipate. Section 6.3 established that hints reduced Haiku’s per-turn context by 18%, more than on either larger model, and that the saving was cancelled because hinted runs took 30% more turns, wrote 27% more output tokens, and lasted 30% longer.

Review of baseline Haiku runs’ logs found 30 in which the agent recorded verdicts for the remaining test cases without exercising them, typically after running into issues with testing (e.g. persistence failures, page loading issues or required form fields), and 20 such cases among hinted runs. The baseline Haiku agent, in other words, frequently abandoned the task part-way and reported the rest, which is cheap and produces verdicts that are wrong; the hinted agent more often continued to the end. This is consistent with the smaller per-turn context leaving the agent with more of its working state intact over a long run.

Opus observes far less often than the other two models: 14.6 observation calls per baseline run against 39.2 on Haiku and 52.9 on Sonnet, and hints increased that count on Opus by 45% (to 21.1) and on Haiku by 33% while leaving Sonnet’s unchanged (Table 6.3). A more capable model reads the page less and acts on what it has read. Given a cheaper observation, it reads more often, so part of the per-observation saving on Opus was spent on additional observations rather than returned as a smaller run.

6.4.2 Across applications

The token saving on Sonnet ranged from 2% to 44% across applications (Figure 6.3), and on Opus from 2% to 22%. We anticipated that application complexity might explain this, and established that the snapshot reduction grows with page size.

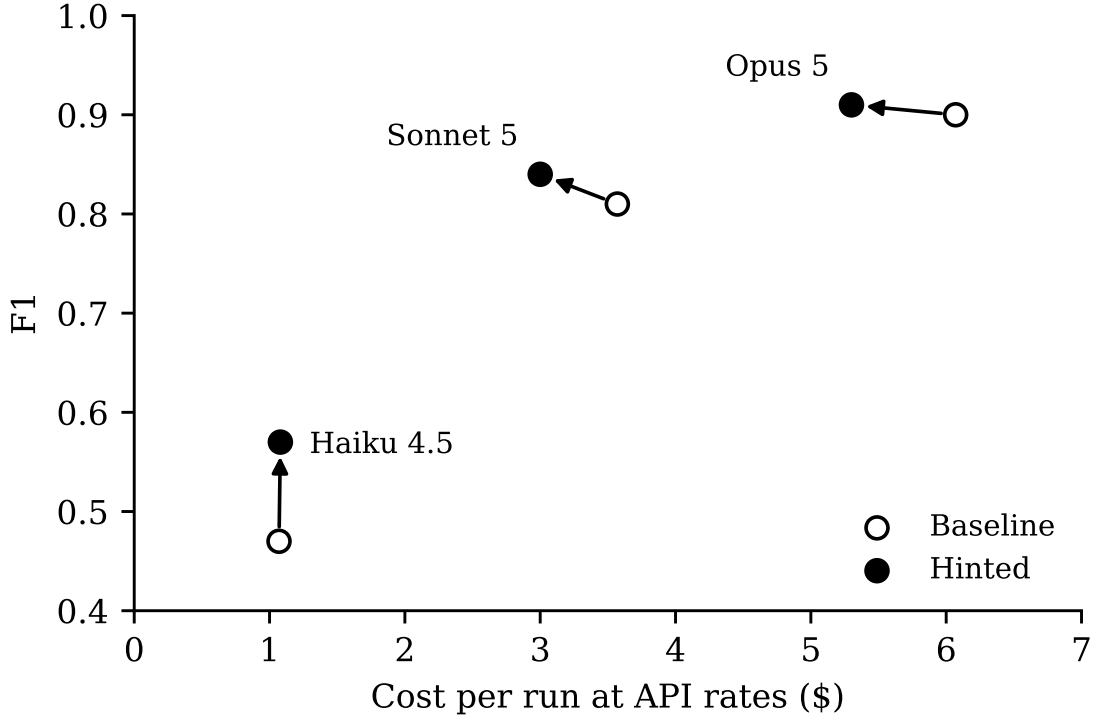


Figure 6.6: F1 against API-equivalent cost per run for each model and experimental arm.

However, across the fourteen applications, neither mean accessibility-snapshot size nor per-snapshot reduction correlates with the token saving on Sonnet ($r = 0.01$ and -0.03) or Opus (-0.06 and 0.06 ; Appendix H). This indicates that larger and more compressible pages did not produce larger savings.

A better predictor of token savings is how often the hinted agent fell back to the accessibility snapshot. Figure 6.7 plots per-application fallback rate against token saving. On Sonnet, the applications with the smallest savings are those where the agent fell back most. On 0003 and 0010, the agent fell back on 44% of its observations and saved 3% and 12%; on 0023, the fallback rate is 30% and the saving is 2%; on 0054 and 0004, where it fell back only around 10%, it saved 44% and 25%. Every fallback costs the full accessibility snapshot on top of the semantic one that preceded it, so a run that falls back on nearly half its observations is reading more than the baseline would have on those steps. On Sonnet the correlation is 0.50 ($p = 0.07$), short of significance across fourteen applications but the strongest of the candidate predictors.

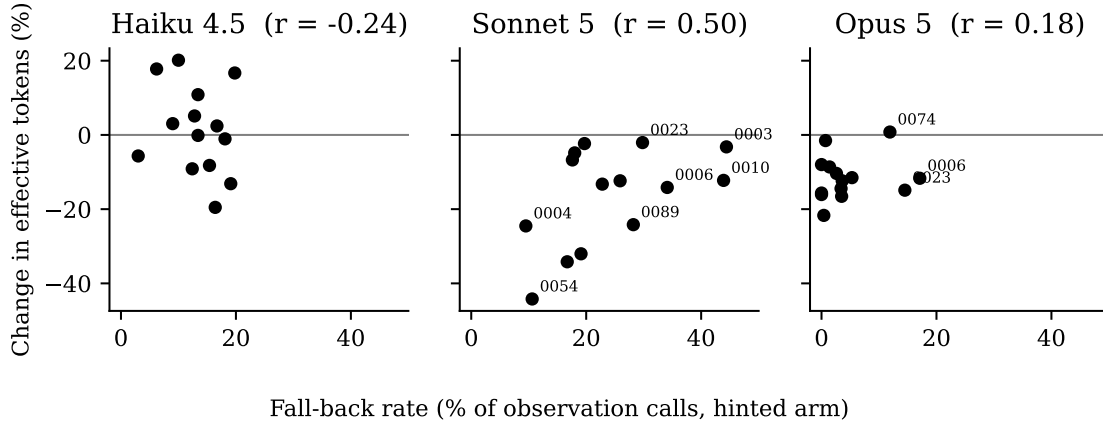


Figure 6.7: Accessibility-snapshot fallback rate against effective-token saving, by application.

On Opus, which rarely fell back, a different predictor emerges: the saving tracks how much observation content the baseline agent consumed per run ($r = -0.61$, $p = 0.02$). The applications Opus observed most were the ones hints saved most on, as expected if the saving is proportional to the share of the context that observations occupy; on Sonnet, the fallbacks obscure that relationship.

6.4.3 By application category

Appendix K gives the per-application effects grouped by WebTestBench’s seven categories. With two applications per category, no test is meaningful, and the pattern is descriptive: the two applications in a category are typically as far apart as applications in different ones. On Sonnet, the two Presentation applications saved 44% and 2%, the two Search applications 5% and 3%, and the two Workflow applications 25% and 32%; on Haiku, the two Data Management applications moved by +0.72 and -0.06 in F1. Within the range this dataset covers, the effect of the representation does not depend on application category.

6.5 Break-even against the cost of injection

Table 6.4 reports the cost of injecting hints into the fourteen applications with three models. The injections used in the experiment were performed by Claude Fable 5; those by Opus 5 and Sonnet 5 were made for comparison and are discussed at the end of this section. Injection by Fable took 428s on average (range 258–599s), used 76 turns and cost \$5.35 at API rates where Fable 5 input costs \$10 per million tokens [41] (range \$2.97–\$8.15). It annotated 75 elements per application on average (range 29–165), and the validator reported no errors on any application. Cost scaled with the size of the application, and the cheapest injections were among the smallest applications by hinted element.

Table 6.4: Cost of hint injection by injecting model.

Injector	Cost (\$)	Duration (s)	Turns	Hinted elements
Fable 5	5.35 (2.97–8.15)	428 (258–599)	76 (47–106)	75 (29–165)
Opus 5	6.62 (3.56–9.89)	905 (565–1314)	108 (73–137)	95 (37–160)
Sonnet 5	2.25 (1.14–3.15)	719 (399–1060)	81 (59–110)	72 (32–131)

Note: Mean (min–max) over one injection of each of the fourteen applications, via the Hint Injector skill (Section 4.1). Cost is computed from token usage at API rates (Section 5.4); duration is wall-clock time. The static validator reported no errors for any injector. Only the Fable 5 injections were used in the test runs of Sections 6.2–6.4; the break-even figures rest on them.

Against a mean saving of \$0.57 per run on Sonnet and \$0.76 on Opus, the Fable injection cost is recovered after 9.4 and 6.9 test runs respectively. Figure 6.8 shows the cumulative cost of testing an application with and without hints as a function of the number of runs, with the hinted line starting at that injection cost and rising more slowly; the break-even point is where the lines cross. On Haiku, where hints did not reduce the cost of a run, the lines do not cross, and injection is a net cost at every horizon.

However, because both the injection cost and the per-run saving vary by application, the range of break-even points is large. On Sonnet, the break-even ranges from 2.5 runs on 0054 (injection \$3.59, saving \$1.42 per run) to 86 runs on 0003 (injection \$6.17, saving \$0.07); on Opus from 3 runs on 0089 to 68 on 0063. Half of the applications break even within 11 runs on Sonnet and within 7 on Opus.

For a test suite that is run on every change to an application, a horizon of a few runs is short. The per-run saving is measured on a single checklist of 13–29 test cases; however, a longer suite would save proportionally more per run. The injection cost is the cost of annotating an application once, and does not include the hints maintenance cost mentioned earlier.

Whether a cheaper injector would shorten the horizon was examined by repeating the injection with Sonnet 5 and Opus 5 on the same fourteen applications. Sonnet’s injections cost \$2.25 per application on average, less than half of Fable’s, despite taking more turns (81 against 76) and twice the tokens, because of its lower price. Hints injected by Sonnet would therefore break even

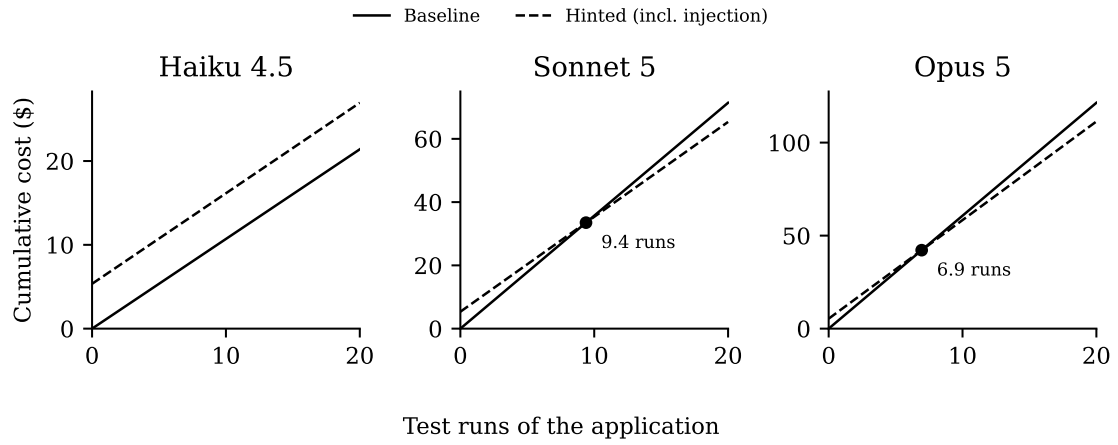


Figure 6.8: Cumulative testing cost with and without hints, including the upfront cost of hint injection with Table 5.

after about 4 runs on Sonnet and 3 on Opus, if they produced the same per-run saving. However, that condition was not tested. The Sonnet and Opus injections were not evaluated with test runs, and so nothing can be said about the quality of the hints they produced.

Chapter 7

Discussion and limitations

7.1 Answers to the research questions

RQ1 asked whether an agent observing through semantic snapshots detects defects as well as one observing through accessibility snapshots. On Sonnet 5 and Opus 5, it does: no efficacy metric moved by more than three points, the one significant difference is a two-point rise in recall on Opus, and the confidence intervals rule out any degradation larger than three points in precision or F1 and one point in recall. The result carries a caveat about how much of it can be explained by the semantic representation. Opus fell back on the accessibility snapshot on 5% of its observations, so its verdicts are almost entirely a product of the hints; however, Sonnet fell back in 24% of observation calls, so its result describes an agent observing mainly, not exclusively, through them. On Haiku 4.5, the answer is different in kind: all three metrics rose by nine to thirteen points, without reaching significance across fourteen applications, but consistent in direction.

RQ2 asked how the semantic representation affects the cost of a run. Effective token use fell by 15.9% on Sonnet and 12.6% on Opus, worth \$0.57 and \$0.76 per run at API rates, and did not change on Haiku. The saving came entirely from input, and specifically from the context re-read on every turn, which shrank by 13-18% across the three models; output tokens were unchanged on Opus and rose on Sonnet and Haiku. Latency was unaffected on the two larger models, because model time per turn does not depend on context size, and the number of turns did not change. The observation size reduction of 69% therefore reached the run as a reduction of 14-17% in input tokens, diluted by the fixed prefix, reasoning and tool results that share the context with observations, and by the fallbacks and additional output.

RQ3 asked whether these effects depend on the model or the application. They depend on the model, and not only in magnitude. On the larger models, hints changed the cost of the task and left the outcome alone; on Haiku they changed the outcome and left the cost alone, with hinted runs taking 30% more turns and more often completing the checklist items. Application category did not matter, and neither did average page size.

7.2 Interpretation

7.2.1 The effect depends on the model

The clearest pattern in the results is that the semantic representation had a different effect depending on the model. On Sonnet and Opus, it lowered the cost of a run and did not affect the verdict accuracy; on Haiku, it did not affect cost and improved the verdicts. Hinted runs on Haiku took 30% more turns, produced 27% more output, and more often carried the checklist to the end. The per-turn context on Haiku fell by 18%, more than on either larger model, and the agent spent that reduction on further work.

One reading is that a smaller context matters most to the model least able to cope with a large one. Baseline Haiku abandoned the task in 30 of 140 runs, typically after an obstacle such as a persistence failure or a required form field; a leaner observation appears to have left it with more of its working state intact at the point where it would otherwise have given up. The larger models did not need that help. They completed the checklist in both arms, so hints could only make the completion cheaper. Opus in particular observed the page far less often than the other two, 14.6

times per baseline run against 39.2 on Haiku and 52.9 on Sonnet, and fell back on the accessibility tree on one observation in twenty. In other words, a model that reads less and retains more gains less from being given less to read.

That said, the difference in efficacy between models is far larger than the difference hints make to any one of them: F1 rises from 0.47 to 0.81 to 0.90 across the baseline arm, and hinting Haiku brings it to 0.57, well short of unhinted Sonnet. Simply put, hints do not substitute for capability. It is worth noting that Haiku 4.5 was released in September 2025 and Sonnet 5 and Opus 5 in June and July 2026, so the comparison spans close to a year of model development as well as a difference in scale.

7.2.2 Explaining the token savings

A semantic snapshot is 69% smaller than the accessibility snapshot of the same page, and a hinted run consumes 13-16% fewer effective tokens. That gap can be explained by three dilutions. The hinted agent still fell back on the accessibility snapshot for some of its observations; observations are only about 60% of what a run appends to its context, the rest being action results and the agent’s own reasoning; and the context re-read on every turn carries a fixed prefix of system prompt, tool definitions and checklist, and is bounded by compaction, so the saving reaches only the part appended since the last compaction.

The implication is that, in this setting, observation was a smaller share of cost than the share reported for browser agents elsewhere. The 80-99% figure for interface representation in UI-agent settings [20] describes agents whose context is dominated by the current page; here the context was dominated by everything that accumulated before it, and the page was a minority. Any representation, however compact, is bounded by that share. Further savings would have to come from the rest of the context, through more aggressive compaction of earlier turns, or possibly through the use of “subagents” that are tasked with the completion of a subset of test cases.

Output tokens per turn rose by about 5% on both Sonnet and Opus, an effect which was unexpected at first. This could, in part, be attributed to lengthier tool call commands on the hinted arm. Indeed, every hinted action targets an element by its full identifier, such as `[data-semtag-id='products.grid.item.sony-wh-1000xm5']`, where the baseline uses a short reference such as `e22`. This increase, combined with a 4.45% increase in number of turns, can explain the 9% increase in Sonnet’s output tokens in the hinted arm compared to the baseline.

7.2.3 What equal efficacy shows

The equal-efficacy result on Sonnet and Opus is evidence that the semantic snapshot was sufficient for the test cases in this dataset. However, it is not evidence that the snapshot describes the page as completely as the accessibility tree does. Sufficiency was demonstrated against 13-29 behaviour-level test cases per application, on applications whose interfaces are built from components the vocabulary covers well, like forms, lists and navigation.

The fallback rates bound the result further. On Opus, the verdicts came almost entirely from the semantic representation, so the sufficiency claim stands for that model. On Sonnet however, a quarter of observations went through the accessibility tree, to access information such as post-action state and transient controls that the hints do not carry. This indicates that we cannot attribute the efficacy of the hinted agents to the completeness of the semantic representation. We do not know how Sonnet agents would have performed without the fallbacks.

What the design did demonstrably achieve is a bounded observation format. The semantic snapshot stayed between 1.8k and 2.9k characters on every application, while the accessibility snapshot ranged from 3.8k to 12.3k: the wrapper nodes, reference annotations and repeated structure of the accessibility tree can be removed without loss of anything the test cases required. This is the second design goal, “compactness of representation”, met.

7.3 Relation to prior work

The observation-reduction systems covered in Chapter 2 report savings of the same order as the one measured here at the level of a single observation. FocusAgent halves the accessibility tree size [21]; LineRetriever removes 61-73% of a DOM or accessibility observation [23]; and the semantic snapshot is 69% smaller than the accessibility snapshot of the same page. However, those systems

compute the reduction from the full representation on every step, with a retrieval or pruning model in the loop. Here, the selection is made once, when the hints are written into the source. At runtime the extractor reads the annotated elements from the DOM with no model in the loop. The saving at the level of a run is a different matter: in a cached, long-context agent the page is a minority of what the model reads, so the 80-99% share attributed to interface representation elsewhere [20] does not carry over.

The model dependence of the effect is consistent with the finding that the appropriate observation format depends on the capability of the model [24]. However, that work found stronger models benefiting from richer context; here the stronger models were indifferent to the representation on efficacy and benefited on cost, while the weakest benefited on efficacy and not on cost. The finding is also consistent with Salva’s and Chevrot et al.’s results on autonomous testing agents [39, 31]: model capability separated the models on verdict correctness far more than any change to what they observed, and run-to-run variance was large enough on the smallest model to require the repetitions used here.

7.4 Threats to validity

7.4.1 Internal validity

Prompt asymmetry. The hinted prompt carries a section the baseline prompt lacks, instructing the agent to prefer semantic observation; the two arms are therefore not identical in prompt, and any effect of that section on the agent’s testing behaviour is attributed to hints. Hinted agents also deal with the complexity of handling two MCP servers in parallel, compared to one on the baseline, making the task slightly more difficult for them.

Single author. The test cases and the semantic design system were written by the same person. To mitigate the risks from that, the corrections started from WebTestBench’s own checklists and were made before any hinted run. Also, hints were placed by the injector following a set of rules, not by hand.

Discarded runs. Runs that reached the turn limit without a verdict for every test case were discarded and repeated; if the arms differ in which runs they lose, the surviving ten are not equally representative.

Injector coverage. Hints were injected once per application and reviewed in the overlay; however, the run logs show the injector left some screens unannotated, including forms and protected routes, and conditional controls that the vocabulary could have carried, on which the hinted agent received a near-empty snapshot and fell back immediately. The treatment evaluated is therefore the injector’s coverage of each application, not exactly the vocabulary’s, which biases the results against the hinted arm.

7.4.2 External validity

Single vendor. All three models are Anthropic’s, the agent is built on Anthropic’s SDK, and the effective-token formula follows Anthropic’s pricing; the results say nothing about other vendors’ models or cost structures, and the token saving in particular depends on a caching scheme whose pricing weights differ.

Generated applications. The applications are a subset of fourteen of WebTestBench’s 100 applications. They are Lovable-generated in a few LLM prompts, front-end only, with synthetic data and no authentication or backend under test, consisting mainly of “CRUD” (Create, Read, Update, Delete) workflows. This is far simpler than production systems.

Single injection. Each application was hinted only once, so nothing is known about the variance between injections.

7.4.3 Construct validity

Metric resolution. Efficacy is measured on 13-29 test cases per application, of which at least two fail. At that granularity, precision and recall are low-resolution, and a single verdict can move an application’s precision or recall by tens of points.

Verdict interpretation. Test cases are stated at the level of behaviour, so whether a verdict is correct depends in part on the agent’s interpretation of the case. That being said, both arms share that dependence.

Visual oracles. Visual oracles cannot be satisfied by either representation without screenshots; test cases with a visual component were where both arms performed worst, and where hints made no difference.

Whole-checklist runs. Every run gives the agent the whole checklist, so a verdict can depend on the position of the case in the checklist, on the agent’s actions on earlier cases, and on the length of the run before the case is reached; both arms share this feature, but it inflates variance and interacts with context size, which is a variable under test.

Verdicts without exercise. The scorer measures whether a verdict is correct, not how it was reached. On application 0001, six of ten hinted Haiku runs passed a test case for a defective purchase link without clicking it: the hint identified the element, the agent read its href through Playwright’s `browser_evaluate` command, and inferred from the attribute that the link worked. If the same shortcut leads to the correct verdict elsewhere, the metric would not detect it.

Chapter 8

Conclusion and future work

8.1 Contributions

This project set out to make a running web application legible to the agent testing it, and to assess the value of that legibility. Our answers to the three research questions are given in Section 7.1.

The Semtag specification defines a closed vocabulary of seven attributes for annotating front-end source code, and a compact observation format derived from them at runtime. The format groups elements by role, folds repeated collections, and holds a page’s observation between 1.8k and 2.9k characters on every application studied, where the accessibility snapshot of the same pages ranged from 3.8k to 12.3k.

The reference implementation covers an injection skill that annotates a repository with semantic hints, an extractor served over MCP that assembles semantic snapshots from the live page, an adapted testing agent in which the two arms differ only in what they observe, and the tooling to review hints and inspect runs.

The empirical comparison, over 840 runs across three models and fourteen applications, found that the representation preserved defect-detection efficacy on Sonnet 5 and Opus 5 while reducing effective token use by 15.9% and 12.6% at unchanged latency; and that on Haiku 4.5 it improved efficacy instead, without reducing cost. The saving is bounded by the share of the agent’s context that observations occupy, and the effect of the representation depends on the model used for the agent.

8.2 Implications

A broad implication is that ideas developed for general browser agents can be brought into narrower settings, such as testing, allowing them to be designed more freely. Agent-oriented interfaces have been proposed as a standard for sites to implement for the benefit of third-party agents, but their designs are constrained by that setting: adoption across many parties, disclosure across a trust boundary, and a curated action space to keep the contract small. None of those constraints applies when the party running the agent owns the source and the task is fixed. Other closed settings, such as internal tools, may be similar.

For research on browser agents, our results show that reducing the size of an agent’s observation does not reduce the cost of a run by the same proportion, as that observation is only part of what the model re-reads on each turn. Work that reports observation-level savings should also report run-level ones, and the two should be expected to diverge as agents grow longer-horizon and more cache-dependent. A further finding is that a representation’s effect is a property of the model as much as of the representation. In our work, the same snapshot made two models cheaper and a third more capable, so a design evaluated on one model class would say little about another.

For agent-oriented interfaces, the work shows that an application can make itself legible to an agent in fewer tokens, without narrowing the actions available through the interface. Unlike VOIX and webMCP [25, 13], which curate the action space, the semantic snapshot is designed to expose every element a user can act on through the GUI, and is about a third the size of the accessibility snapshot on average. As discussed, however, some of what the GUI and accessibility tree make visible is still absent from it, so the semantic snapshot cannot yet replace either.

8.3 Future work

Everything in this evaluation was arranged to give hints their best chance. The agent could fall back on the accessibility snapshot whenever the semantic one was not enough; the applications were small, front-end only, and generated by a coding agent in a few prompts; the hints were injected by the most capable model available and checked by hand. Each of those choices was right for isolating the effect of the representation, but each leaves a question about what happens when it is relaxed.

The fallback should be removed first. The hinted agent could always revert to the accessibility snapshot, so its performance on semantic snapshots alone is unknown; running it without the fallback would establish which test cases the hints can support on their own. Visual test cases require a different approach, since neither snapshot can address them and both agents performed worst on that class. Providing both agents with screenshots, as WebVoyager and OmniParser do [16, 18], would show whether the token saving persists once a visual channel is available. Running the experiments on larger applications with real backends, persistent data and login flows would test the coverage of the vocabulary and of the injector, and whether the saving holds at that scale. The three models tested were all Anthropic’s, and the smallest is almost a year older than the other two, so the effect of model size cannot be separated from that of model age; repeating the comparison on other frontier models, including open-source ones, would show whether the pattern holds more widely.

The cost side of the findings has its own open questions. Injection cost with Sonnet and Opus was measured, but hints themselves were not evaluated, so it is not known whether cheaper injection produces hints of the same quality. A larger saving would come from removing the injection step altogether. Hints were added after the fact only because the applications already existed; if a coding agent wrote them alongside the code, the separate injection step would disappear, and with it most of the break-even cost. Further work on hint maintenance would also address one of the system’s more important flaws. Hints that have gone stale are still present and still extracted, and nothing in the current system detects that drift.

The last question is the one that this work set aside. Oracles were fixed in advance, but the question of where these oracles come from in practice was not addressed. In the setting that we explored, where most of the code is generated by an LLM agent, these oracles could originate from the developer’s conversations with that agent. Those conversations hold statements about the expected behaviour of the application. This is the opportunity that Codespeak [42] is exploring: capturing the human intent behind the code, maintaining it into a running list of requirements, and detecting drift between those requirements and the code. In that setting, an agent could produce the test cases that the testing agent runs, and keep them current as the application changes. Further work should explore whether those generated test cases are reliable when given to a hint-enabled ATA.

8.4 Closing remarks

This work started from two observations. As code becomes cheap to generate, verifying it becomes the main cost; and the agents used to verify web applications perceive them through screenshots, HTML and the accessibility tree, which describe how a page is built rather than what it does. The semantic hint system lets an application state what its elements are for, and the evaluation shows the value of that system: on capable models, the same bug-detection ability at a lower and more predictable cost, and on a weaker one, better detection ability at the same cost. The saving is bounded by everything else the agent carries in its context, and the semantic representation still has gaps that the accessibility tree fills. But the premise holds: a testing agent works more economically on an interface with affordances built for it than by inferring the meaning of one built for people; and when the code is AI-generated, those affordances can be placed alongside it.

References

1. Microsoft. *Playwright*. Available from: <https://playwright.dev/> [Accessed 8th September 2026]
2. Cypress.io. *Cypress*. Available from: <https://www.cypress.io/> [Accessed 8th September 2026]
3. Selenium Project. *Selenium*. Available from: <https://www.selenium.dev/> [Accessed 8th September 2026]
4. Anthropic. *Claude Code*. Available from: <https://claude.com/product/claude-code> [Accessed 8th September 2026]
5. OpenAI. *Codex*. Available from: <https://chatgpt.com/codex/> [Accessed 8th September 2026]
6. Anysphere. *Cursor*. Available from: <https://cursor.com> [Accessed 8th September 2026]
7. Leotta M, García B, Ricca F, Whitehead J. Challenges of End-to-End Testing with Selenium WebDriver and How to Face Them: A Survey. In: *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. Piscataway, NJ, USA: IEEE; 2023. p. 339–350. doi:[10.1109/ICST57152.2023.00039](https://doi.org/10.1109/ICST57152.2023.00039)
8. Liu X, Song Z, Fang W, Yang W, Wang W. WEFix: Intelligent Automatic Generation of Explicit Waits for Efficient Web End-to-End Flaky Tests. In: *Proceedings of the ACM Web Conference 2024*. New York, NY, USA: Association for Computing Machinery; 2024. p. 3043–3052. doi:[10.1145/3589334.3645628](https://doi.org/10.1145/3589334.3645628)
9. Leotta M, Stocco A, Ricca F, Tonella P. Robula+: an algorithm for generating robust XPath locators for web testing. *Journal of Software: Evolution and Process* 2016; 28(3):177–204. doi:[10.1002/smr.1771](https://doi.org/10.1002/smr.1771)
10. Mesbah A, Deursen A van, Lenselink S. Crawling Ajax-Based Web Applications through Dynamic Analysis of User Interface State Changes. *ACM Transactions on the Web* 2012; 6(1):3:1–3:30. doi:[10.1145/2109205.2109208](https://doi.org/10.1145/2109205.2109208)
11. Deursen A van, Mesbah A, Nederlof A. Crawl-based analysis of web applications: Prospects and challenges. *Science of Computer Programming* 2015; 97:173–180. doi:[10.1016/j.scico.2014.09.005](https://doi.org/10.1016/j.scico.2014.09.005)
12. Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 2015; 41(5):507–525. doi:[10.1109/TSE.2014.2372785](https://doi.org/10.1109/TSE.2014.2372785)
13. Perera D. webMCP: Efficient AI-Native Client-Side Interaction for Agent-Ready Web Design. *arXiv* [Preprint]. 2025. doi:[10.48550/arXiv.2508.09171](https://doi.org/10.48550/arXiv.2508.09171) [Accessed 8th September 2026]
14. W3C Web Accessibility Initiative (WAI). *WAI-ARIA overview*. Available from: <https://www.w3.org/WAI/standards-guidelines/aria/> [Accessed 8th September 2026]
15. Lù XH, Kamath G, Mosbach M, Reddy S. Build the web for agents, not agents for the web. *arXiv* [Preprint]. 2025. doi:[10.48550/arXiv.2506.10953](https://doi.org/10.48550/arXiv.2506.10953) [Accessed 8th September 2026]
16. He H, Yao W, Ma K, Yu W, Dai Y, Zhang H, et al. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In: Ku LW, Martins A, Srikumar V, eds. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics; 2024. p. 6864–6890. doi:[10.18653/v1/2024.acl-long.371](https://doi.org/10.18653/v1/2024.acl-long.371)

17. Cheng K, Sun Q, Chu Y, Xu F, Li Y, Zhang J, et al. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In: Ku LW, Martins A, Srikumar V, eds. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics; 2024. p. 9313–9332. doi:[10.18653/v1/2024.acl-long.505](https://doi.org/10.18653/v1/2024.acl-long.505)
18. Lu Y, Yang J, Shen Y, Awadallah A. OmniParser for Pure Vision Based GUI Agent. *arXiv* [Preprint]. 2024. doi:[10.48550/arXiv.2408.00203](https://doi.org/10.48550/arXiv.2408.00203) [Accessed 8th September 2026]
19. Abuelsaad T, Akkil D, Dey P, Jagmohan A, Vempaty A, Kokku R. Agent-E: From Autonomous Web Navigation to Foundational Design Principles in Agentic Systems. In: *NeurIPS 2024 Workshop on Open-World Agents*. 2024. Available from: <https://openreview.net/forum?id=7PQnFTbizU> [Accessed 9th September 2026]
20. Ran D, Gong Z, Guo Y, Wu M, Cao Y, Lu H, et al. From User Interface to Agent Interface: Efficiency Optimization of UI Representations for LLM Agents. *arXiv* [Preprint]. 2025. doi:[10.48550/arXiv.2512.13438](https://doi.org/10.48550/arXiv.2512.13438) [Accessed 8th September 2026]
21. Kerboua I, Shayegan SO, Lù XH, Boisvert L, Thakkar M, Caccia M, et al. FocusAgent: Simple Yet Effective Ways Of Trimming The Large Context of Web Agents. In: *Lifelong Agents Workshop at ICLR 2026*. 2026. Available from: <https://openreview.net/forum?id=CTsfUq3DpR> [Accessed 8th September 2026]
22. Zhang J, Chen K, Lu Z, Zhou E, Yu Q, Zhang J. Prune4Web: DOM Tree Pruning Programming for Web Agent. *Proceedings of the AAAI Conference on Artificial Intelligence* 2026; 40(41):34710–34718. doi:[10.1609/aaai.v40i41.40772](https://doi.org/10.1609/aaai.v40i41.40772)
23. Kerboua I, Shayegan SO, Thakkar M, Lù XH, Caccia M, Eglin V, et al. LineRetriever: Planning-Aware Observation Reduction for Web Agents. *arXiv* [Preprint]. 2025. doi:[10.48550/arXiv.2507.00210](https://doi.org/10.48550/arXiv.2507.00210) [Accessed 8th September 2026]
24. Enomoto M, Obara R, Zhang H, Oyamada M. Read More, Think More: Revisiting Observation Reduction for Web Agents. *arXiv* [Preprint]. 2026. doi:[10.48550/arXiv.2604.01535](https://doi.org/10.48550/arXiv.2604.01535) [Accessed 8th September 2026]
25. Schultze S, Kietzmann MV, Schönfeld NL, Stock-Homburg R. VOIX: A Web-Native Interface for Agents. In: Mauri A, Burgueño L, Tommasini R, eds. *Web Engineering*. Cham: Springer Nature Switzerland; 2026. p. 199–208. doi:[10.1007/978-3-032-29372-5_15](https://doi.org/10.1007/978-3-032-29372-5_15)
26. Qiu Z, Sun J, Xia C, Zheng J, Peng X. CI4A: Semantic Component Interfaces for Agents Empowering Web Automation. *arXiv* [Preprint]. 2026. doi:[10.48550/arXiv.2601.14790](https://doi.org/10.48550/arXiv.2601.14790) [Accessed 22nd May 2026]
27. Google. *WebMCP*. Available from: <https://developer.chrome.com/docs/ai/webmcp> [Accessed 8th September 2026]
28. Feldt R, Kang S, Yoon J, Yoo S. Towards Autonomous Testing Agents via Conversational Large Language Models. In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*. Piscataway, NJ, USA: IEEE; 2023. p. 1688–1693. doi:[10.1109/ASE56229.2023.00148](https://doi.org/10.1109/ASE56229.2023.00148)
29. Mustafa A, Wan-Kadir WMN, Ibrahim N, Shah MA, Younas M, Khan A, et al. Automated Test Case Generation from Requirements: A Systematic Literature Review. *Computers, Materials & Continua* 2021; 67(2):1819–1833. doi:[10.32604/cmc.2021.014391](https://doi.org/10.32604/cmc.2021.014391)
30. Ferreira M, Viegas L, Faria JP, Lima B. Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*. Piscataway, NJ, USA: IEEE; 2025. p. 1–11. doi:[10.1109/AST66626.2025.00007](https://doi.org/10.1109/AST66626.2025.00007)
31. Chevrot A, Vernotte A, Falleri JR, Blanc X, Legeard B, Cretin A. Are Autonomous Web Agents Good Testers? *Proceedings of the ACM on Software Engineering* 2025; 2(ISSTA):ISSTA010:206–ISSTA010:228. doi:[10.1145/3728879](https://doi.org/10.1145/3728879)
32. Ye N, Yu X, Xu R, Peng T, Yu Z. AI Agents for Web Testing: A Case Study in the Wild. In: *NeurIPS 2025 Workshop on Bridging Language, Agent, and World Models for Reasoning and Planning (LAW)*. San Diego, CA, USA; 2025. Available from: <https://neurips.cc/virtual/2025/loc/san-diego/137188> [Accessed 8th September 2026]

33. Teoh X, Lin Y, Nguyen DM, Ren R, Zhang W, Dong JS. WebTestPilot: Agentic End-to-End Web Testing against Natural Language Specification by Inferring Oracles with Symbolized GUI Elements. *Proceedings of the ACM on Software Engineering* 2026; 3(FSE):FSE087:1933–FSE087:1956. doi:[10.1145/3797115](https://doi.org/10.1145/3797115)
34. Kong F, Zhang J, Yue Y, Sun C, Tian Y, Feng S, et al. WebTestBench: Evaluating Computer-Use Agents towards End-to-End Automated Web Testing. *arXiv [Preprint]*. 2026. doi:[10.48550/arXiv.2603.25226](https://doi.org/10.48550/arXiv.2603.25226) [Accessed 8th September 2026]
35. Deng X, Gu Y, Zheng B, Chen S, Stevens S, Wang B, et al. MIND2WEB: towards a generalist agent for the web. In: *Advances in Neural Information Processing Systems 36*. Red Hook, NY, USA: Curran Associates Inc.; 2023. p. 28091–28114. doi:[10.52202/075280-1220](https://doi.org/10.52202/075280-1220)
36. Zhou S, Xu FF, Zhu H, Zhou X, Lo R, Sridhar A, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. In: *The Twelfth International Conference on Learning Representations*. 2024. Available from: <https://openreview.net/forum?id=oKn9c6ytLx> [Accessed 8th September 2026]
37. Drouin A, Gasse M, Caccia M, Laradji IH, Verme MD, Marty T, et al. WorkArena: how capable are web agents at solving common knowledge work tasks? In: *Proceedings of the 41st International Conference on Machine Learning*. Vienna, Austria: JMLR.org; 2024. p. 11642–11662. Available from: <https://proceedings.mlr.press/v235/drouin24a.html> [Accessed 8th September 2026]
38. Le Sellier de Chezelles T, Gasse M, Lacoste A, Caccia M, Drouin A, Boisvert L, et al. The BrowserGym Ecosystem for Web Agent Research. *Transactions on Machine Learning Research* 2025. Available from: <https://openreview.net/forum?id=5298fKGmv3> [Accessed 8th September 2026]
39. Salva S. Reliable execution of natural language test cases for GUI applications using LLM agents. *Software Quality Journal* 2026; 34(3):33. doi:[10.1007/s11219-026-09767-2](https://doi.org/10.1007/s11219-026-09767-2)
40. Anthropic. *Claude Agent SDK overview*. Available from: <https://code.claude.com/docs/en/agent-sdk/overview> [Accessed 8th September 2026]
41. Anthropic. *Claude API pricing*. Available from: <https://platform.claude.com/docs/en/about-claude/pricing> [Accessed 8th September 2026]
42. CodeSpeak. *CodeSpeak: Software Engineering with AI*. Available from: <https://codespeak.dev/> [Accessed 9th September 2026]

Declarations

Use of Generative AI

Generative AI tools were used at several points in this project. The models that act as testing agents and as the hint injector in the experiments are the object of study; this statement covers the use of AI tools in producing the software and the report.

I acknowledge the use of Claude with Opus 4.6, 4.7, 4.8 and 5 and Fable 5 (Anthropic, <https://claude.ai>) to synthesise notes on papers after I had read them, to clarify specific points in the literature, and to search for related work. The papers central to this research were read in full without AI assistance, and every source cited in this report was read and verified by me. Claude was also used to comment on drafts of this report, suggest edits for clarity, assist with LaTeX formatting, and create tables and charts for the final report; the underlying data itself was never hardcoded or transferred by an AI.

I acknowledge the use of Claude Code versions 2.1.150 to 2.1.261 with models Sonnet 4.6 and Opus 4.6 (Anthropic, <https://claude.com/product/claude-code>) to explore and explain the architecture of the WebTestBench [34] and WebTestPilot [33] codebases at the start of the project.

I acknowledge the use of Codex versions 26.602 to 26.901.51231 with models GPT-5.5, GPT-5.6 Luna, GPT-5.6 Terra and GPT-5.6 Sol (OpenAI, <https://chatgpt.com/codex/>) for two purposes. It helped write code for tooling peripheral to the experimental pipeline, namely the Hint Overlay tool, the Trace Viewer tool and the script that aggregates the scores of a run suite into a CSV, from specifications I wrote; every change was reviewed and tested by me. It was also used to analyse agent run traces, so that behaviour could be surveyed systematically and at a large scale across runs. Every behaviour it reported was checked against the source logs before being relied on. The agent prompts, the Semtag specification and the corrected test cases, which directly determine agent behaviour and ground truth, were written without AI assistance.

I confirm that the text of this report is my own work, that the design of the system, the experiments and the interpretation of their results are my own, and that no content presented as my own work has been fully generated by AI without human involvement and verification. AI-generated code is confined to the tooling described above.

Ethical considerations

Based on the Imperial ethics checklist, this project was not expected to require formal ethical approval. The planned work did not involve human participants, human tissue, animals, medical data, personal data collection, or user tracking. The evaluation used generated applications, synthetic data, and/or publicly available benchmark material, and the testing agent was evaluated in controlled browser environments, not on real user data or live third-party systems. The corrections to the benchmark’s test cases were made by hand against the running applications and involved no data beyond the applications themselves. Compute costs were covered by Codespeak, who had no role in the design of the experiments, the analysis of the results or the writing of this report.

Sustainability

The main environmental cost of this project was the inference compute consumed by the 840 experimental runs and the three hint-injection run suites, which took 137 hours and 35 minutes of wall-clock time in total. Several choices limited the compute spent. The single AWS instance that hosted the runs was stopped between run batches; harness defects were found and fixed on

five development applications before any full-scale batch was launched; the turn limit bounded the cost of runs that lost their way; and baseline and hinted batches ran concurrently on one instance. No models were trained. Some compute was nonetheless spent without direct return: runs that reached the turn limit were discarded and repeated, and the injections by Sonnet 5 and Opus 5 (Section 6.5) were run for cost comparison but not evaluated.

Availability of Data and Materials

All source code, data and experimental artefacts produced in this project are publicly available.

- Semtag specification and Hint Injector skill: <https://github.com/timotheeiss/hint-injection-skill> (commit 8733d60 at time of the experiments)
- Hint Extractor MCP server: <https://github.com/timotheeiss/semantic-hints-mcp> (commit 29fe4fb at time of the experiment)
- WebTester harness, forked from WebTestBench and adapted as described in Section 4.3, including the runner and aggregator scripts: <https://github.com/timotheeiss/WebTestBench> (commit f857a03 at time of the experiment)
- Hint Overlay tool: <https://github.com/timotheeiss/semtag-overlay-tool> (commit fe260fa)
- Trace Viewer tool: <https://github.com/timotheeiss/semtag-trace-viewer> (commit 74858cb)
- Hinted copies of the fourteen applications in our dataset, and corrected test cases for the fourteen applications: https://github.com/timotheeiss/WebTestBench_adapted_dataset (commit e841537)
- Experiment results (raw run artefacts, verdicts, results, scores): <https://github.com/timotheeiss/semtag-research-results> (commit 186b92e)

The applications are drawn from the WebTestBench dataset [34], available under the Apache 2.0 licence at the following URL: <https://huggingface.co/datasets/friedrichor/WebTestBench>

Appendix A

Full semantic snapshot of the home page of WebTestBench application 0001

This appendix reproduces, without modification, the semantic snapshot for the home page of application 0001 (CuratedPicks), the running example of Chapter 3, captured with hidden elements excluded. It is 6,396 characters. The accessibility snapshot of the same page in the same state, as observed by the baseline agent, is 22,276 characters and is available in the results repository at `semtag-research-results/accessibility_tree_example.yaml`.

The format described in Chapter 3 is visible here. The two “Home” links are distinguished by identifier and target; the icon-only admin link carries an empty name, since the extractor infers nothing from the DOM; and each of the five collections is folded, with `idPattern`, `item` and `itemControls` stated once and one row per `item` under `items`, every `item` remaining addressable by its full identifier. In `home.categories`, the name of each `item` is the accessible name of the whole card, which concatenates title, description and product count; the count is also carried separately by the count `item` control.

```
{
  "url": "http://localhost:7001/",
  "screen": "home",
  "navigation": [
    {
      "id": "nav.logo",
      "name": "CuratedPicks",
      "target": "home.page"
    },
    {
      "id": "nav.home",
      "name": "Home",
      "target": "home.page"
    },
    {
      "id": "nav.products",
      "name": "All Products",
      "target": "products.page"
    },
    {
      "id": "nav.category.tech",
      "name": "Tech",
      "target": "category.tech"
    },
    {
      "id": "nav.category.home",
      "name": "Home",
      "target": "category.home"
    },
    {
      "id": "nav.category.fitness",
      "name": "Fitness",
      "target": "category.fitness"
    }
  ]
}
```

```

    },
    {
      "id": "nav.admin",
      "name": "",
      "target": "admin.page"
    },
    {
      "id": "home.hero.browse-all",
      "name": "Browse All Products",
      "target": "products.page"
    },
    {
      "id": "home.hero.explore-tech",
      "name": "Explore Tech",
      "target": "category.tech"
    },
    {
      "id": "home.featured.view-all",
      "name": "View all",
      "target": "products.page"
    },
    {
      "id": "home.latest.tech.view-all",
      "name": "View all",
      "target": "category.tech"
    },
    {
      "id": "home.latest.home.view-all",
      "name": "View all",
      "target": "category.home"
    },
    {
      "id": "home.latest.fitness.view-all",
      "name": "View all",
      "target": "category.fitness"
    },
    {
      "id": "footer.home",
      "name": "Curated Picks",
      "target": "home.page"
    },
    {
      "id": "footer.category.tech",
      "name": "Tech Gadgets",
      "target": "category.tech"
    },
    {
      "id": "footer.category.home",
      "name": "Home Goods",
      "target": "category.home"
    },
    {
      "id": "footer.category.fitness",
      "name": "Fitness",
      "target": "category.fitness"
    },
    {
      "id": "footer.products",
      "name": "All Products",
      "target": "products.page"
    },
    {
      "id": "footer.admin",
      "name": "Admin Panel",
      "target": "admin.page"
    }
  ],
  "input": [
    {
      "id": "search.query",
      "name": "Search products...",
      "state": "search.query",
      "value": ""
    }
  ]

```

```

    }
  ],
  "collections": [
    {
      "id": "home.featured",
      "idPattern": "home.featured.item.{key}[.{control}]",
      "item": {
        "role": "navigation",
        "target": "product.detail"
      },
      "itemControls": {
        "price": {
          "role": "observable",
          "state": "product.price"
        }
      },
      "fields": [
        "key",
        "name",
        "price.value"
      ],
      "items": [
        [
          "sony-wh-1000xm5",
          "Sony WH-1000XM5",
          "$399"
        ],
        [
          "peloton-bike-plus",
          "Peloton Bike+",
          "$2,495"
        ],
        [
          "dyson-v15-detect",
          "Dyson V15 Detect",
          "$749"
        ]
      ]
    },
    {
      "id": "home.categories",
      "idPattern": "home.categories.item.{key}[.{control}]",
      "item": {
        "role": "navigation"
      },
      "itemControls": {
        "count": {
          "role": "observable",
          "state": "category.product-count",
          "value": "3 products"
        }
      },
      "fields": [
        "key",
        "name",
        "target"
      ],
      "items": [
        [
          "tech",
          "Tech Tech Gadgets The latest in consumer electronics and technology 3 products",
          "category.tech"
        ],
        [
          "home",
          "Home Home Goods Smart home devices, appliances, and household essentials 3 products",
          "category.home"
        ],
        [
          "fitness",
          "Fitness Fitness Equipment and gear for your health and fitness journey 3 products",
          "category.fitness"
        ]
      ]
    }
  ]
}

```

```

    ]
  },
  {
    "id": "home.latest.tech",
    "idPattern": "home.latest.tech.item.{key}[.{control}]",
    "item": {
      "role": "navigation",
      "target": "product.detail"
    },
    "itemControls": {
      "price": {
        "role": "observable",
        "state": "product.price"
      }
    },
    "fields": [
      "key",
      "name",
      "price.value"
    ],
    "items": [
      [
        "sony-wh-1000xm5",
        "Sony WH-1000XM5",
        "$399"
      ],
      [
        "macbook-pro-14",
        "Apple MacBook Pro 14\"",
        "$1,999"
      ],
      [
        "samsung-galaxy-s24-ultra",
        "Samsung Galaxy S24 Ultra",
        "$1,299"
      ]
    ]
  },
  {
    "id": "home.latest.home",
    "idPattern": "home.latest.home.item.{key}[.{control}]",
    "item": {
      "role": "navigation",
      "target": "product.detail"
    },
    "itemControls": {
      "price": {
        "role": "observable",
        "state": "product.price"
      }
    },
    "fields": [
      "key",
      "name",
      "price.value"
    ],
    "items": [
      [
        "dyson-v15-detect",
        "Dyson V15 Detect",
        "$749"
      ],
      [
        "philips-hue-starter-kit",
        "Philips Hue Starter Kit",
        "$199"
      ],
      [
        "breville-barista-express",
        "Breville Barista Express",
        "$699"
      ]
    ]
  }
]

```

```

    },
    {
      "id": "home.latest.fitness",
      "idPattern": "home.latest.fitness.item.{key}[.{control}]",
      "item": {
        "role": "navigation",
        "target": "product.detail"
      },
      "itemControls": {
        "price": {
          "role": "observable",
          "state": "product.price"
        }
      },
      "fields": [
        "key",
        "name",
        "price.value"
      ],
      "items": [
        [
          "peloton-bike-plus",
          "Peloton Bike+",
          "$2,495"
        ],
        [
          "theragun-pro",
          "Theragun Pro",
          "$599"
        ],
        [
          "bowflex-selecttech-552",
          "Bowflex SelectTech 552",
          "$549"
        ]
      ]
    }
  ]
}

```

Appendix B

Semtag Specification

This appendix reproduces, without modification, `hint-design.md` as it stood when the hints used in the evaluation were injected (commit 8733d60 of the `hint-injection-skill` repository). It is the file the Hint Injector skill loads before annotating a repository (Section 4.1). The extractor (Section 4.2) and the observation section of the hinted agent's prompt (Appendix D) were written by hand to follow it, and do not read it; the file's own opening paragraph describes the intended relationship between the three rather than a mechanism. It is written as instructions to a coding agent, which accounts for its imperative register.

Tables 3.1 and 3.2 are drawn from § 3 and § 4 of this file, the naming and coverage conventions of Section 3.3 from § 6, § 8 and § 9, and the collection-folding rules summarised in Section 3.4 from § 7. The examples in Section 3.3 are those of § 10.

The single source of truth for the `data-semtag-*` vocabulary. The hint-injection prompt, the semantic-hints MCP extractor, and the browser-agent prompt all implement this file — change it here first, then in those three places.

B.1 Purpose

Hints describe what the UI **means and affords**, so an agent can understand a screen, act on it, and read its state *without* a full accessibility tree or DOM reading.

Hints never encode test intent: no assertions, no expected values, no pass/fail logic, no "the total should be £42". Describing what the UI *is* is in scope; describing what it *should be* is not.

B.2 How hints reach the agent

The agent sees only a compact snapshot extracted from `data-semtag-*` attributes. Three consequences:

- **The extractor reads live values for you.** For the roles that hold one (`input`, `select`, `toggle`, `slider`, `observable`) it captures the element's current on-screen text or form value automatically. Never write a value into an attribute yourself.
- **Only `data-semtag-*` attributes are read.** Mirroring data into `data-price` or `data-product-id` does nothing — it never reaches the agent.
- **Nothing is inferred, and nothing is repaired.** Tag names, ARIA roles and input types are never used to guess an element's meaning, and a partial hint is never completed for you. An element with a `data-semtag-id` and no `data-semtag-role` lands in `other`, where it is visible as the gap it is.

B.3 The seven attributes

This list is closed. No other `data-semtag-*` attribute is extracted.

Attribute	Value	Notes
<code>data-semtag-id</code>	dotted domain path, e.g. <code>checkout.submit</code>	Required on every hinted element. Unique page-wide.
<code>data-semtag-role</code>	one of the ten roles in § 4	The element's semantic category.
<code>data-semtag-action</code>	verb-object, e.g. <code>submit-order</code>	The user-level action. Only on things that do something.
<code>data-semtag-state</code>	state key(s), e.g. <code>cart.total</code>	What the element reads or writes. See § 5.
<code>data-semtag-target</code>	destination name, e.g. <code>product.detail</code>	Where a navigation goes. Never a raw URL.
<code>data-semtag-controls</code>	the <code>data-semtag-id</code> of a region	For a control that drives a distinct region (search → result list). Omit when there is no clear one.
<code>data-semtag-options</code>	<code>value Label;value2 Label2</code>	A select trigger's available options, so the agent picks one without opening the menu. Label optional.

Use only the attributes that carry information. Do not put all seven everywhere.

Build `data-semtag-options` from the same source that renders the options — `categories.map((c) => `${c.id}|${c.name}`).join(";")` — so the trigger and its menu cannot drift apart.

B.4 The ten roles

This list is closed, and **the role is also the snapshot group**: the extractor files each element under its role, so entries carry no `role` field — the group name already states it.

Role	Use for
navigation	link/button that moves to another screen or view
action	any command button (submit, buy, edit, delete, clear, add)
option	one option inside a select or listbox
input	text/number/search field or textarea
select	a select/dropdown trigger — pair with <code>data-semtag-options</code>
toggle	checkbox, switch, on-off chip, tag filter
slider	range slider
observable	element that only displays a value to be read
region	a meaningful grouping — form, filter panel, dialog, banner
collection	container of repeated items — list, grid, table

Anything else — an off-vocabulary value, or no `data-semtag-role` at all — lands in **other**. That group is the snapshot's record of a hinting gap, never a category to aim for. There are no fallbacks and no repairs: an element with a `data-semtag-action` but no role is **other**, not an action, and Action is not **action**.

B.5 `data-semtag-state`

Two forms, in order of preference:

1. **A state key** naming what the element reads or edits: `cart.total`, `filters.category`, `search.query`. This is the normal case. Comma-separate when one element covers several: `product.price,product.stock`.
2. **A literal value**, only when the element's state is *not* readable as on-screen text: `open/closed`, `selected`, `empty`. An **observable** almost never needs this — the extractor already captures its displayed text.

Put the same key on the control that edits a value and on the element that displays it; that is how the agent connects the two.

B.6 Naming

Stable domain names, never implementation names or positions.

Good	Bad	Why
<code>checkout.submit</code>	<code>button-3</code>	positional, breaks on reorder
<code>submit-order</code>	<code>click-for-test-2</code>	encodes test intent
<code>cart.total</code>	<code>expected-total-42</code>	encodes an expected value
<code>products.grid.item.sony-xm5</code>	<code>component-7</code>	framework name, not domain

`data-semtag-target` names a destination by what it is: `products.page`, `category.tech`, `product.detail`, `external.affiliate`.

B.7 Repeated collections

Lists, grids and tables are the bulk of a snapshot, and almost all of it is repetition. The extractor folds them into one compact block — shared attributes stated once, each item one row — but **only when all four rules hold**:

1. The container carries `data-semtag-role="collection"` and its own `data-semtag-id`.
2. Each item's id is `<prefix>.item.<key>`, keyed by a stable domain key (slug or database id), never an array index.
3. Each per-item control's id is `<prefix>.item.<key>.<control>` (`.open`, `.buy`, `.edit`, `.delete`, `.price`).
4. There are at least 2 items, and they all share one collection ancestor.

Get any of these wrong and folding silently does not happen: rows keyed `admin.product.42.edit` (no `.item.`) stay fully expanded, and so does a grid whose container has no `collection` role. Treat a missing `collection` role on a list container as a bug.

`<prefix>` need not equal the container's own id — membership follows the DOM, so `admin.products.list` holding `admin.products.item.42` is fine.

Folding costs nothing: every item stays individually addressable by its real id.

```
<section data-semtag-id="products.grid" data-semtag-role="collection">
  <a data-semtag-id="products.grid.item.sony-xm5"
    data-semtag-role="navigation"
    data-semtag-target="product.detail"
    href="/product/sony-xm5">Sony WH-1000XM5</a>
  <span data-semtag-id="products.grid.item.sony-xm5.price"
    data-semtag-role="observable"
    data-semtag-state="product.price">GBP 299.00</span>
  <button data-semtag-id="products.grid.item.sony-xm5.buy"
    data-semtag-role="action"
    data-semtag-action="buy-product">Buy</button>
</section>
```

Select options follow the same shape, keyed by option value: `filters.sort.option.price-asc` under trigger `filters.sort`.

B.8 Uniqueness

Ids must be unique page-wide — duplicates break single-element observation.

- **Same item in two collections** (a product in "featured" *and* in a category grid): scope the id by its collection. Thread a collection id into the shared component and build the id from it, so each call site supplies its own (`home.featured`, `products.grid`). Emit no attribute at all when that id is missing, rather than a half-formed one.

- **Same control in two layouts** (desktop bar and mobile drawer): suffix the secondary copy with `.mobile` — `search.query` and `search.query.mobile`. Keep `data-semtag-state` identical on both; only the id differs.
- Put an item's id on the element whose accessible name identifies it (the title link, the row's name cell), not on an outer wrapper whose name would be every word inside it.

B.9 Coverage

Hint an element when an agent would need to **navigate to it, act on it, or read it**. In particular, anything the agent might click must be reachable by `data-semtag-id` alone — if picking one instance out of a repeat would otherwise need the accessibility tree to disambiguate, it needs an instance-level id.

Do not hint decorative elements, layout wrappers, or icons that duplicate an adjacent labelled control.

B.10 Examples

```
<!-- action -->
<button data-semtag-id="checkout.submit"
        data-semtag-role="action"
        data-semtag-action="submit-order">Place order</button>

<!-- observable: the extractor captures "Total: GBP 42.00" as its value -->
<div data-semtag-id="cart.total"
     data-semtag-role="observable"
     data-semtag-state="cart.total">Total: GBP 42.00</div>

<!-- navigation -->
<a data-semtag-id="nav.orders"
   data-semtag-role="navigation"
   data-semtag-target="orders.page" href="/orders">Orders</a>

<!-- input -->
<input data-semtag-id="shipping.postcode"
       data-semtag-role="input"
       data-semtag-state="shipping.postcode" name="postcode" />

<!-- select trigger listing its options, driving a region -->
<button data-semtag-id="filters.sort"
        data-semtag-role="select"
        data-semtag-action="sort-products"
        data-semtag-controls="products.grid"
        data-semtag-options="newest|Newest;price-asc|Price: Low to High">Sort by</button>

<!-- region -->
<section data-semtag-id="checkout.form" data-semtag-role="region">...</section>
```

Appendix C

Hint Injection Validator Checks

The validator (`validate-semtags.mjs`, in `semtag-inject/scripts/` of the Hint Injector skill repository) is a static script that scans front-end source files (`.jsx`, `.tsx`, `.vue`, `.svelte`, `.html`) for `data-semtag-*` attributes and checks them against the specification in Appendix B. It runs without building or executing the application. Each finding is reported at one of three levels. Errors fail the run; warnings fail it only when the script is invoked with `-strict`; information findings never fail it. Table C.1 lists the checks with the rule name the script reports.

The validator is source-based, so the following are outside its reach: DOM ancestry, since a collection container and its items may be in different components; the uniqueness of ids built at runtime from data, since it sees only the template; and which elements render together on a page. These are the checks the Hint Overlay tool (Section 4.5) exists for.

Check	Rule	Level	Spec.
Only the seven <code>data-semtag-*</code> attributes are used	<code>unknown-attribute</code>	Error	§ 3
Every hinted element carries a <code>data-semtag-id</code>	<code>missing-id</code>	Error	§ 3
<code>data-semtag-role</code> values are among the ten roles	<code>unknown-role</code>	Error	§ 4
A hinted element has no <code>data-semtag-role</code>	<code>no-role</code>	Warning	§ 4
A select trigger has no <code>data-semtag-options</code>	<code>select-without-options</code>	Warning	§ 3
<code>data-semtag-target</code> contains a raw URL	<code>url-target</code>	Error	§ 6
An attribute value contains test wording (expected, expect, assert, oracle, pass-fail)	<code>test-intent</code>	Error	§ 1, § 6
A name segment reads as test intent (should, test, testcase)	<code>test-intent-segment</code>	Warning	§ 1, § 6
An id is positional or framework-derived	<code>positional-id</code>	Warning	§ 6
An id is a single segment rather than a dotted domain path	<code>flat-id</code>	Info	§ 6
A collection item id is keyed by array index	<code>index-keyed-item</code>	Error	§ 7
A literal id is duplicated across files	<code>duplicate-id</code>	Warning	§ 8
An item id of the form <code><prefix>.item.<key></code> has no enclosing collection container	<code>no-collection-container</code>	Warning	§ 7
An ordinary <code>data-*</code> attribute resembles a semantic hint and will not be extracted	<code>decoy-attribute</code>	Info	§ 2

Table C.1: Static checks performed by the hint-injection validator.

Appendix D

Baseline and Hinted Agent Prompts

Both arms run the same prompt template, reproduced in D.1 as it stood at the time of the experiments (commit f857a03 of the WebTester fork). Seven slots in the template, marked «NAME», are filled per arm at build time; D.2 and D.3 give the text each arm supplies for them. Everything outside the slots is identical between the arms, and the description of the Playwright **target** parameter is shared by both addressing sections. Four variables are filled per run: `$max_turns` (400 in all experiments), `$instruction`, `$server_url` and `$checklist`. A screenshot-enabled variant of each arm exists in the code and was not used in any run reported here.

D.1 Shared prompt core

```
# Role
You are an expert Quality Assurance Test Engineer specializing in automated UI/UX testing. Your task is to
  validate a web application against a provided checklist. You must systematically execute actions, verify
  results, and update the checklist status.

<<CHANNEL_INTRO>>

# Tooling

## Observing the page
<<CHANNEL_OBSERVE>>

## Acting on the page (Playwright MCP)
- Perform interactions: click, type, select, hover, drag, key presses, navigation, dialogs, etc.
- Inspect lower level when needed: `browser_evaluate` (runs JavaScript in the page), console and network tools.

### How to address an element
<<CHANNEL_ADDRESSING>>

# Execution Standards

## 1. Interaction Strategy
- Observe first: begin every screen with ONE <<OBSERVE_CALL>> to map the page, then work from what it told you.
- Act with Playwright, addressing elements as described above.
- Never re-observe the whole page to find something your last observation already reported.
<<VISUAL_POLICY>>
- Tool Use: Operate the page only through the <<CHANNEL_TOOL_USE>> Disallow the use of `Bash`, `Read`, and `
  Write` tools to operate web pages.
- Integrity: Execute all items; never skip. If an item cannot be done, mark FAIL with a concrete reason (no
  hallucination).
- Batching: For pure data entry, fill a whole form in ONE `browser_fill_form` call rather than one `browser_type
  ` call per field. For a repeated interaction (e.g. clicking the same button N times) or a bulk DOM read,
  use ONE `browser_evaluate` loop rather than many separate tool calls.
- Limited Budget: The complete agent session, including structured result recording, has a maximum of $max_turns
  agentic turns. Plan first, record each result promptly, and execute with as few browser operations as
  possible.
- Navigation: Navigate within the app by clicking links and buttons. Do not
  refresh, reload, or re-enter a URL by default because doing so may reset
  in-memory state. Exception: when a checklist item explicitly tests behavior
  after a reload, refresh, revisit, or later visit, perform that operation and
  verify the resulting state. To reload when no dedicated reload tool is
  available, navigate to the current page URL.
<<CHANNEL_EXTRA_RULES>>

## 2. Verification Logic
- Infer Action: Based on the test item description, determine the appropriate user actions needed to test.
- Infer Expected Behavior: Based on the test item description, determine what the correct/expected behavior
  should be.
```

```

- Strict Verification: After acting, re-read the affected state and compare the actual behavior of the page
  against your inferred expected behavior. Do not assume an action worked because the tool call returned
  without error.
- Pass: The feature works exactly as described.
- Fail: Any deviation (missing element, wrong text, no response, error message) is a FAIL.

## 3. Workflow
1. Initialize: Navigate to the Target URL, then take ONE <<OBSERVE_CALL>> to map the page.
2. Iterate: Go through the Checklist items.
3. Infer: Determine the action to perform and expected outcome from the description, and pick the target element
  (s) from your observation.
4. Execute: Perform the action with Playwright, addressing the element as described above.
5. Verify: Re-read the affected state and compare it to the expected outcome.
6. Record: Immediately call `mcp_structured_results_record_result` after verifying each item. Supply its
  checklist ID and PASS/FAIL verdict. For FAIL, also supply a concise issue and the observed actual behavior;
  optionally include concise evidence for either verdict. This persisted record is the source of truth and
  survives context compaction; do not postpone recording until the end.
7. Complete: Call `mcp_structured_results_get_result_progress` after recording all items, or after context
  compaction if you need to restore coverage. If it reports missing IDs, test and record those items. Once it
  reports complete, finish with a brief completion message without repeating every verdict.

# Input

## User Instruction
$instruction

## Application URL
$server_url

## Test Checklist
```markdown
$checklist
```

# Completion

```

D.2 Baseline agent prompt

```

# Role
You are an expert Quality Assurance Test Engineer specializing in automated UI/UX testing. Your task is to
  validate a web application against a provided checklist. You must systematically execute actions, verify
  results, and update the checklist status.

You perceive the page through the **Playwright MCP**'s accessibility snapshot, and act through the same MCP.

# Tooling

## Observing the page
- `browser_snapshot()` -> the accessibility tree of the current page as YAML. Every interactive node carries a
  handle of the form `[ref=eNNN]`:

```yaml
- link "Admin Panel" [ref=e461] [cursor=pointer]:
 - /url: /admin
- searchbox "Search products..." [ref=e21]
- button "Add Product" [ref=e30]
```

- The tree covers the whole page, so it is large. Take one per screen and read from it, rather than re-taking it
  to look something up again.

## Acting on the page (Playwright MCP)
- Perform interactions: click, type, select, hover, drag, key presses, navigation, dialogs, etc.
- Inspect lower level when needed: `browser_evaluate` (runs JavaScript in the page), console and network tools.

### How to address an element
The Playwright action tools (`browser_click`, `browser_type`, `browser_select_option`, `browser_fill_form`) take
  a `target` parameter, documented as *Exact target element reference from the page snapshot, or a unique
  element selector.*

Take the ref from the snapshot and pass the **bare** ref as `target` -- just the `eNNN` from inside `[ref=...]`:

- Click a link: `browser_click` with `target: "e461"`, `element: "Admin Panel link"`.
- Fill one input: `browser_type` with `target: "e21"`, `text: "laptop"`.
- Fill a whole form in ONE call: `browser_fill_form` with a `fields` list where each field's `target` is a bare
  ref.

Do **not** pass the brackets (`"[ref=e461]"`) or the whole snapshot line (`"link \"Admin Panel\" [ref=e461]"`)
  -- neither resolves. A unique CSS selector is also accepted as a `target` where you have one.

Refs are assigned per snapshot: if the page has re-rendered since you took it, take a fresh `browser_snapshot`
  rather than reusing an old ref.

```

```

# Execution Standards

## 1. Interaction Strategy
- Observe first: begin every screen with ONE `browser_snapshot` to map the page, then work from what it told you
.
- Act with Playwright, addressing elements as described above.
- Never re-observe the whole page to find something your last observation already reported.
- DOM-Only: Do NOT use screenshots or visual validation. Rely on DOM attributes (text, id, class, role, state, accessibility) for verification.
- Tool Use: Operate the page only through the **Playwright MCP**. Disallow the use of `Bash`, `Read`, and `Write` tools to operate web pages.
- Integrity: Execute all items; never skip. If an item cannot be done, mark FAIL with a concrete reason (no hallucination).
- Batching: For pure data entry, fill a whole form in ONE `browser_fill_form` call rather than one `browser_type` call per field. For a repeated interaction (e.g. clicking the same button N times) or a bulk DOM read, use ONE `browser_evaluate` loop rather than many separate tool calls.
- Limited Budget: The complete agent session, including structured result recording, has a maximum of $max_turns agentic turns. Plan first, record each result promptly, and execute with as few browser operations as possible.
- Navigation: Navigate within the app by clicking links and buttons. Do not refresh, reload, or re-enter a URL by default because doing so may reset in-memory state. Exception: when a checklist item explicitly tests behavior after a reload, refresh, revisit, or later visit, perform that operation and verify the resulting state. To reload when no dedicated reload tool is available, navigate to the current page URL.

## 2. Verification Logic
- Infer Action: Based on the test item description, determine the appropriate user actions needed to test.
- Infer Expected Behavior: Based on the test item description, determine what the correct/expected behavior should be.
- Strict Verification: After acting, re-read the affected state and compare the actual behavior of the page against your inferred expected behavior. Do not assume an action worked because the tool call returned without error.
- Pass: The feature works exactly as described.
- Fail: Any deviation (missing element, wrong text, no response, error message) is a FAIL.

## 3. Workflow
1. Initialize: Navigate to the Target URL, then take ONE `browser_snapshot` to map the page.
2. Iterate: Go through the Checklist items.
3. Infer: Determine the action to perform and expected outcome from the description, and pick the target element (s) from your observation.
4. Execute: Perform the action with Playwright, addressing the element as described above.
5. Verify: Re-read the affected state and compare it to the expected outcome.
6. Record: Immediately call `mcp_structured_results_record_result` after verifying each item. Supply its checklist ID and PASS/FAIL verdict. For FAIL, also supply a concise issue and the observed actual behavior; optionally include concise evidence for either verdict. This persisted record is the source of truth and survives context compaction; do not postpone recording until the end.
7. Complete: Call `mcp_structured_results_get_result_progress` after recording all items, or after context compaction if you need to restore coverage. If it reports missing IDs, test and record those items. Once it reports complete, finish with a brief completion message without repeating every verdict.

# Input

## User Instruction
$instruction

## Application URL
$server_url

## Test Checklist
``markdown
$checklist
```

Completion

```

## D.3 Hinted agent prompt

```

Role
You are an expert Quality Assurance Test Engineer specializing in automated UI/UX testing. Your task is to validate a web application against a provided checklist. You must systematically execute actions, verify results, and update the checklist status.

This application has been annotated with **semantic hints** (`data-semtag-*` attributes), and you have a dedicated **Semantic Hints MCP** that reads them. You perceive the page through the Semantic Hints MCP and act through the **Playwright MCP**: `observe` with Semantic Hints, act with Playwright.*

Tooling

Observing the page
- `semantic_snapshot({ "url"?, "scope"?, "includeHidden"? })` -> a compact map of the hinted elements on the current page, **grouped by role**: `navigation`, `action`, `option`, `input`, `select`, `toggle`, `slider`,

```

```

 `observable`, `region`, `collection` -- plus `other` and `collections`. The group key IS the element's
 role, so entries carry no `role` field; empty groups are omitted. Each element carries a stable `id` (its
 data-semtag-id), a `name`, and where relevant `value`, `state`, `enabled`, `target`, `controls`, `options`
 `, `visible`.
- A `select` entry's `options` field lists its `{ "value", "label" }` pairs. Each option is individually
 addressable as `folded in `collections` -- shared attributes stated once, each item one row:

```jsonc
{ "id": "products.grid",
  "idPattern": "products.grid.item.{key}[.{control}]",
  "item": { "role": "navigation" },
  "itemControls": { "price": { "role": "observable", "state": "product.price" },
    "buy": { "role": "action", "action": "buy-product" } },
  // (a folded member has no group key around it, so its role is stated here)
  "fields": [ "key", "name", "target", "price.value", "buy" ],
  "items": [ [ "alpha", "Alpha Phone", "product.detail", "GBP 10.00", true ],
    [ "beta", "Beta Laptop", "product.detail", "GBP 20.00", null ] ] }
...

```

- `fields` is the column header for `items`; read a row by zipping it against `fields`. A `null` cell means that item does not have that element -- above, `beta` has no Buy button.

- Fields in `item` / `itemControls` apply to **every** item -- they are not repeated per row.

- **Every** folded element is still individually clickable and observable. Substitute into `idPattern`: item `beta` is `[data-semtag-id='products.grid.item.beta']`, its price is `products.grid.item.beta.price`, its buy button `products.grid.item.beta.buy`.

This output is far smaller than a full accessibility tree. Both MCPs observe and control the **same** browser page, so a `semantic\_snapshot` reflects exactly what Playwright is acting on.

Acting on the page (Playwright MCP)

- Perform interactions: click, type, select, hover, drag, key presses, navigation, dialogs, etc.
- Inspect lower level when needed: `browser\_evaluate` (runs JavaScript in the page), console and network tools.

How to address an element

The Playwright action tools (`browser\_click`, `browser\_type`, `browser\_select\_option`, `browser\_fill\_form`) take a `target` parameter, documented as **Exact target element reference from the page snapshot, or a unique element selector.**

Pick the element's `id` from `semantic\_snapshot` and pass the stable selector `[data-semtag-id='<id>']` as `target` -- no `browser\_snapshot` and no element ref needed:

- Click an action: `browser\_click` with `target: "[data-semtag-id='checkout.submit']"`, `element: "Checkout submit button"`.
- Fill one input: `browser\_type` with `target: "[data-semtag-id='filters.search']"`, `text: "laptop"`.
- Fill a whole form in ONE call: `browser\_fill\_form` with a `fields` list where each field's `target` is a `[data-semtag-id='<id>']` selector.

These ids are stable across re-renders, so a selector stays valid for as long as the element is on the page.

For an element with **no** `data-semtag-id`, take a `browser\_snapshot` and pass the bare ref from `[ref=eNNN]` instead (e.g. `target: "e461"`, not `[ref=e461]` and not the whole snapshot line). Refs are assigned per snapshot, so re-snapshot if the page has re-rendered.

Execution Standards

1. Interaction Strategy

- Observe first: begin every screen with ONE `semantic\_snapshot` to map the page, then work from what it told you.
- Act with Playwright, addressing elements as described above.
- Never re-observe the whole page to find something your last observation already reported.
- DOM-Only: Do NOT use screenshots or visual validation. Rely on DOM attributes (text, id, class, role, state, accessibility) for verification.
- Tool Use: Operate the page only through the **Playwright MCP** (actions / fallback inspection) and the **Semantic Hints MCP** (observation). Disallow the use of `Bash`, `Read`, and `Write` tools to operate web pages.
- Integrity: Execute all items; never skip. If an item cannot be done, mark FAIL with a concrete reason (no hallucination).
- Batching: For pure data entry, fill a whole form in ONE `browser\_fill\_form` call rather than one `browser\_type` call per field. For a repeated interaction (e.g. clicking the same button N times) or a bulk DOM read, use ONE `browser\_evaluate` loop rather than many separate tool calls.
- Limited Budget: The complete agent session, including structured result recording, has a maximum of \$max_turns agentic turns. Plan first, record each result promptly, and execute with as few browser operations as possible.
- Navigation: Navigate within the app by clicking links and buttons. Do not refresh, reload, or re-enter a URL by default because doing so may reset in-memory state. Exception: when a checklist item explicitly tests behavior after a reload, refresh, revisit, or later visit, perform that operation and verify the resulting state. To reload when no dedicated reload tool is

```

    available, navigate to the current page URL.
- Fallback: If a checklist item concerns an element with no `data-semtag-id`, or `semantic_snapshot`/'
  semantic_observe` is insufficient, fall back to `browser_snapshot` for that item only.
- Hints are descriptive, not an oracle: `data-semtag-*` describe what the UI *is/does*; they never tell you
  whether a test passes. Always judge actual behavior against your own inferred expectation.

## 2. Verification Logic
- Infer Action: Based on the test item description, determine the appropriate user actions needed to test.
- Infer Expected Behavior: Based on the test item description, determine what the correct/expected behavior
  should be.
- Strict Verification: After acting, re-read the affected state and compare the actual behavior of the page
  against your inferred expected behavior. Do not assume an action worked because the tool call returned
  without error.
- Pass: The feature works exactly as described.
- Fail: Any deviation (missing element, wrong text, no response, error message) is a FAIL.

## 3. Workflow
1. Initialize: Navigate to the Target URL, then take ONE `semantic_snapshot` to map the page.
2. Iterate: Go through the Checklist items.
3. Infer: Determine the action to perform and expected outcome from the description, and pick the target element
  (s) from your observation.
4. Execute: Perform the action with Playwright, addressing the element as described above.
5. Verify: Re-read the affected state and compare it to the expected outcome.
6. Record: Immediately call `mcp_structured_results_record_result` after verifying each item. Supply its
  checklist ID and PASS/FAIL verdict. For FAIL, also supply a concise issue and the observed actual behavior;
  optionally include concise evidence for either verdict. This persisted record is the source of truth and
  survives context compaction; do not postpone recording until the end.
7. Complete: Call `mcp_structured_results_get_result_progress` after recording all items, or after context
  compaction if you need to restore coverage. If it reports missing IDs, test and record those items. Once it
  reports complete, finish with a brief completion message without repeating every verdict.

# Input

## User Instruction
$instruction

## Application URL
$server_url

## Test Checklist
``markdown
$checklist
```

Completion

```

## Appendix E

# Test-Case Corrections

This appendix records every change made to the WebTestBench “gold” checklists of the fourteen applications in the dataset, as described in Section 5.2. Each of the 253 original test cases was audited against the running application. Table E.1 gives, for each, the original wording, class and verdict as published, the defect the benchmark reported where the verdict was FAIL, the finding of the audit, the correction made, and a classification of the reason for the change.

The classification uses the four reasons stated in Section 5.2. *Ambiguous* covers wording that left the agent too much room for interpretation: an undefined term, an unstated scope, or an observable outcome the case did not name. *Incorrect verdict* covers cases whose published PASS or FAIL did not match the behaviour of the running application, or whose reported defect described something other than the observed failure. *Irrelevant* covers cases that referred to a feature or domain the application does not have, or that did not follow from the application’s brief. *Compound or duplicate* covers cases that stated more than one oracle, or that substantially overlapped another case.

Table E.1: Full log of test-case corrections.

| Application       | Original test case ID | Original test case                                                                                                                         | Class         | Original verdict | Original reported bug                                                                                                                                  | Audit finding                                                                                                                                                                                                                                      | Correction made                                                                                                                                                          | Classification        | Outcome         |
|-------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------------|
| WebTestBench_0001 |                       | Supports marking specific products as featured on the homepage and displaying them.                                                        | functionality | PASS             |                                                                                                                                                        | The intent is valid, but the original PASS label is incorrect for more than three featured products. A newly marked fourth product is omitted from the homepage, and the route intended to expose the complete featured set displays all products. | Required marking and unmarking to update the homepage and required visitors to be able to browse the complete featured set; changed to FAIL and documented both defects. | incorrect verdict     | verdict changed |
| WebTestBench_0001 |                       | It provides a product keyword search function, covering multi-dimensional information.                                                     | functionality | PASS             |                                                                                                                                                        | Valid but vague: 'multi-dimensional information' does not define the searchable fields.                                                                                                                                                            | Defined the searchable fields as product name, brief description, and tags, with matching visible results and result count; retained PASS.                               | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | Users can add product tags to any existing product.                                                                                        | functionality | FAIL             | Unable to update one or more tag information added to the product entry to the filter interface.                                                       | The FAIL label is valid, but the content and bug do not describe the same oracle. Tags can be added and are searchable; the reproducible defect is that newly assigned tags never become available as tag-filter choices.                          | Reframed the case around propagation of newly assigned tags into filtering; retained FAIL and replaced the bug with the runtime-confirmed behavior.                      | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | Users can create product labels.                                                                                                           | functionality | PASS             |                                                                                                                                                        | Ambiguous terminology: the app has tags, not a separate product-label entity, and 'create' could mean defining a reusable label or assigning a new tag while creating a product.                                                                   | Interpreted labels as tags and specified assigning new tag values during product creation; retained PASS.                                                                | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | Users can edit any existing product label.                                                                                                 | functionality | PASS             |                                                                                                                                                        | Ambiguous terminology and scope: tags belong to products and are edited as a comma-separated set; there is no standalone label-management feature.                                                                                                 | Replaced labels with a product's tag set and required the edited tags to be visible and searchable; retained PASS.                                                       | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | Users can categorize any existing product labels.                                                                                          | functionality | FAIL             | The inability to verify the existence or validity of the product's assigned category results in the product being classified into an invalid category. | Incorrect and internally confused: the app categorizes products, not labels. The admin UI only offers the three valid categories, so the stated invalid-category bug is not reproducible through the product workflow.                             | Changed the subject from labels to products, required one supported category and correct category placement, and changed to PASS.                                        | incorrect verdict     | verdict changed |
| WebTestBench_0001 |                       | Users can filter products by category (such as technology products, home furnishings, fitness equipment, etc.).                            | functionality | PASS             |                                                                                                                                                        | Valid and directly testable on the complete product list. A separate inconsistency exists in the category-page selector and is covered by new test 21.                                                                                             | Aligned the category names with the app and required the visible results and count to update; retained PASS.                                                             | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | Users can browse the full product list, which displays key information, brief reviews, and purchase links for each product.                | functionality | PASS             |                                                                                                                                                        | Valid, though 'key information' is underspecified and successful retailer navigation overlaps test 15.                                                                                                                                             | Enumerated the list-card fields and required a visible purchase action, leaving destination validity to test 15; retained PASS.                                          | compound or duplicate | rewritten       |
| WebTestBench_0001 |                       | Users can sort and filter products by price range or rating attributes.                                                                    | functionality | PASS             |                                                                                                                                                        | Valid but underspecified: it does not name the available sort orders or distinguish price filtering from price sorting.                                                                                                                            | Specified newest, rating, ascending price, descending price, price-range filtering, and minimum-rating filtering; retained PASS.                                         | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | Product categories are limited to three vertical sectors: technology, home furnishings, and fitness equipment.                             | constraint    | PASS             |                                                                                                                                                        | Valid, although the wording should use the category labels presented by the app.                                                                                                                                                                   | Named the supported categories as Tech Gadgets, Home Goods, and Fitness and required every product to use exactly one; retained PASS.                                    | ambiguous             | rewritten       |
| WebTestBench_0001 |                       | After a user applies filtering or sorting criteria, the product results displayed by the system must strictly match the selected criteria. | constraint    | PASS             |                                                                                                                                                        | Valid intent but 'strictly match' is ambiguous, especially when several tags are selected and the app uses any-selected-tag matching.                                                                                                              | Defined the conjunction across search, category, price, and rating, and explicitly defined tag selection as any-match; retained PASS.                                    | ambiguous             | rewritten       |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                                                                                                         | Class         | Original verdict | Original reported bug                                                                     | Audit finding                                                                                                                                                                                         | Correction made                                                                                                                                                  | Classification        | Outcome   |
|-------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0001 |             | After switching sorting/filtering criteria, the product list updates the matching results in real time.                                                                                                    | interaction   | PASS             |                                                                                           | Mostly valid but partially duplicates tests 10 and 12; 'real time' should not imply an unmeasurable zero-delay requirement because keyword search is intentionally debounced.                         | Required the list, order, and result count to update without a manual reload after each criterion change; retained PASS.                                         | compound or duplicate | rewritten |
| WebTestBench_0001 |             | Clicking on a product will redirect you to its own details page without any lag.                                                                                                                           | interaction   | PASS             |                                                                                           | The navigation check is valid, but 'without any lag' has no threshold and makes the oracle non-reproducible.                                                                                          | Scoped the interaction to selecting a product image or title and verifying the matching detail page; removed the subjective performance claim and retained PASS. | ambiguous             | rewritten |
| WebTestBench_0001 |             | The external retailer link button on the product details page redirects to the target page normally.                                                                                                       | interaction   | FAIL             | When verifying the URL format of the purchase link, clicking it results in an empty page. |                                                                                                                                                                                                       | Required a real external retailer destination and replaced the bug with the placeholder-link defect; retained FAIL.                                              | ambiguous             | rewritten |
| WebTestBench_0001 |             | The product details page fully displays reviews, specifications, and retailer links.                                                                                                                       | content       | PASS             |                                                                                           | The FAIL label is valid, but the stated bug is inaccurate. The destination is not empty: every seeded product points to an example.com documentation placeholder rather than a retailer product page. | Scoped the case to complete product records and enumerated the matching detail fields; retained PASS.                                                            | ambiguous             | rewritten |
| WebTestBench_0001 |             | Product categorization and label matching are accurate, with no mislabeling or omissions.                                                                                                                  | content       | PASS             |                                                                                           | The intent is relevant, but 'accuracy' is subjective and labels are actually tags. The case also overlaps category and tag functionality tests.                                                       | Required consistent category and tag values across list and detail views for every seeded product; retained PASS.                                                | compound or duplicate | rewritten |
| WebTestBench_0001 |             | The core content of the same product details remains consistent across different views. The detailed view displays complete and accurate conditions and final results without any truncation or data loss. | content       | PASS             |                                                                                           | Overly broad and partly nonsensical: 'conditions and final results' are not product fields, and summary views are intentionally shorter than detail pages.                                            | Defined the shared fields that must remain consistent and required the full review and specifications to be complete on the detail page; retained PASS.          | irrelevant            | rewritten |
| WebTestBench_0003 |             | Employers can create and publish job listings, fully entering core information such as name and description.                                                                                               | functionality | PASS             |                                                                                           | Valid intent, but "name" should be "title" and the case omits most listing fields named by the product brief.                                                                                         | Expanded the corrected case to cover the complete listing data and visibility in both employer and job-seeker views.                                             | ambiguous             | rewritten |
| WebTestBench_0003 |             | Employers can edit and delete published jobs to manage their positions.                                                                                                                                    | functionality | PASS             |                                                                                           | Partly valid, but the gold PASS is wrong: deletion is implemented behind a confirmation dialog, while there is no edit control, route, or state-update operation.                                     | Split deletion and editing into separate atomic cases; deletion remains PASS and editing is a new FAIL case.                                                     | incorrect verdict     | rewritten |
| WebTestBench_0003 |             | The employer dashboard allows you to view all the job listings you have posted.                                                                                                                            | functionality | PASS             |                                                                                           | Valid but slightly imprecise because the app has one fixed current employer and shows only that employer's jobs.                                                                                      | Clarified that the dashboard lists the current employer's active postings and summary counts.                                                                    | ambiguous             | rewritten |
| WebTestBench_0003 |             | The employer dashboard allows you to view a list of applications submitted by job seekers for all the positions you have posted.                                                                           | functionality | PASS             |                                                                                           | Valid intent, but "for all" could imply one aggregate list; the dashboard exposes applications separately under each job.                                                                             | Reworded the case around expanding each posted job to view its associated applications.                                                                          | ambiguous             | rewritten |
| WebTestBench_0003 |             | Job seekers can browse the platform's full list of job openings.                                                                                                                                           | functionality | PASS             |                                                                                           | Valid and directly testable.                                                                                                                                                                          | Added the visible result count and the ability to open a listing.                                                                                                | ambiguous             | rewritten |
| WebTestBench_0003 |             | The website allows users to filter job listings by criteria such as industry, location, and job type.                                                                                                      | functionality | PASS             |                                                                                           | Valid but underspecified about individual filters versus combinations and reset behavior.                                                                                                             | Specified all three supported filters plus combined filtering and clearing.                                                                                      | ambiguous             | rewritten |
| WebTestBench_0003 |             | The website supports keyword search, which allows users to retrieve job information.                                                                                                                       | functionality | PASS             |                                                                                           | Valid but vague: "retrieve job information" does not define the searchable fields or observable result.                                                                                               | Specified case-insensitive matching on job title company and skills with an updated result list and count.                                                       | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                    | Class         | Original verdict | Original reported bug                                                                                                                   | Audit finding                                                                                                                                                                                                                                                                                                       | Correction made                                                                                                                           | Classification        | Outcome   |
|-------------------|-------------|-----------------------------------------------------------------------------------------------------------------------|---------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0003 |             | Job seekers can submit an application form that includes contact information and a message on the job details page.   | functionality | PASS             |                                                                                                                                         | Valid but ambiguous about which contact fields are required and what confirms success.                                                                                                                                                                                                                              | Specified required name email and message optional phone and visible submission confirmation.                                             | ambiguous             | rewritten |
| WebTestBench_0003 |             | Job seekers must not submit job applications with missing or incorrect personal information.                          | constraint    | FAIL             | You can still submit even if the email format is incorrect.                                                                             | The reported bug is wrong: the runtime uses a required email input and blocks malformed email syntax as well as missing required name email or message. "Incorrect personal information" is otherwise too broad because the app cannot verify whether data is truthful and does not validate optional phone syntax. | Narrowed the case to required fields and email syntax and changed it to PASS.                                                             | incorrect verdict     | rewritten |
| WebTestBench_0003 |             | Job postings with incomplete information cannot be published; there is a clear notification.                          | constraint    | PASS             |                                                                                                                                         | The gold PASS is wrong and the case is vague. The runtime publishes jobs with empty industry employment type location and skills; it also accepts an inverted salary range. No clear validation message is shown for these defects.                                                                                 | Defined the required listing fields and salary-range validity explicitly and changed the case to FAIL.                                    | incorrect verdict     | rewritten |
| WebTestBench_0003 |             | Job seekers are not allowed to submit multiple applications for the same position using the same contact information. | constraint    | FAIL             | Job seekers can use the same contact information to submit multiple applications for the same position.                                 | Valid FAIL case. The runtime accepted the same name and email twice for one job and displayed both records on the dashboard.                                                                                                                                                                                        | Clarified duplicate identity as the same email for the same job and required the original application to remain unchanged.                | compound or duplicate | rewritten |
| WebTestBench_0003 |             | Job seekers cannot post job openings.                                                                                 | constraint    | FAIL             | Even after switching to a job seeker profile, the "Post Job" button, which shouldn't be there, still appears in the upper right corner. | Irrelevant and based on a nonexistent flow: the app has no accounts roles profile switch or authentication state, so it cannot identify a visitor as a job seeker. The stated reproduction is fabricated.                                                                                                           | Removed the case rather than inventing an access-control requirement that is absent from the brief and runtime.                           | irrelevant            | removed   |
| WebTestBench_0003 |             | After job seekers submit their application forms, the system receives them normally without any submission failures.  | interaction   | PASS             |                                                                                                                                         | Substantially duplicates test 8 and uses the unobservable phrase "receives them normally".                                                                                                                                                                                                                          | Turned it into an end-to-end interaction check covering success feedback and appearance under the matching job on the employer dashboard. | compound or duplicate | rewritten |
| WebTestBench_0003 |             | After switching the filtering and search criteria, the job list updates the matching results in real time.            | interaction   | PASS             |                                                                                                                                         | Valid but awkwardly worded and missing the empty-result and reset states.                                                                                                                                                                                                                                           | Specified immediate updates for search and filter changes including empty results and clearing criteria.                                  | ambiguous             | rewritten |
| WebTestBench_0003 |             | The job listings clearly display key information such as job title, location, and salary range.                       | content       | PASS             |                                                                                                                                         | Valid but incomplete relative to the information consistently shown on every card.                                                                                                                                                                                                                                  | Added company industry employment type posting date and skills.                                                                           | ambiguous             | rewritten |
| WebTestBench_0003 |             | The job details page fully displays all information, including the job description and required skills.               | content       | PASS             |                                                                                                                                         | Valid intent, but "all information" is an absolute and does not define the expected fields.                                                                                                                                                                                                                         | Enumerated the listing details that must match the selected job.                                                                          | ambiguous             | rewritten |
| WebTestBench_0003 |             | Employers can view a complete list of applications, clearly distinguishing between different job seekers.             | content       | PASS             |                                                                                                                                         | Valid but overlaps test 4 and does not define what makes an application complete or distinguishable.                                                                                                                                                                                                                | Specified the applicant name email optional phone message and submission date shown for each application.                                 | compound or duplicate | rewritten |
| WebTestBench_0004 |             | Couples can create a personal wedding planning page and enter key information such as the wedding date.               | functionality | PASS             |                                                                                                                                         | Valid intent, but "personal page" suggests user accounts that do not exist and "such as" leaves the required fields undefined.                                                                                                                                                                                      | Specified the four supported wedding-detail fields and the observable saved dashboard state.                                              | ambiguous             | rewritten |
| WebTestBench_0004 |             | New users can update information such as dates and budgets on the planning page.                                      | functionality | PASS             |                                                                                                                                         | Valid intent, but "new users" is irrelevant because there are no accounts and the editable fields are underspecified.                                                                                                                                                                                               | Specified that every supported wedding-detail field can be reopened edited saved and redisplayed.                                         | irrelevant            | rewritten |
| WebTestBench_0004 |             | The webpage allows users to filter venues/suppliers by location type, price range, and service type.                  | functionality | FAIL             | When browsing the venue list, none of the venue listings displayed specific wedding venue information (name, location, price, type).    | The gold FAIL and bug are incorrect: venue cards visibly show name location venue type and price range while the venue and vendor filters work individually and in combination.                                                                                                                                     | Clarified which filters apply to venues versus vendors and added combined filtering and reset behavior.                                   | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                   | Class         | Original verdict | Original reported bug                                                                                                                                | Audit finding                                                                                                                                                                          | Correction made                                                                                                             | Classification        | Outcome   |
|-------------------|-------------|----------------------------------------------------------------------------------------------------------------------|---------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0004 |             | Provides a keyword search function for venues/suppliers.                                                             | functionality | FAIL             | When browsing the supplier list, the price range field from the relevant wedding supplier information is not displayed in any of the supplier lists. | The gold FAIL and bug are incorrect and unrelated: both listing pages have working keyword search and vendor price-range labels are visible.                                           | Defined the searchable listing information and the observable result/count update.                                          | ambiguous             | rewritten |
| WebTestBench_0004 |             | The website allows users to tag their preferred venues/suppliers for easy access later.                              | functionality | PASS             |                                                                                                                                                      | Valid, but "tag" is imprecise and the later-access behavior is not stated.                                                                                                             | Reworded this around adding and removing venue/vendor favorites and viewing them together.                                  | ambiguous             | rewritten |
| WebTestBench_0004 |             | The website provides wedding planning tips and checklists for users to view.                                         | functionality | PASS             |                                                                                                                                                      | Partly valid: the app provides categorized checklists with explanatory task descriptions, not a separate collection of planning tips.                                                  | Made the case testable by requiring checklist browsing category filtering expansion and task descriptions.                  | ambiguous             | rewritten |
| WebTestBench_0004 |             | New users can save planning tips/lists to their personal planning page.                                              | functionality | PASS             |                                                                                                                                                      | Valid for whole checklists, but not for individual tips; "new users" and "personal planning page" are also imprecise.                                                                  | Specified saving and unsaving whole checklists and observing their dashboard preview/count.                                 | ambiguous             | rewritten |
| WebTestBench_0004 |             | The wedding budget cannot be plural.                                                                                 | constraint    | FAIL             | Unable to meet the constraint that the budget must be a positive value in the creation of the planning dashboard.                                    | The content is a meaningless mistranslation, although the bug text reveals a legitimate missing positive-budget constraint and the gold FAIL is correct for that intended requirement. | Replaced the sentence with explicit positive-number validation and rejection behavior.                                      | irrelevant            | rewritten |
| WebTestBench_0004 |             | The number of guests cannot be negative.                                                                             | constraint    | FAIL             | The restriction that the number of guests must be a non-negative integer in the creation of the plan dashboard has not been implemented.             | Valid and correctly marked FAIL, but it checks only negativity while the bug also identifies the whole-number requirement.                                                             | Expanded the constraint to reject both negative and fractional guest counts without overwriting the saved value.            | ambiguous             | rewritten |
| WebTestBench_0004 |             | Tagged favorite options and saved planning content are persistently stored so they are not lost when accessed again. | constraint    | PASS             |                                                                                                                                                      | Valid, but "accessed again" does not define whether navigation or a page reload is required and "planning content" is vague.                                                           | Specified persistence of favorite venues/vendors and saved checklists across navigation and reload.                         | ambiguous             | rewritten |
| WebTestBench_0004 |             | The wedding date cannot be an invalid date in the past.                                                              | constraint    | FAIL             | The validation function requiring the wedding date to be a valid future date (not a past date) cannot be implemented.                                | Valid and correctly marked FAIL, but the wording conflates invalid formatting with a valid date that is in the past and the bug explanation is malformed.                              | Clarified the intended future-date constraint and its rejection behavior.                                                   | ambiguous             | rewritten |
| WebTestBench_0004 |             | When a user clicks the "View Details" button, they are smoothly redirected to the details page.                      | interaction   | FAIL             | Clicking the button does not redirect.                                                                                                               | The functional failure is real, but "smoothly" is subjective and the case covers only venue details even though vendor profile buttons fail identically.                               | Generalized the requirement to opening complete details for any venue or vendor.                                            | ambiguous             | rewritten |
| WebTestBench_0004 |             | Planning Tips / The checklist can be marked as completed normally, and the status is updated in real time.           | interaction   | PASS             |                                                                                                                                                      | Valid for checklist tasks, but the sentence is malformed and "normally"/"in real time" are vague.                                                                                      | Specified marking and unmarking a task immediate checklist and dashboard progress updates and persistence.                  | ambiguous             | rewritten |
| WebTestBench_0004 |             | After switching the filter/search criteria, the venue/supplier list is updated in real time.                         | interaction   | PASS             |                                                                                                                                                      | Valid but vague about the observable update and substantially overlaps the filter and search functionality cases.                                                                      | Focused it on immediate visible results/count changes without submission plus restoring the full list when cleared.         | compound or duplicate | rewritten |
| WebTestBench_0004 |             | The personal planning page fully displays all key wedding information, with no content missing.                      | content       | PASS             |                                                                                                                                                      | Overbroad and untestable because "all key" and "no content missing" do not identify the expected information.                                                                          | Enumerated the saved wedding details planning progress favorite counts and saved-checklist previews shown on the dashboard. | ambiguous             | rewritten |
| WebTestBench_0004 |             | The venue/supplier information includes key details (location, price, services offered).                             | content       | FAIL             | The specific price cannot be displayed correctly.                                                                                                    | Partly valid, but it ambiguously asks for a specific price although the brief and data model use price ranges; the gold bug wrongly claims those visible ranges are missing.           | Defined the concrete summary information available for every venue and vendor and changed the corrected result to PASS.     | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                          | Class         | Original verdict | Original reported bug                                                                                        | Audit finding                                                                                                                                                                                                                                                                                                     | Correction made                                                                                                             | Classification        | Outcome   |
|-------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------|---------------|------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0004 |             | Planning tips / The list is comprehensive and has practical reference value.                                                | content       | PASS             |                                                                                                              | Subjective and non-falsifiable: "comprehensive" and "practical reference value" have no objective oracle.                                                                                                                                                                                                         | Replaced the value judgment with measurable category coverage task counts and explanatory descriptions.                     | ambiguous             | rewritten |
| WebTestBench_0004 |             | Filter/search results with precise matching criteria, displaying results without errors or omissions.                       | content       | PASS             |                                                                                                              | Vague and absolute: the matching rules are undefined and proving no omissions for arbitrary input is not a bounded test.                                                                                                                                                                                          | Specified AND-combination semantics accurate result counts a no-match state and reset behavior.                             | ambiguous             | rewritten |
| WebTestBench_0005 |             | Users can create blank resumes.                                                                                             | functionality | PASS             |                                                                                                              | Valid and correctly labelled, although a new resume has a default title, template, and section structure rather than being literally empty.                                                                                                                                                                       | Clarified that the new resume has blank user-content fields and opens ready for editing.                                    | ambiguous             | rewritten |
| WebTestBench_0005 |             | Users can select preset templates to create professional resumes.                                                           | functionality | PASS             |                                                                                                              | Valid and correctly labelled, but templates are selected while editing rather than as part of the initial creation action.                                                                                                                                                                                        | Specified the three available styles and required the previewed content to remain intact when switching styles.             | ambiguous             | rewritten |
| WebTestBench_0005 |             | Users can enter and edit personal information (contact information, work experience, and other core items).                 | functionality | PASS             |                                                                                                              | Valid and correctly labelled, but "other core items" is vague and omits several major editor areas.                                                                                                                                                                                                               | Enumerated contact details, summary, work experience, education, and skills, including management of repeated entries.      | ambiguous             | rewritten |
| WebTestBench_0005 |             | Supports customizable resume layout (chapter order and title modification).                                                 | functionality | FAIL             | Unable to customize the layout order                                                                         | Mixed and incorrectly diagnosed. Section reordering and visibility both work and update the preview. Heading changes work for most headings, but the contact heading is ignored by every template and the Minimal template omits the summary heading, so one combined PASS/FAIL label obscures the real behavior. | Split layout ordering/visibility into a PASS case and exposed-heading propagation into a focused FAIL case.                 | compound or duplicate | rewritten |
| WebTestBench_0005 |             | It provides a resume preview function, allowing users to see the edited result.                                             | functionality | PASS             |                                                                                                              | Valid and correctly labelled, but it does not say whether the preview can be shown or hidden or what changes it must display.                                                                                                                                                                                     | Required a showable/hideable preview that reflects the active resume and its selected template.                             | ambiguous             | rewritten |
| WebTestBench_0005 |             | Supports copying and renaming existing resume versions.                                                                     | functionality | PASS             |                                                                                                              | The core actions work, but the case combines two operations and misses title validation: the dashboard rename dialog accepts an empty title.                                                                                                                                                                      | Kept copy/rename coverage, required the copy to be independent, and added a separate FAIL case for empty titles.            | ambiguous             | rewritten |
| WebTestBench_0005 |             | It can save multiple resume versions to suit different job search scenarios.                                                | functionality | PASS             |                                                                                                              | Correct only within the current page session. Multiple versions can coexist and be edited independently, but none are durably saved across a reload; persistence is covered separately by test 11.                                                                                                                | Scoped this case to creating and managing multiple distinct variations in the current session.                              | ambiguous             | rewritten |
| WebTestBench_0005 |             | Supports filtering and searching saved resumes by title/creation date.                                                      | functionality | FAIL             | Unable to search web pages by creation date                                                                  | Partly valid and broadly correctly labelled, but the bug wording is inaccurate. Title search and newest/oldest creation-date sorting work; there is no way to search or filter by a creation date or range.                                                                                                       | Made title search plus creation-date ordering a PASS case and added a focused FAIL case for creation-date filtering/search. | ambiguous             | rewritten |
| WebTestBench_0005 |             | When adding past work or education experience, the time period must be a valid time frame and there should be no conflicts. | constraint    | FAIL             | There are no restrictions on whether work experience or educational practice ends later than the start date. | Valid concern and correct FAIL direction, but "no conflicts" is ambiguous because concurrent work or study can be legitimate. The runtime accepts an end month before the start month and also accepts future months without feedback.                                                                            | Narrowed the rule to chronological validity within each entry and to rejecting future dates for past entries.               | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                              | Class         | Original verdict | Original reported bug                                                                             | Audit finding                                                                                                                                                                                                                                                                   | Correction made                                                                                                                                        | Classification        | Outcome   |
|-------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0005 |             | When adding past work or education experience, you must use the correct time format; otherwise, you will not be able to add it. | constraint    | FAIL             | The available working hours are much greater than the current time.                               | Vague and incorrectly diagnosed. There is no submit-to-add validation step: an empty entry is added first and edited inline. Native month controls enforce a month/year value shape, while the reported bug describes future dates rather than format.                          | Replaced it with a test for consistent month/year dates and the defined current-position behavior; future and reversed dates remain covered by test 9. | ambiguous             | rewritten |
| WebTestBench_0005 |             | The saved resume version is persistently stored and will not be lost upon re-access.                                            | constraint    | FAIL             | Cannot be persistently saved                                                                      | Valid FAIL case and accurate bug. All state lives in React memory; reloading restores the three seeded resumes and discards creations, edits, copies, renames, and deletions.                                                                                                   | Defined re-access as reloading the application and enumerated the resume operations that must persist.                                                 | ambiguous             | rewritten |
| WebTestBench_0005 |             | The preview function synchronizes edited content in real time, with no delay or display errors.                                 | interaction   | PASS             |                                                                                                   | Valid and correctly labelled, though "no delay or display errors" is an unbounded absolute. Runtime edits to fields, template, order, and visibility update immediately.                                                                                                        | Specified the observable editor changes that must appear without a manual refresh or save action.                                                      | ambiguous             | rewritten |
| WebTestBench_0005 |             | The changes to chapter order and titles take effect immediately, and the operation is smooth.                                   | interaction   | PASS             |                                                                                                   | Only partly correct. Reordering and visibility changes take effect immediately, but not every editable heading is rendered by every template, so the gold PASS is too broad.                                                                                                    | Limited this interaction case to order and visibility, and moved heading propagation to a separate FAIL case.                                          | ambiguous             | rewritten |
| WebTestBench_0005 |             | Users can save their resumes and provide feedback after successful saving.                                                      | interaction   | FAIL             | There is no save button, and no feedback is given when saving is successful.                      | Incorrect requirement and diagnosis. The product uses automatic in-session updates, so the absence of an explicit Save button is not itself a defect. Live preview and the dashboard's updated date provide observable feedback, while durable persistence fails under test 11. | Reframed the case around automatic in-session retention across editor/dashboard navigation and visible update feedback, and changed it to PASS.        | incorrect verdict     | rewritten |
| WebTestBench_0005 |             | The resume template is displayed in its entirety, with no layout issues.                                                        | content       | PASS             |                                                                                                   | Relevant but too subjective and absolute. The seeded resumes render their visible populated sections in each template and the preview area is scrollable, but "no layout issues" has no defined oracle.                                                                         | Defined completeness as rendering every visible populated section in the selected template with readable, scrollable content.                          | ambiguous             | rewritten |
| WebTestBench_0005 |             | The entered personal information was completely retained, with no fields missing.                                               | content       | PASS             |                                                                                                   | Valid within the current session, but the past tense and "completely" are vague and could conflict with the known reload-loss defect.                                                                                                                                           | Enumerated the personal-information areas and scoped retention to tab changes, dashboard navigation, and reopening before reload.                      | ambiguous             | rewritten |
| WebTestBench_0005 |             | Resume information for multiple versions is stored independently, and there is no content sharing between versions.             | content       | PASS             |                                                                                                   | Valid and correctly labelled. Runtime testing confirmed that editing a duplicated resume's contact name leaves the original unchanged.                                                                                                                                          | Specified editing a duplicate and checking that the source version remains unchanged.                                                                  | compound or duplicate | rewritten |
| WebTestBench_0005 |             | The resume title should remain consistent across multiple pages.                                                                | content       | PASS             |                                                                                                   | Valid intent and correctly labelled, but the app is a single-page application with dashboard and editor views rather than multiple pages.                                                                                                                                       | Required the renamed title to match in the editor header and dashboard card.                                                                           | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can create financial records by writing text input fields.                                                                | functionality | PASS             |                                                                                                   | Vague; implementation-specific; overstates the app's record types.                                                                                                                                                                                                              | Reworded to state the information a financial transaction can record; successful creation is covered separately by corrected test 6.                   | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can view the financial dashboard.                                                                                         | functionality | FAIL             | The constructed line chart in the report is unreasonable; all numbers on the vertical axis are 0. | Conflates dashboard access with a report-chart defect; the dashboard is viewable.                                                                                                                                                                                               | Corrected test 2 checks dashboard content and passes. The real small-value tick-label defect is isolated as new FAIL test 22.                          | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                                         | Class         | Original verdict | Original reported bug                                                                  | Audit finding                                                                                                                 | Correction made                                                                                                                                  | Classification        | Outcome   |
|-------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0006 |             | Users can search for financial information by entering keywords.                                                                           | functionality | PASS             |                                                                                        | Vague about page, searchable fields, and expected result.                                                                     | Specified keyword search over transaction descriptions and references.                                                                           | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can view and print basic financial summary tables.                                                                                   | functionality | PASS             |                                                                                        | Conflates viewing tables with invoking the browser print flow.                                                                | Corrected test 4 checks year-specific tax-summary tables; corrected test 19 checks Print Report separately.                                      | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can filter the product list by product category (such as expenditure, date, transaction).                                            | functionality | FAIL             | The tax and reporting screening types differ; the latter is based on annual screening. | Incorrect entity and incoherent criteria; month-based Reports and year-based Tax Summary are intentionally different views.   | Rewritten to cover filtering financial transactions by type, category, and date range.                                                           | incorrect verdict     | rewritten |
| WebTestBench_0006 |             | Users can create new financial records.                                                                                                    | functionality | PASS             |                                                                                        | Valid but underspecified and partly duplicates test 1.                                                                        | Made the observable postcondition explicit: a valid transaction is retained with the supplied values.                                            | compound or duplicate | rewritten |
| WebTestBench_0006 |             | Users can manage the financial records they have created.                                                                                  | functionality | PASS             |                                                                                        | Too vague to execute reliably.                                                                                                | Specified that existing transactions can be updated or deleted and that the records reflect those changes.                                       | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can customize account categories or accounts.                                                                                        | functionality | PASS             |                                                                                        | Ambiguous disjunction; categories are customizable, accounts are not exposed for customization.                               | Narrowed to supported category create/edit/delete behavior.                                                                                      | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can categorize the financial records they create.                                                                                    | functionality | PASS             |                                                                                        | Valid but vague; it omits the type-dependent category choices and displayed outcome.                                          | Specified compatible category assignment and retention; missing-category validation is added as FAIL test 23.                                    | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users can view and adjust various items.                                                                                                   | functionality | PASS             |                                                                                        | Untestably vague and duplicates transaction/category management cases.                                                        | Removed from the corrected suite.                                                                                                                | compound or duplicate | removed   |
| WebTestBench_0006 |             | Users can mark projects as tax-related.                                                                                                    | functionality | PASS             |                                                                                        | Wrong entity: the app marks transactions, not projects.                                                                       | Reworded so transactions can be marked or unmarked as tax-related when created or updated.                                                       | ambiguous             | rewritten |
| WebTestBench_0006 |             | After a user applies filtering or sorting criteria, the product results displayed by the system must strictly match the selected criteria. | constraint    | PASS             |                                                                                        | Wrong entity and mentions sorting that the UI does not offer; the matching constraint itself is useful.                       | Rewritten for the intersection of transaction search, type, category, and date filters.                                                          | ambiguous             | rewritten |
| WebTestBench_0006 |             | Users cannot create financial records without any input.                                                                                   | constraint    | PASS             |                                                                                        | Vague about which inputs are required.                                                                                        | Specified that transactions missing a description or amount cannot be created and that the missing information is identified.                    | ambiguous             | rewritten |
| WebTestBench_0006 |             | When adding a new financial record, the selected time period cannot be a future time.                                                      | constraint    | FAIL             | For future time                                                                        | Unjustified by the product brief; future-dated bookkeeping entries can be legitimate, and 'time period' is the wrong control. | Removed rather than encoding an unsupported business rule.                                                                                       | ambiguous             | removed   |
| WebTestBench_0006 |             | The start time of the filtering dates must be earlier than the end time.                                                                   | constraint    | FAIL             | You can still choose if the start time is later than the end time.                     | Valid intent, but terminology and failure description are imprecise.                                                          | Retained as a precise FAIL case requiring validation when From is later than To.                                                                 | ambiguous             | rewritten |
| WebTestBench_0006 |             | Dashboard information changes in real time when financial information is added, edited, or deleted.                                        | interaction   | PASS             |                                                                                        | Broad and ambiguous about persistence versus same-session reactive updates.                                                   | Scoped to same-session updates of financial summaries, reports, and recent activity.                                                             | ambiguous             | rewritten |
| WebTestBench_0006 |             | When a user marks a project as tax-related, the corresponding page should provide feedback.                                                | interaction   | FAIL             | The user did not mark the tax operation.                                               | Wrong entity and incorrect oracle; a transaction's tax status and annual summary inclusion both reflect the marking.          | Rewritten to verify the recorded tax status and annual-summary inclusion; reviewed result is PASS.                                               | ambiguous             | rewritten |
| WebTestBench_0006 |             | When users create and manage financial records, custom accounts show the already assigned records.                                         | interaction   | PASS             |                                                                                        | Unsupported feature: transactions cannot be assigned to accounts and there is no custom-account page.                         | Replaced with the supported equivalent: category cards update transaction counts and totals for assigned records.                                | ambiguous             | rewritten |
| WebTestBench_0006 |             | Click "Print Report," and the printed report will appear in the document selection menu.                                                   | interaction   | FAIL             | Clicking "Save" has no effect.                                                         | Incorrect oracle and mismatched action; the app has no Save action and native print-dialog wording is platform-specific.      | Rewritten as the general requirement that users can print the selected year's tax summary without an application error; reviewed result is PASS. | incorrect verdict     | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                                                                 | Class         | Original verdict | Original reported bug                                                                             | Audit finding                                                                                                                                                                                    | Correction made                                                                                                                                                                              | Classification        | Outcome   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0006 | Users can view their spending and expenditure details.                                                                                                                                                | content       | PASS             |                                                                                                   | Tautological and underspecified.                                                                                                                                                                 | Specified the transaction information and aggregate totals users can review.                                                                                                                 | ambiguous             | rewritten |
| WebTestBench_0006 | The information on the financial dashboard must match the actual financial information entered.                                                                                                       | content       | PASS             |                                                                                                   | Overbroad: Total Balance comes from static account seed data, while transaction-derived cards use only the current month.                                                                        | Scoped to current-month income, expenses, net profit/loss, recent transactions, and matching monthly-report totals.                                                                          | ambiguous             | rewritten |
| WebTestBench_0010 | Users can browse the restaurant list and view detailed information for each restaurant, including cuisine type and average price.                                                                     | functionality | FAIL             | The restaurant list is missing essential information about dishes and prices.                     | The requirement is valid, but the gold bug is partly irrelevant: dishes were never required. The app shows cuisine and symbolic price tiers, not the required average price per diner.           | Retained as a precise FAIL case covering restaurant information and the missing average price.                                                                                               | irrelevant            | rewritten |
| WebTestBench_0010 | Users can search for restaurants by entering a text field.                                                                                                                                            | functionality | PASS             |                                                                                                   | Valid but vague about which restaurant fields are searchable and what the results must do.                                                                                                       | Specified search by name, cuisine or neighborhood, its combination with the available filters, and the matching-results oracle.                                                              | ambiguous             | rewritten |
| WebTestBench_0010 | Users can create reservations for restaurants by selecting the date, time, and number of diners, and then submitting the reservation after confirming the information.                                | functionality | FAIL             | An error occurred on the page when selecting the number of people.                                | Valid intent, but the reported page error is not reproducible: party-size selection works, and the creation path stores a confirmed reservation.                                                 | Specified the required booking and contact information plus the observable confirmation and upcoming-reservation result; reviewed result is PASS.                                            | incorrect verdict     | rewritten |
| WebTestBench_0010 | Users can view their upcoming booking list and cancel or modify their bookings within this list.                                                                                                      | functionality | FAIL             | Page error                                                                                        | Valid but combines several behaviors and gives no useful failure description. The reservation list and cancellation flow are implemented; modification coverage is isolated in corrected test 8. | Scoped this case to viewing and canceling an upcoming reservation, including its resulting cancelled state; reviewed result is PASS.                                                         | ambiguous             | rewritten |
| WebTestBench_0010 | Users can rate and review the restaurants they have visited. The ratings and reviews will be linked to the corresponding restaurants and displayed correctly.                                         | functionality | PASS             |                                                                                                   | Conflates review submission, visit eligibility, restaurant linkage, and rating-summary accuracy. The gold PASS hides both missing visit enforcement and inconsistent rating summaries.           | Corrected test 5 covers submitting and displaying a restaurant-linked review; corrected test 10 covers visit eligibility and duplicates; new FAIL test 17 covers rating-summary consistency. | compound or duplicate | rewritten |
| WebTestBench_0010 | During the booking process, users can selectively enter discount coupon codes and enjoy corresponding price reductions.                                                                               | functionality | FAIL             | Unable to verify if a user is using an invalid or expired coupon code; page error during booking. | Overstates the observable price effect because the app has no booking total, while the claimed page error is not reproducible: invalid codes receive an explicit error.                          | Reworded to cover applying or removing a supported coupon, displaying its discount, and retaining the applied code with the reservation; reviewed result is PASS.                            | incorrect verdict     | rewritten |
| WebTestBench_0010 | When verifying booking times, the system does not allow users to book past dates and times.                                                                                                           | functionality | PASS             |                                                                                                   | Valid constraint but imprecise about date selection and the source of available time slots.                                                                                                      | Specified that past dates are unavailable and that future reservations use the restaurant's available time choices.                                                                          | ambiguous             | rewritten |
| WebTestBench_0010 | When a user attempts to modify their booking, the system can correctly process the modification request and update the booking information while maintaining the accuracy of the booking information. | functionality | PASS             |                                                                                                   | Valid but circular and underspecified. The gold PASS overlooks that a reservation date cannot be changed at all.                                                                                 | Specified date, time, and party-size modification with persisted results; corrected result is FAIL because the modify flow omits the date.                                                   | ambiguous             | rewritten |
| WebTestBench_0010 | Users cannot perform a search without entering any content.                                                                                                                                           | constraint    | PASS             |                                                                                                   | Incorrect and unjustified. Search is live filtering rather than a submitted operation, and an empty query correctly restores all restaurants.                                                    | Removed from the corrected suite.                                                                                                                                                            | incorrect verdict     | removed   |
| WebTestBench_0000 | Users cannot leave comments on the same order repeatedly.                                                                                                                                             | constraint    | FAIL             | You can leave a review for the same order.                                                        | Uses the wrong entity ('order') and never defines user identity or duplicate equivalence, but it points toward the brief's valid visited-restaurant restriction.                                 | Rewritten so only a completed reservation can authorize one review of its restaurant; retained as a precise FAIL case.                                                                       | compound or duplicate | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                                                                                                                                   | Class         | Original verdict | Original reported bug                                                                                                                                  | Audit finding                                                                                                                                                                                         | Correction made                                                                                                                                                             | Classification        | Outcome   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0010 | Discount coupons must be used within their validity period and comply with the applicable rules. Only one basic discount coupon can be used per order; they cannot be combined.                                                                                         | constraint    | PASS             |                                                                                                                                                        | Valid intent, but the gold PASS is wrong. WEEKEND20 is accepted on a weekday, WELCOME10 has no first-reservation check, and GROUP15 remains applied after the party becomes ineligible.               | Specified eligibility at application and after booking changes, plus the one-coupon limit; corrected result is FAIL.                                                        | incorrect verdict     | rewritten |
| WebTestBench_0020 | Restaurant average prices, opening hours, and other information are maintained by the merchants; users can only view them, not edit them. Reservations cannot exceed the restaurant's maximum capacity per table.                                                       | constraint    | PASS             |                                                                                                                                                        | Combines unsupported merchant administration with an undefined capacity rule. The app exposes neither opening hours nor merchant editing nor per-table capacity data.                                 | Removed rather than inventing untestable merchant and capacity requirements. Average-price coverage remains in corrected test 1.                                            | ambiguous             | removed   |
| WebTestBench_0030 | After selecting the date, time, and number of people, users can select discount coupons, and the system will calculate the discounted price in real time. After clicking submit, the system will verify the validity of the information and provide the booking result. | constraint    | PASS             |                                                                                                                                                        | Conflates coupon behavior, live price calculation, form validation, and submission. There is no reservation price or discounted total to calculate, and coupon behavior is already covered elsewhere. | Rewritten as a focused required-field validation case; coupon application and eligibility remain in corrected tests 6 and 11.                                               | ambiguous             | rewritten |
| WebTestBench_0040 | Users can click on their booking record in their personal center and choose "Modify" or "Cancel". When modifying, users need to re-select information and submit; when canceling, a confirmation pop-up will appear.                                                    | interaction   | FAIL             | An error occurred on the information modification page.                                                                                                | Mostly valid interaction intent, but there is no separate personal center or modification page, and the claimed error is not supported by the dialog-based implementation.                            | Reworded at the feature level to require an editable upcoming reservation and cancellation confirmation; reviewed result is PASS.                                           | ambiguous             | rewritten |
| WebTestBench_0050 | When the booking status changes, the page refreshes in real time and a pop-up notification appears. You can also view historical notifications through the message center.                                                                                              | interaction   | FAIL             | After submitting the reservation information, the page displayed an error, and the reservation information did not appear on the personal center page. | Mixes immediate feedback with an unsupported message center. The claimed submission error conflicts with the implemented confirmation and persisted reservation state.                                | Removed the message-center requirement and retained immediate confirmation plus same-session reservation-state updates; reviewed result is PASS.                            | ambiguous             | removed   |
| WebTestBench_0060 | Users can view their comment history, applications, and completed order information in their personal center.                                                                                                                                                           | content       | FAIL             | No comment history information is displayed.                                                                                                           | Mostly irrelevant: applications, orders, comment history, and a general personal center are not part of the product brief. Past and cancelled reservation history is supported and relevant.          | Replaced with a precise content case for separating upcoming reservations from past or cancelled reservations and displaying their booking details.                         | irrelevant            | rewritten |
| WebTestBench_0019 | Users can successfully register new accounts and log in to the community forum, and their login status is maintained correctly.                                                                                                                                         | functionality | PASS             |                                                                                                                                                        | Valid feature area but compound and vague about the duration of login status. The gold PASS hides that passwords are never checked and all account and session state disappears on reload.            | Corrected test 1 covers the same-session registration and login happy path. Incorrect-password rejection and reload persistence are isolated as new FAIL tests 17 and 18.   | compound or duplicate | rewritten |
| WebTestBench_0019 | Registered users can create or edit their profiles, including personal introductions and hobbies, and the information will display correctly after being saved.                                                                                                         | functionality | PASS             |                                                                                                                                                        | Mostly valid, but 'create' is misleading because registration creates the profile automatically, and the visible postcondition is underspecified.                                                     | Specified editing the user's bio and interests and verifying the saved values on both the private and public profile views.                                                 | ambiguous             | rewritten |
| WebTestBench_0019 | Logged-in users can create new discussion threads in different discussion categories, including titles and content, and successfully publish them.                                                                                                                      | functionality | PASS             |                                                                                                                                                        | Valid but vague about how successful publication is observed.                                                                                                                                         | Specified category, title, and body creation plus appearance on the discussion page and in the chosen category.                                                             | ambiguous             | rewritten |
| WebTestBench_0019 | Users can post replies to any discussion thread, and the replies will be displayed in the thread's reply list in real time.                                                                                                                                             | functionality | PASS             |                                                                                                                                                        | Overbroad about unauthenticated users and uses ambiguous 'real time' wording. The gold PASS also overlooks contradictory seeded reply totals.                                                         | Scoped the behavior to logged-in users and immediate same-session display with a one-step count increase. Absolute reply-total consistency is isolated as new FAIL test 19. | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                   | Class         | Original verdict | Original reported bug             | Audit finding                                                                                                                                                                                                                            | Correction made                                                                                                                                                                                  | Classification        | Outcome   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0019 | Users can like or vote on posts or replies made by other users, and the count of likes/votes is updated in real time.                                   | functionality | PASS             |                                   | Valid but combines threads and replies, does not define repeat voting, and uses ambiguous 'real time' wording.                                                                                                                           | Specified that an authenticated user can toggle one upvote on another user's discussion or reply and immediately see the count and selected state change.                                        | ambiguous             | rewritten |
| WebTestBench_0019 | Users can use the keyword search function to find posts, and the search results accurately match the title and body content.                            | functionality | FAIL             | No search function                | The requirement is valid and the FAIL label is reproducible, but the bug description is wrong: a search field and filtering code exist. Submitting the visible field leaves the full discussion list unchanged.                          | Retained as a precise FAIL case for case-insensitive title/body search and replaced the bug description with the observed non-filtering behavior.                                                | ambiguous             | rewritten |
| WebTestBench_0019 | Users can filter the post list by discussion category, displaying only posts from the selected category.                                                | functionality | PASS             |                                   | Valid but uses 'posts' ambiguously and omits the category-label oracle.                                                                                                                                                                  | Specified that a selected category shows only discussions assigned to it and that changing categories changes the result set accordingly.                                                        | ambiguous             | rewritten |
| WebTestBench_0019 | The forum homepage or specific sections can display a list of the most active discussions, sorted by recent interaction frequency.                      | functionality | PASS             |                                   | Partly vague and partly unsupported: Most Active is deterministically sorted by total reply count, not by a defined recent interaction window.                                                                                           | Corrected test 8 verifies the implemented reply-count ordering. The missing recent-period requirement from the brief is isolated as new FAIL test 20.                                            | ambiguous             | rewritten |
| WebTestBench_0019 | Users can view their entire posting and reply history.                                                                                                  | functionality | FAIL             | Unable to view my own replies     | Irrelevant to the product brief and compound. Profiles show authored discussions but there is no requirement for a complete personal history or reply-history view.                                                                      | Removed from the corrected suite rather than adding an unsupported history feature.                                                                                                              | irrelevant            | removed   |
| WebTestBench_0009 | When browsing posts, users can view the poster's public profile information.                                                                            | functionality | PASS             |                                   | Valid but underspecified about entry points and visible profile information.                                                                                                                                                             | Specified navigation from discussion or reply authors to public username, bio, interests, join date, reputation, and authored discussions.                                                       | ambiguous             | rewritten |
| WebTestBench_0019 | After a user edits a published post or reply, the page will display an "Edited" mark or the last edit time.                                             | functionality | PASS             |                                   | Incorrect and irrelevant. The app provides no discussion or reply editing, and editing published content was not requested in the product brief.                                                                                         | Removed from the corrected suite rather than inventing editing and edit-marker requirements.                                                                                                     | incorrect verdict     | removed   |
| WebTestBench_0029 | The forum supports basic rich text editing features, such as bolding, links, and lists, and renders them correctly after posting.                       | functionality | FAIL             | Unsupported rich text editing     | The reported limitation is accurate, but rich-text authoring was not requested in the product brief and the case is therefore irrelevant to this app's required behavior.                                                                | Removed from the corrected suite.                                                                                                                                                                | irrelevant            | removed   |
| WebTestBench_0039 | When a user logs out and logs back in, their profile, posting history, and interaction status are correctly restored.                                   | functionality | PASS             |                                   | Valid persistence intent but vague and compound; 'posting history' is undefined. State is restored only while the current page instance remains loaded and is lost on reload.                                                            | Scoped the case to same-session restoration of profile details, authored discussions, replies, and votes after logout/login. Cross-reload persistence is covered separately by new FAIL test 18. | compound or duplicate | rewritten |
| WebTestBench_0049 | When a user attempts to post a thread with an empty title or empty body, the system will block the submission and provide a warning.                    | constraint    | PASS             |                                   | Valid but incomplete because category is also required and the warning should identify the missing information.                                                                                                                          | Expanded the validation to missing category, title, or body and required visible feedback without creating a discussion.                                                                         | ambiguous             | rewritten |
| WebTestBench_0059 | The system can prevent duplicate submissions of the same content (such as repeatedly and rapidly posting the same reply).                               | constraint    | FAIL             | You can publish the same function | Unsupported and underspecified. Duplicate equivalence, time window, and desired handling are undefined, and duplicate prevention is not part of the product brief.                                                                       | Removed from the corrected suite.                                                                                                                                                                | compound or duplicate | removed   |
| WebTestBench_0069 | The forum homepage or specific sections can display a list of recently popular discussions, sorted by the number of likes, replies, or activity levels. | content       | FAIL             | Popular list sorted randomly      | The gold bug is wrong: the lists are deterministic. Trending sorts by upvotes and Most Active sorts by reply count. The case is also ambiguous because it offers three alternative ranking metrics and says 'recently' without a period. | Rewritten as a focused PASS case for Trending order by upvote count. Recent-window semantics remain in new FAIL test 20.                                                                         | incorrect verdict     | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                         | Class         | Original verdict | Original reported bug                                                                                               | Audit finding                                                                                                                                                                                                               | Correction made                                                                                                                                         | Classification        | Outcome   |
|-------------------|-----------------------------------------------------------------------------------------------|---------------|------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0023 | Sellers can create product listings that include photos, descriptions, prices, and locations. | functionality | FAIL             | Users cannot upload photos themselves; photos can only be randomly generated when creating new product information. | Valid requirement and FAIL label, but the bug description is inaccurate: the photos are not randomly generated; the UI can only attach up to four fixed stock images.                                                       | Specified that sellers must be able to attach their own photos and retain all submitted listing details; kept as a precise FAIL case.                   | ambiguous             | rewritten |
| WebTestBench_0023 | Sellers can update the details of existing product information.                               | functionality | FAIL             | Unable to update the details of existing product information                                                        | Valid but vague. The gold failure is reproducible: seller listings expose view, availability, and delete actions but no way to edit listing details.                                                                        | Specified the editable fields and persisted outcome; kept as a FAIL case.                                                                               | ambiguous             | rewritten |
| WebTestBench_0023 | Sellers can update the information of the products they own.                                  | functionality | PASS             |                                                                                                                     | Duplicates test 2 and its gold PASS contradicts the app, which has no detail-editing flow. The brief separately requires sellers to remove listings that are no longer available.                                           | Repurposed this duplicate to cover marking an owned listing sold or available and deleting it, which the app supports.                                  | compound or duplicate | rewritten |
| WebTestBench_0023 | The shopping cart cannot contain items that have been removed or are no longer available.     | functionality | FAIL             | When I log out and then come back, my shopping cart is empty and I can't verify it.                                 | Valid availability intent, but the reported logout behavior is unrelated and does not establish a defect. The single-session demo cannot reliably reproduce a second seller changing an item already held by a buyer.       | Reframed the case around the observable requirement that sold or deleted listings are no longer offered for new buyer actions; reviewed result is PASS. | ambiguous             | rewritten |
| WebTestBench_0023 | Users can register new accounts                                                               | functionality | PASS             |                                                                                                                     | Valid but underspecified about required account data and the successful result.                                                                                                                                             | Specified name, unique email, password, location, confirmation, and signed-in state.                                                                    | ambiguous             | rewritten |
| WebTestBench_0023 | Buyers can filter and sort products by category, price range, condition, and distance.        | functionality | PASS             |                                                                                                                     | Valid, but the gold PASS is wrong. The price range renders only one adjustable bound, and new listings have no distance and are treated as zero miles away.                                                                 | Specified combined filters, available-listing scope, supported sorts, and the matching-results oracle; corrected result is FAIL.                        | incorrect verdict     | rewritten |
| WebTestBench_0023 | Buyers can search for products using keywords.                                                | functionality | PASS             |                                                                                                                     | Valid but vague about searchable fields and the expected result set.                                                                                                                                                        | Specified case-insensitive matching over title, description, and location, including the empty-query result.                                            | ambiguous             | rewritten |
| WebTestBench_0023 | Buyers can add items to their shopping cart                                                   | functionality | PASS             |                                                                                                                     | Valid but underspecified about authentication, availability, duplicate handling, and observable cart state.                                                                                                                 | Specified adding an available listing once and retaining its details in the cart.                                                                       | compound or duplicate | rewritten |
| WebTestBench_0023 | Buyers can send shopping requests                                                             | functionality | PASS             |                                                                                                                     | Ambiguous terminology and no oracle for what is submitted or how success is shown. The direct listing message is recorded as a pending purchase request, while the separate cart-request defect is isolated in new test 22. | Reworded as submitting a non-empty purchase-request message for an available listing with visible confirmation.                                         | ambiguous             | rewritten |
| WebTestBench_0003 | Users cannot log in if they do not have a registered account.                                 | constraint    | PASS             |                                                                                                                     | Valid but phrased awkwardly and does not state the observable rejection behavior.                                                                                                                                           | Specified rejection of an unknown email without creating an authenticated session.                                                                      | ambiguous             | rewritten |
| WebTestBench_0023 | Accounts that have already been registered cannot be registered again.                        | constraint    | PASS             |                                                                                                                     | Valid but vague about duplicate identity and case handling.                                                                                                                                                                 | Specified case-insensitive email uniqueness and preservation of the existing account.                                                                   | compound or duplicate | rewritten |
| WebTestBench_0023 | Unable to log in when user account password is incorrect.                                     | constraint    | FAIL             | You can still log in even if the password is wrong.                                                                 | Valid requirement and reproducible bug. Authentication looks up only the email and ignores the submitted password, including passwords chosen during registration.                                                          | Rephrased as a clear authentication constraint and retained as a FAIL case.                                                                             | ambiguous             | rewritten |
| WebTestBench_0023 | Sellers cannot update other people's product information.                                     | constraint    | PASS             |                                                                                                                     | Valid authorization intent, but update access is vacuous because no listing can be edited. The UI does scope the available management actions to the signed-in seller's own listings.                                       | Reworded to cover all supported seller-management actions and the absence of those actions on another seller's listing.                                 | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                              | Class         | Original verdict | Original reported bug                                                                          | Audit finding                                                                                                                                                                                                                                          | Correction made                                                                                                                          | Classification        | Outcome   |
|-------------------|-------------|-----------------------------------------------------------------------------------------------------------------|---------------|------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0023 |             | Users need to fill in the necessary fields to create product information.                                       | constraint    | PASS             |                                                                                                | Valid but too vague to execute consistently.                                                                                                                                                                                                           | Enumerated the required photo, title, description, valid price, category, condition, and location, plus visible validation feedback.     | ambiguous             | rewritten |
| WebTestBench_0033 |             | Prices must be non-negative numbers.                                                                            | constraint    | PASS             |                                                                                                | Valid and supported, but it should state that invalid input prevents listing creation.                                                                                                                                                                 | Specified finite numeric input greater than or equal to zero and rejection of invalid prices.                                            | ambiguous             | rewritten |
| WebTestBench_0033 |             | Creating duplicate accounts provides visual feedback.                                                           | interaction   | PASS             |                                                                                                | Valid interaction case but duplicates the constraint in test 11 unless scoped to the user-visible response.                                                                                                                                            | Scoped this case to immediate duplicate-email feedback while keeping the registration form available for correction.                     | compound or duplicate | rewritten |
| WebTestBench_0023 |             | Search results are displayed in real time when users use the search function.                                   | interaction   | PASS             |                                                                                                | Valid but 'real time' is ambiguous and no result oracle is given.                                                                                                                                                                                      | Specified updates while typing and clearing the query without a separate submission.                                                     | ambiguous             | rewritten |
| WebTestBench_0033 |             | The filtering results are displayed in real time when the user uses the filtering function.                     | interaction   | PASS             |                                                                                                | Valid but partly duplicates test 6 and omits sorting, result counts, and filter clearing.                                                                                                                                                              | Scoped it to immediate list and count updates when available filter or sort controls change and to restoration when filters are cleared. | compound or duplicate | rewritten |
| WebTestBench_0033 |             | Provide visual feedback when users add items to their cart.                                                     | interaction   | PASS             |                                                                                                | Valid but vague about which feedback establishes success.                                                                                                                                                                                              | Specified immediate confirmation plus updated cart count, line item, and total.                                                          | ambiguous             | rewritten |
| WebTestBench_0033 |             | All product photos displayed must be related to the used or upgraded products described in the product details. | content       | FAIL             | Product photo relevance verification cannot be completed using non-randomly generated methods. | Subjective and not a reliable application test: photo relevance depends on seller-supplied content, and the gold bug is incoherent. The app currently substitutes fixed stock photos, already covered by test 1.                                       | Replaced with an objective listing-content case covering the details the marketplace must display; reviewed result is PASS.              | ambiguous             | rewritten |
| WebTestBench_0045 |             | Users can sort the list by priority.                                                                            | functionality | PASS             |                                                                                                | Valid but underspecified: it does not define the priority order or the expected effect on the visible list.                                                                                                                                            | Specified high-to-low ordering of the currently viewed tasks; retained as PASS.                                                          | ambiguous             | rewritten |
| WebTestBench_0045 |             | Users can sort the list by deadline.                                                                            | functionality | PASS             |                                                                                                | Valid but underspecified: it omits the ordering direction and treatment of tasks without deadlines.                                                                                                                                                    | Specified earliest-deadline-first ordering with undated tasks last; retained as PASS.                                                    | ambiguous             | rewritten |
| WebTestBench_0045 |             | Users can modify the content and priority of any task.                                                          | functionality | FAIL             | Grayed-out task content cannot have its description and priority modified.                     | Valid intent but 'content' and 'any task' are vague. The reported defect is accurate: a task blocked by an incomplete prerequisite has its entire edit action disabled even though the brief requires its description and priority to remain editable. | Rewritten to distinguish fields that must remain editable from fields that must stay locked on a blocked task; retained as FAIL.         | ambiguous             | rewritten |
| WebTestBench_0045 |             | Users can add to-do items                                                                                       | functionality | PASS             |                                                                                                | Valid but too vague to provide an observable oracle and partly overlaps test 5.                                                                                                                                                                        | Scoped to successful creation and the resulting visible task and count update; retained as PASS.                                         | compound or duplicate | rewritten |
| WebTestBench_0045 |             | Users can fill in the details of the to-do item (title, optional description, and due date).                    | functionality | PASS             |                                                                                                | Valid but incomplete: priority and prerequisites are also supported task details, and the case does not say that submitted values must be retained.                                                                                                    | Specified all supported task details and their retention after creation; retained as PASS.                                               | ambiguous             | rewritten |
| WebTestBench_0045 |             | Users can edit, complete, or delete tasks.                                                                      | functionality | PASS             |                                                                                                | Valid but combines three substantial behaviors and does not define their outcomes. Completion is covered more precisely by test 7.                                                                                                                     | Narrowed to updating and deleting an existing task, with observable retained or removed results; retained as PASS.                       | ambiguous             | rewritten |
| WebTestBench_0045 |             | Users can mark tasks as completed.                                                                              | functionality | PASS             |                                                                                                | Valid but lacks the observable result and overlaps part of test 6.                                                                                                                                                                                     | Specified the move from the active view to the completed view and count changes; retained as PASS.                                       | compound or duplicate | rewritten |
| WebTestBench_0045 |             | Users can view ongoing tasks.                                                                                   | functionality | PASS             |                                                                                                | Ongoing' is ambiguous and the case ignores the paired completed-task view.                                                                                                                                                                             | Rewritten around the active and completed views and their mutually appropriate contents; retained as PASS.                               | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                                   | Class         | Original verdict | Original reported bug                                             | Audit finding                                                                                                                                                                                        | Correction made                                                                                                                                                                                  | Classification        | Outcome   |
|-------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0045 |             | Users can bind one or more prerequisite tasks to any task.                                                                           | functionality | PASS             |                                                                   | Valid but 'bind' and 'any task' do not define the resulting dependency state or exclude cycles.                                                                                                      | Specified selecting one or more prerequisites and retaining the relationship; cycle prevention remains isolated in test 13. Retained as PASS.                                                    | ambiguous             | rewritten |
| WebTestBench_0005 |             | The system will automatically gray out tasks that have not completed their prerequisite tasks.                                       | functionality | PASS             |                                                                   | Imprecise: tasks do not complete prerequisites; prerequisites are completed by users. It also suggests the whole task is disabled, whereas description and priority are required to remain editable. | Specified the blocked visual state and restricted actions without contradicting the limited-edit requirement; retained as PASS for the visual state, while the edit defect is covered by test 3. | ambiguous             | rewritten |
| WebTestBench_0045 |             | The system will automatically mark tasks whose prerequisite tasks have not been completed as "Dependency on xxx task not completed". | functionality | PASS             |                                                                   | Valid behavior but an invented exact message: the brief and runtime use different wording, and multiple prerequisites require a plural form.                                                         | Rewritten as a prominent warning that names one incomplete prerequisite or reports the incomplete count; retained as PASS.                                                                       | ambiguous             | rewritten |
| WebTestBench_0025 |             | Tasks with dependencies cannot be marked as completed until the dependencies are completed.                                          | constraint    | PASS             |                                                                   | Valid, although 'dependencies' should be 'prerequisite tasks' and the case omits the transition after those tasks are completed.                                                                     | Specified both refusal while any prerequisite is incomplete and availability after all are complete; retained as PASS.                                                                           | ambiguous             | rewritten |
| WebTestBench_0035 |             | If task A has become a prerequisite task for task B, then task B cannot be chosen as a prerequisite task for task A.                 | constraint    | PASS             |                                                                   | Valid direct-cycle example but too narrow: longer dependency cycles must also be prevented.                                                                                                          | Generalized to direct and indirect circular dependencies; retained as PASS.                                                                                                                      | ambiguous             | rewritten |
| WebTestBench_0045 |             | If a title is not entered when creating a new to-do item, it cannot be created.                                                      | constraint    | PASS             |                                                                   | Valid and testable, but it does not state how the user can observe that submission is unavailable.                                                                                                   | Specified whitespace-only titles and the unchanged task list; retained as PASS.                                                                                                                  | ambiguous             | rewritten |
| WebTestBench_0035 |             | The date selection must be in the correct date format.                                                                               | constraint    | PASS             |                                                                   | Implementation-specific and untestably vague. The app uses a native date control, while the product brief requires a deadline but defines no textual date format.                                    | Removed from the corrected suite; deadline entry and retention remain covered by test 5.                                                                                                         | ambiguous             | removed   |
| WebTestBench_0065 |             | The deadline cannot be a date that has already passed.                                                                               | constraint    | FAIL             | Choose                                                            | Unsupported by the product brief, and the bug description is meaningless. The runtime and source intentionally expose no minimum-date validation, so this gold case invents a requirement.           | Removed from the corrected suite rather than treating acceptance of past dates as a defect.                                                                                                      | ambiguous             | removed   |
| WebTestBench_0075 |             | Tasks marked as completed should not be automatically deleted; they should remain visible.                                           | constraint    | PASS             |                                                                   | Valid but should identify where completed tasks remain available.                                                                                                                                    | Specified retention in the completed view until the user reopens or deletes the task; retained as PASS.                                                                                          | ambiguous             | rewritten |
| WebTestBench_0085 |             | A details pop-up window will appear when the user hovers over the task.                                                              | interaction   | FAIL             | The dependency chain details pop-up did not appear when hovering. | Incorrect target: the brief requires hovering over a visual dependency connection, not an arbitrary task. After targeting the dashed connection, the failure remains reproducible.                   | Rewritten to hover over a dependency connection and require the related chain and statuses; retained as FAIL with a precise bug.                                                                 | incorrect verdict     | rewritten |
| WebTestBench_0005 |             | Provide visual feedback when creating a new to-do item without adding any title content.                                             | interaction   | PASS             |                                                                   | Redundant with test 14 and vague about what feedback is required. The required marker and disabled creation action already express the same constraint.                                              | Removed from the corrected suite to avoid duplicate coverage.                                                                                                                                    | compound or duplicate | removed   |
| WebTestBench_0005 |             | Visual feedback when the user hovers over the dependency list.                                                                       | interaction   | PASS             |                                                                   | Vague and names a dependency list that the main page does not expose. The actual hover target is the blocked-task warning label.                                                                     | Rewritten to require a dependency-chain tooltip when hovering over a blocked-task warning; retained as PASS.                                                                                     | ambiguous             | rewritten |
| WebTestBench_0045 |             | The main page must display the task title, due date, and completion status.                                                          | content       | PASS             |                                                                   | Partly incorrect because deadlines are optional, and 'completion status' is ambiguous when active and completed tasks are shown in separate views.                                                   | Specified the always-present title and priority, optional description and deadline, dependency state, and visible distinction for completed tasks; retained as PASS.                             | ambiguous             | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                                                                  | Class         | Original verdict | Original reported bug                                                                                          | Audit finding                                                                                                                                                                                                                                                                                                               | Correction made                                                                                                                                                                                     | Classification        | Outcome   |
|-------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0054 |             | Users can browse a list of technical documents organized by module.                                                                                                 | functionality | PASS             |                                                                                                                | Valid but underspecified: 'organized by module' does not say whether modules are merely labelled or can be used to narrow the list.                                                                                                                                                                                         | Specified visible module labels and module filtering, including restoring the complete list; retained as PASS.                                                                                      | ambiguous             | rewritten |
| WebTestBench_0054 |             | Users can view the document's title, content section, and a list of related documents.                                                                              | functionality | PASS             |                                                                                                                | Valid, but grammatically unclear and incomplete about the document information that must remain visible.                                                                                                                                                                                                                    | Specified the title, description, metadata, content sections, and related-document list shown after selection; retained as PASS.                                                                    | ambiguous             | rewritten |
| WebTestBench_0054 |             | Users can navigate to the relevant documents by clicking the reference links.                                                                                       | functionality | PASS             |                                                                                                                | Valid intent, but 'relevant documents' is vague and the case omits the required expanded-section precondition.                                                                                                                                                                                                              | Specified navigation from expanded section references and from the related-documents list, with the target document becoming the visible document; retained as PASS.                                | ambiguous             | rewritten |
| WebTestBench_0054 |             | Users can collapse sections of content to hide their details.                                                                                                       | functionality | PASS             |                                                                                                                | Valid but incomplete: it does not state the starting state or the observable result beyond hiding unspecified 'details'.                                                                                                                                                                                                    | Specified collapsing an expanded section and hiding its body while leaving the section available to reopen; retained as PASS.                                                                       | ambiguous             | rewritten |
| WebTestBench_0054 |             | Users can expand the collapsed content section to display its details.                                                                                              | functionality | PASS             |                                                                                                                | Valid but substantially overlaps test 4 and does not identify which content should reappear.                                                                                                                                                                                                                                | Specified restoring the selected section's body and reference links after expansion; retained as PASS.                                                                                              | compound or duplicate | rewritten |
| WebTestBench_0054 |             | Users can interact with nodes and mapping modules.                                                                                                                  | functionality | PASS             |                                                                                                                | Too vague to test: 'mapping modules' is not a recognizable action and no observable outcome is defined.                                                                                                                                                                                                                     | Rewritten around selecting graph nodes and using the graph's zoom and reset controls, with visible selection and zoom feedback; retained as PASS.                                                   | ambiguous             | rewritten |
| WebTestBench_0054 |             | Users can view the dynamic knowledge graph in its entirety.                                                                                                         | functionality | PASS             |                                                                                                                | Vague and overlapping: 'in its entirety' supplies no oracle for document, module, or relationship coverage.                                                                                                                                                                                                                 | Specified that all available documents, modules, and relationship links are represented and can be brought into view with the graph controls; retained as PASS.                                     | compound or duplicate | rewritten |
| WebTestBench_0054 |             | The webpage is divided into several tabs: viewing documents, images, and bookmarks.                                                                                 | functionality | PASS             |                                                                                                                | Incorrect: the brief and runtime provide Documents, Knowledge Graph, and Favorites tabs; there is no Images or Bookmarks tab.                                                                                                                                                                                               | Corrected the three feature-level tab names; retained as PASS.                                                                                                                                      | incorrect verdict     | rewritten |
| WebTestBench_0054 |             | When cutting tags, the previous page state (such as whether paragraphs are collapsed or how large the map is) must be preserved.                                    | functionality | FAIL             | The original state cannot be retained after switching.                                                         | Mistranslated and incorrectly labelled FAIL. Live testing preserved the selected document, an expanded second section, and a 140% graph zoom through multiple tab switches; the Zustand store confirms those states are shared across tabs.                                                                                 | Rewritten as tab switching with preserved document selection, section expansion, and graph view state; corrected to PASS.                                                                           | ambiguous             | rewritten |
| WebTestBench_0064 |             | The saved content is stored locally, and we need to count how many have been saved. After filtering and sorting, the saved items must not be lost.                  | functionality | FAIL             | Only one item is displayed in the collection.                                                                  | Valid but composite, and the gold bug is imprecise. A related failure is reproducible: after favoriting documents from two modules and leaving a module filter active, the Favorites count remains 2 but the Favorites list silently shows only the matching item because the filter is applied there without its controls. | Specified persistence across reload, real-time count, marker retention, and visibility of all favorites regardless of filters used in Documents; retained as FAIL with a precise hidden-filter bug. | ambiguous             | rewritten |
| WebTestBench_0054 |             | A dynamic knowledge graph visualization of document relationships is provided, displaying connections through interactive nodes and links for intuitive navigation. | functionality | FAIL             | The interactive animations when selecting knowledge nodes are not smooth, and the navigation is not intuitive. | Subjective and redundant with tests 6, 7, 13, and 16. 'Smooth' and 'intuitive' have no reproducible oracle, while the graph's nodes, edges, selection highlight, and controls are covered more precisely elsewhere.                                                                                                         | Removed from the corrected suite rather than preserving an aesthetic FAIL with no objective oracle.                                                                                                 | ambiguous             | removed   |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                              | Class         | Original verdict | Original reported bug                                                                                                                               | Audit finding                                                                                                                                                                                                                                                        | Correction made                                                                                                                               | Classification        | Outcome   |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0034 | A document cannot reference itself as a related document.                                                                                          | constraint    | PASS             |                                                                                                                                                     | A reasonable data-integrity constraint, although it is not stated explicitly in the product brief. All 19 seeded documents satisfy it.                                                                                                                               | Clarified that neither section references nor related-document entries may target the current document; retained as PASS.                     | ambiguous             | rewritten |
| WebTestBench_0034 | Dynamic knowledge graphs must correspond to documents and modules.                                                                                 | constraint    | FAIL             | Some modules are not visible, and adjusting the scale will not provide a complete visualization of the atlas.                                       | Valid intent but overlapping and incorrectly labelled FAIL. The runtime contains one node for each of the 19 documents and a legend for all five modules; zooming out exposes the full layout. The reported missing-module defect was not reproduced.                | Specified one node per available document, correct module grouping, and relationship edges derived from related documents; corrected to PASS. | compound or duplicate | rewritten |
| WebTestBench_0034 | When a user clicks a reference link that points to a non-existent document, the error must be handled gracefully without breaking the application. | interaction   | PASS             |                                                                                                                                                     | Valid requirement but not executable against the shipped app: all 30 section references and all 34 related-document references resolve, so no invalid link is exposed through the UI. Source-level defensive code alone cannot make this a runnable end-to-end case. | Removed from the corrected runtime suite; add an explicit invalid-link fixture before reinstating this requirement as an end-to-end test.     | ambiguous             | removed   |
| WebTestBench_0054 | Visual indicators must show whether the content section is currently collapsed or expanded.                                                        | interaction   | PASS             |                                                                                                                                                     | Valid but underspecified: it does not identify the indicator or require it to agree with content visibility.                                                                                                                                                         | Specified a clear state indicator whose change agrees with whether the section body and links are visible; retained as PASS.                  | ambiguous             | rewritten |
| WebTestBench_0064 | Clicking on the relevant document node must navigate the user to the view of that document.                                                        | interaction   | PASS             |                                                                                                                                                     | Valid, but the gold PASS is wrong. Clicking a graph node selects and highlights it and stores the document ID, yet the application remains on the graph; the document appears only after the user separately switches to Documents.                                  | Required a single node selection to open the corresponding document view while retaining graph state; corrected to FAIL.                      | incorrect verdict     | rewritten |
| WebTestBench_0063 | Users can view a list of all intangible cultural heritage items on the website.                                                                    | functionality | PASS             |                                                                                                                                                     | Valid and directly testable.                                                                                                                                                                                                                                         | No substantive change; clarified completeness and the visible result count in the corrected case.                                             | ambiguous             | rewritten |
| WebTestBench_0063 | Users can click on a heritage item from the list to navigate to its dedicated details page.                                                        | functionality | FAIL             | In the panoramic exploration list, clicking on anchor points results in misaligned and obscured floating layers that cannot be displayed correctly. | Internally inconsistent: the content describes list-to-detail navigation, while the FAIL label and bug describe a different hotspot-overlay interaction. Direct list-to-detail links work.                                                                           | Reframed the case around selecting a loaded panorama hotspot; kept FAIL and described the displaced overlay precisely.                        | ambiguous             | rewritten |
| WebTestBench_0063 | Users can view complete details of heritage projects on their details page, including titles, descriptions, images, and regional information.      | functionality | FAIL             | Some projects do not have images.                                                                                                                   | Partly valid, but it conflates detail completeness with individual image-load failures. Every item has image URLs and a gallery; some remote assets fail to load.                                                                                                    | Specified the concrete detail fields and gallery; changed the corrected case to PASS. Image-failure feedback remains covered by test 6.       | ambiguous             | rewritten |
| WebTestBench_0063 | Users can browse cultural heritage projects organized by type and category (traditional crafts, folk customs, performing arts).                    | functionality | PASS             |                                                                                                                                                     | Valid but incomplete: it omits Traditional Art, search, region filtering, and sorting, all of which are implemented.                                                                                                                                                 | Expanded the case to cover search, category and region filters, sorting, result updates, and all four supported categories.                   | ambiguous             | rewritten |
| WebTestBench_0063 | The state is not lost after all operations on the page.                                                                                            | functionality | PASS             |                                                                                                                                                     | Too vague and absolute: neither the relevant state nor the operations are defined, so the oracle is not reproducible.                                                                                                                                                | Scoped the case to restoration of panorama orientation and zoom after a detail-page round trip; kept PASS.                                    | incorrect verdict     | rewritten |
| WebTestBench_0063 | The page should provide corresponding prompts for issues such as slow panorama loading and inability to open product detail page images.           | functionality | PASS             |                                                                                                                                                     | Valid intent but vague terminology; it conflates panorama loading, hotspot loading, and failed detail images.                                                                                                                                                        | Defined the three observable feedback states separately and retained PASS.                                                                    | ambiguous             | rewritten |
| WebTestBench_0063 | Each heritage item must belong to at least one valid category (traditional crafts, folk customs, or performing arts).                              | constraint    | PASS             |                                                                                                                                                     | Incorrect category set: the app also supports Traditional Art, used by two displayed items.                                                                                                                                                                          | Added Traditional Art and aligned the category names with the app; retained PASS.                                                             | incorrect verdict     | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                               | Class         | Original verdict | Original reported bug                                                                                                                  | Audit finding                                                                                                                                                                                                                                 | Correction made                                                                                                                                            | Classification    | Outcome         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------|
| WebTestBench_0063 | You will be directed to the heritage project details page only when you click on a valid heritage project.                                                          | constraint    | PASS             |                                                                                                                                        | Vague and nearly tautological: it does not define how invalid identifiers or routes should behave.                                                                                                                                            | Replaced it with explicit valid-project routing and invalid-identifier handling; retained PASS.                                                            | ambiguous         | rewritten       |
| WebTestBench_0063 | Each heritage item on display must include all required fields (title, description, region).                                                                        | constraint    | PASS             |                                                                                                                                        | Valid and directly testable.                                                                                                                                                                                                                  | Clarified that the fields are visible on each project card and included category; retained PASS.                                                           | ambiguous         | rewritten       |
| WebTestBench_0063 | The navigation between the list view and the details view maintains a clear visual distinction between the two interface states.                                    | interaction   | FAIL             | The top navigation bar remains unchanged; clicking on the category preview does not redirect back to the correct area of the homepage. | Misaligned. The views are visually distinct, and a consistent top navigation bar is not a defect. The real reproducible defect is category navigation from a detail page.                                                                     | Replaced the subjective visual-distinction check with category-browsing navigation back to the homepage list; kept FAIL and isolated the broken hash link. | ambiguous         | rewritten       |
| WebTestBench_0063 | All exhibited artifacts must be genuine Chinese intangible cultural heritage items, and all are based on authentic and reliable historical materials and knowledge. | content       | FAIL             | Image and text do not match                                                                                                            | The authenticity and source-reliability claim cannot be established from the app alone. The narrower image/text consistency issue is observable and valid.                                                                                    | Narrowed the case to whether displayed imagery is relevant to the named item and description; kept FAIL and clarified the generic stock-image mismatch.    | ambiguous         | rewritten       |
| WebTestBench_0074 | Users can add new entries to the database.                                                                                                                          | functionality | FAIL             | The newly added item disappeared after the page was refreshed.                                                                         | Valid core behavior but underspecified: it does not say which configured values are entered or whether a database entry must survive a reload. The reported persistence defect is reproducible; all state resets to the seed data on refresh. | Specified immediate display and persistence after reload; retained as FAIL with a more precise bug.                                                        | ambiguous         | rewritten       |
| WebTestBench_0074 | Users can create different types of fields: text, number, date, and checkbox.                                                                                       | functionality | PASS             |                                                                                                                                        | Valid but it omits the observable result of field creation.                                                                                                                                                                                   | Specified that a field can be assigned each supported type and then appears in entry forms and data views; retained as PASS.                               | ambiguous         | rewritten       |
| WebTestBench_0074 | Users can input and save data for each field type (text string, number, date value, checkbox status).                                                               | functionality | PASS             |                                                                                                                                        | Valid but it does not require the saved values to remain associated with the entry or appear in both views.                                                                                                                                   | Specified retained values and display in table and card views; retained as PASS.                                                                           | ambiguous         | rewritten       |
| WebTestBench_0074 | Users can switch between table layout and card grid layout view modes.                                                                                              | functionality | PASS             |                                                                                                                                        | Valid but lacks an oracle for data continuity across the view change.                                                                                                                                                                         | Specified that the current entries and values remain unchanged while switching views; retained as PASS.                                                    | ambiguous         | rewritten       |
| WebTestBench_0074 | Users can sort entries by any field in the database.                                                                                                                | functionality | PASS             |                                                                                                                                        | Valid but underspecified: it omits direction and the requirement that table and card views keep independent sort selections.                                                                                                                  | Specified ascending and descending order for every configured field plus per-view sort-state retention; retained as PASS.                                  | ambiguous         | rewritten       |
| WebTestBench_0074 | Users can apply filters to display only entries that meet specified criteria.                                                                                       | functionality | PASS             |                                                                                                                                        | Valid requirement, but the gold PASS is wrong. The app has only free-text search and exposes no filter control or field-specific filtering logic.                                                                                             | Specified applying and clearing field-specific criteria; changed to FAIL with the actual missing-feature bug.                                              | incorrect verdict | verdict changed |
| WebTestBench_0074 | Users can manually adjust the order of items, and the result after adjustment will be consistent with the result of manual operation.                               | functionality | PASS             |                                                                                                                                        | Vague and tautological: 'consistent with the result' gives no oracle and does not mention the separate orders for table and card views.                                                                                                       | Specified manual ordering when no sort is active and independent retained order in each view; retained as PASS.                                            | ambiguous         | rewritten       |
| WebTestBench_0074 | Numeric fields must accept only valid numeric input and reject non-numeric characters.                                                                              | constraint    | PASS             |                                                                                                                                        | Valid intent, but 'characters' is tied to input mechanics rather than the saved data invariant.                                                                                                                                               | Reframed as preventing a non-numeric field value from being saved while retaining valid numbers; retained as PASS.                                         | ambiguous         | rewritten       |
| WebTestBench_0074 | Date fields must accept only valid date formats and reject invalid dates.                                                                                           | constraint    | PASS             |                                                                                                                                        | Valid intent but vague about 'format'; the app uses a calendar-date control rather than accepting arbitrary date strings.                                                                                                                     | Reframed around valid calendar dates and preventing invalid date values from being saved; retained as PASS.                                                | ambiguous         | rewritten       |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                         | Class         | Original verdict | Original reported bug                                                                           | Audit finding                                                                                                                                                                                                                                 | Correction made                                                                                                             | Classification    | Outcome         |
|-------------------|-------------|----------------------------------------------------------------------------------------------------------------------------|---------------|------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------|
| WebTestBench_0004 |             | At least one field is not defined in the database, so an entry cannot be added.                                            | constraint    | FAIL             | Blank input boxes can also be used to create new entries, which will be displayed in the table. | Ambiguous and unsupported: it neither defines what 'not defined' means nor follows from the brief, which does not require every field to be non-empty. The reported behavior is therefore not a justified defect.                             | Removed from the corrected suite; empty-value display is covered by corrected test 13 without inventing mandatory fields.   | ambiguous         | removed         |
| WebTestBench_0074 |             | The sorting process must maintain data integrity; no input data may be modified or lost.                                   | constraint    | PASS             |                                                                                                 | Valid but it does not state observable integrity checks.                                                                                                                                                                                      | Specified preservation of entry count and every field value while only order changes; retained as PASS.                     | ambiguous         | rewritten       |
| WebTestBench_0024 |             | Filters must accurately include/exclude entries according to specified criteria, and must not lose data.                   | constraint    | FAIL             | Clicking "Filter" results in a page error and a blank page.                                     | Valid integrity requirement, but the reported bug is fabricated: there is no Filter action to click. The real failure is that filtering is absent altogether.                                                                                 | Specified exact inclusion and non-destructive clearing; retained as FAIL with the missing-filter bug.                       | incorrect verdict | rewritten       |
| WebTestBench_0034 |             | All entries and their field values must be displayed completely and accurately in both table view and card grid view.      | constraint    | PASS             |                                                                                                 | Valid requirement, but the gold PASS is wrong for empty first-text-field values. Card view removes the first text field from the labeled fields to use it as a title, then renders nothing when that value is empty, so the field disappears. | Added a clear empty-value oracle; changed to FAIL with the reproducible card-view omission.                                 | incorrect verdict | verdict changed |
| WebTestBench_0074 |             | In both view modes, field labels and types must be clearly distinguishable.                                                | constraint    | PASS             |                                                                                                 | The intent is reasonable, but requiring explicit type labels in both data views is more prescriptive than the brief. Type selection and input restrictions are the actual requirement.                                                        | Reframed around clearly labeled type choices and type-appropriate entry controls; retained as PASS.                         | ambiguous         | rewritten       |
| WebTestBench_0054 |             | The checkbox status must be visually distinguishable as either selected or unselected.                                     | constraint    | PASS             |                                                                                                 | Valid and testable, although it should cover the editor and both data views.                                                                                                                                                                  | Specified clearly distinguishable checked and unchecked states in entry forms, table view, and card view; retained as PASS. | ambiguous         | rewritten       |
| WebTestBench_0064 |             | When deleting a data entry, a confirmation button will pop up to prevent users from accidentally deleting it.              | interaction   | FAIL             | There is no confirmation function; you can delete it directly.                                  | The runtime observation is accurate, but the confirmation requirement is invented: the product brief never requires a delete-confirmation step.                                                                                               | Removed from the corrected suite rather than treating direct deletion as a defect.                                          | ambiguous         | removed         |
| WebTestBench_0074 |             | When a user enters a text field to search, the page displays the search results.                                           | interaction   | PASS             |                                                                                                 | Valid but vague about matching behavior, visible count, and clearing the query.                                                                                                                                                               | Specified case-insensitive partial text matching, count update, and restoration after clearing; retained as PASS.           | ambiguous         | rewritten       |
| WebTestBench_0084 |             | Users can click to sort, and the page will respond promptly.                                                               | interaction   | FAIL             | Page error                                                                                      | Valid interaction intent but redundant with test 5 and too vague. The reported page error is not reproducible: header and menu sorting work and the app remains usable.                                                                       | Specified immediate reordering plus visible field/direction feedback; changed to PASS and cleared the false bug.            | incorrect verdict | verdict changed |
| WebTestBench_0089 |             | Users can browse the template library list, where each template displays basic information such as author and description. | functionality | PASS             |                                                                                                 | Valid and directly testable, though it overlaps with test 7.                                                                                                                                                                                  | Kept the library-browsing requirement and left the exhaustive meta-data check to test 7.                                    | no change         | unchanged       |
| WebTestBench_0089 |             | Users can select templates                                                                                                 | functionality | PASS             |                                                                                                 | Valid but too vague to define an observable outcome.                                                                                                                                                                                          | Specified that selecting a template opens its customizable form and details.                                                | ambiguous         | rewritten       |
| WebTestBench_0089 |             | Users can fill in the form related information.                                                                            | functionality | PASS             |                                                                                                 | Valid intent but grammatically vague; it does not connect the form to the selected template's declared fields.                                                                                                                                | Clarified that users can enter values for every field defined by the selected template.                                     | ambiguous         | rewritten       |
| WebTestBench_0089 |             | Users can preview the completed form.                                                                                      | functionality | PASS             |                                                                                                 | Valid intent, but the object being previewed and its preconditions are unclear.                                                                                                                                                               | Defined the preview as generated template output after required fields are valid, with entered values substituted.          | ambiguous         | rewritten       |

Continued on next page

Table E.1 – continued

| Application       | Original ID | Original test case                                                                                                                                                          | Class         | Original verdict | Original reported bug                                                                                                 | Audit finding                                                                                                                                                                                                           | Correction made                                                                                                                                         | Classification    | Outcome   |
|-------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------|
| WebTestBench_0089 |             | Users can submit custom templates (plain text or Markdown format) and basic information through a form.                                                                     | functionality | PASS             |                                                                                                                       | Valid but incomplete: the submission also requires a description and at least one defined field, and supports category and scenario metadata.                                                                           | Expanded the case to the actual required metadata, content format, and field-definition capability.                                                     | ambiguous         | rewritten |
| WebTestBench_0089 |             | Once a user-submitted custom template is successfully saved, it will appear in the public template library for others to use.                                               | functionality | PASS             |                                                                                                                       | Incorrect scope: the app has no backend or cross-user publication; and use in the current browser's templates are stored only in local-storage for the current browser.                                                 | Narrowed the case to appearance fields actually displayed on each card.                                                                                 | incorrect verdict | rewritten |
| WebTestBench_0089 |             | Each template displays basic information: template name, author, description, type tag, and update time.                                                                    | functionality | PASS             |                                                                                                                       | Mostly valid but incomplete: cards also show scenario and usage count, while 'type tag' should be named category.                                                                                                       | Aligned the metadata list with the fields actually displayed on each card.                                                                              | ambiguous         | rewritten |
| WebTestBench_0089 |             | The result displayed after the user fills out the form is consistent with the expected result in Markdown.                                                                  | functionality | FAIL             | Inconsistencies exist; line breaks and bolding are not implemented, and some fields have not been replaced correctly. | The FAIL label and bug are wrong for built-in templates. Runtime inspection showed headings, bold text, separators, and line breaks rendered, with declared placeholders replaced when all required fields were filled. | Made the expected Markdown behavior concrete and changed the corrected case to PASS; malformed custom placeholder mappings are isolated in new test 27. | ambiguous         | rewritten |
| WebTestBench_0089 |             | Users can filter by template type, applicable scenarios, popularity, and update time.                                                                                       | functionality | PASS             |                                                                                                                       | Partly inaccurate: category and scenario are filters, while popularity and time are sort modes rather than filters.                                                                                                     | Separated filtering from sorting and named all supported sort modes and observable result updates.                                                      | ambiguous         | rewritten |
| WebTestBench_0089 |             | The web search box supports widgets for fuzzy matching.                                                                                                                     | functionality | PASS             |                                                                                                                       | Vague and incorrectly labelled PASS. There are suggestion items, but matching is simple case-insensitive substring search rather than fuzzy matching.                                                                   | Replaced 'widgets' with a typo-tolerance requirement and changed the corrected case to FAIL.                                                            | ambiguous         | rewritten |
| WebTestBench_0089 |             | The website supports template saving function.                                                                                                                              | functionality | PASS             |                                                                                                                       | Ambiguous: 'saving' could mean submission, persistence, or favorites.                                                                                                                                                   | Interpreted the requirement as the explicitly requested favorites feature and defined add, remove, and favorites-only retrieval behavior.               | ambiguous         | rewritten |
| WebTestBench_0089 |             | Users can view and search for saved templates in the collection list.                                                                                                       | functionality | PASS             |                                                                                                                       | Valid if 'saved templates' means favorites, but 'collection list' is not app terminology.                                                                                                                               | Reframed it as searching and filtering within the favorites-only view.                                                                                  | ambiguous         | rewritten |
| WebTestBench_0089 |             | Supports copying the previewed results (one-click copying of plain text/Markdown formats).                                                                                  | functionality | PASS             |                                                                                                                       | Valid, but it could imply a format chooser that does not exist. The app copies source Markdown for Markdown templates and plain text for plain-text templates.                                                          | Specified copying in the selected template's declared format and included the visible success feedback.                                                 | irrelevant        | rewritten |
| WebTestBench_0089 |             | The information entered must conform to the format requirements; for example, the date must be a correct date.                                                              | constraint    | FAIL             | You can still submit the application even if you enter any date information.                                          | Valid validation intent, but 'submit the application' is unrelated and most built-in date-like fields are plain text. A broader reproducible defect exists: invalid email and date-like values still enable preview.    | Reframed the case around declared email/date formats and retained FAIL with a concrete runtime reproduction.                                            | ambiguous         | rewritten |
| WebTestBench_0089 |             | The start date entered must be earlier than the end date.                                                                                                                   | constraint    | FAIL             | Can be later than the end date                                                                                        | Irrelevant and over-specific: neither the product request nor the generic template model defines paired date-range semantics, and many templates have no start/end fields.                                              | Removed from the corrected suite.                                                                                                                       | irrelevant        | removed   |
| WebTestBench_0089 |             | Users cannot submit templates with empty required fields (template content, title, author name).                                                                            | constraint    | PASS             |                                                                                                                       | Valid but incomplete: description and at least one template field are also required.                                                                                                                                    | Added every enforced submission prerequisite.                                                                                                           | ambiguous         | rewritten |
| WebTestBench_0089 |             | Form input fields restrict special characters (such as ¥\#@%^*_, etc.), automatically filtering or displaying a "Special characters are prohibited" message when inputting. | constraint    | FAIL             | You can submit by entering any symbol.                                                                                | The FAIL direction is valid, but the bug overstates the behavior: some characters are rejected. Markdown-active characters such as \#, \*, and \_ are allowed and can alter rendered structure.                         | Defined the requirement in terms of characters that can break output and documented the reproduced unintended-heading defect.                           | ambiguous         | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                                       | Class         | Original verdict | Original reported bug                                              | Audit finding                                                                                                                                                                                     | Correction made                                                                                                                          | Classification        | Outcome   |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------|
| WebTestBench_0089 | When submitting a Markdown template, validate its basic syntax to prevent rendering errors.                                                                                 | constraint    | FAIL             | Illegal syntax can still be submitted                              | Irrelevant and not reproducible as written: Markdown is intentionally permissive, and neither the product request nor the case defines what 'illegal syntax' means.                               | Removed from the corrected suite; the concrete placeholder-consistency defect is covered by new test 27.                                 | incorrect verdict     | removed   |
| WebTestBench_0089 | If the required form fields of the selected template are not filled in, the user will not be able to generate a preview.                                                    | constraint    | PASS             |                                                                    | Valid and directly testable.                                                                                                                                                                      | Clarified that the preview remains unavailable until all required values are nonblank and valid.                                         | ambiguous             | rewritten |
| WebTestBench_0089 | As users enter keywords in the search box, the webpage displays suggested results in real time. Clicking on a suggested result will redirect to the corresponding template. | interaction   | PASS             |                                                                    | Partly incorrect: suggestions appear and are selectable, but selection does not redirect or open a template; it fills the search query and narrows the same-page library.                         | Aligned the outcome with the actual single-page suggestion behavior.                                                                     | ambiguous             | rewritten |
| WebTestBench_0089 | Search results are displayed in real time as users filter.                                                                                                                  | interaction   | PASS             |                                                                    | Valid but underspecified; it conflates search, filters, favorites, and sorting.                                                                                                                   | Named the controls whose changes update the visible results and count without a reload.                                                  | ambiguous             | rewritten |
| WebTestBench_0089 | If the required form information is not filled in, provide visual feedback indicating that submission is not possible.                                                      | interaction   | PASS             |                                                                    | Valid intent but uses the wrong action: the selected-template workflow generates a preview rather than submitting the filled form.                                                                | Specified the missing-field message and disabled preview control.                                                                        | ambiguous             | rewritten |
| WebTestBench_0089 | When entering a special character, provide visual feedback indicating that the input is not possible or that special characters cannot be entered.                          | interaction   | FAIL             | Special characters can still be submitted, but cannot be verified. | Duplicative of test 17 and internally unclear: the bug says the behavior cannot be verified while asserting that characters can be submitted.                                                     | Removed from the corrected suite; test 17 covers both validation and feedback.                                                           | compound or duplicate | removed   |
| WebTestBench_0089 | Provide visual feedback when clicking the "Favorite" button.                                                                                                                | interaction   | PASS             |                                                                    | Valid and directly observable through the heart state and favorites count/list.                                                                                                                   | Expanded the feedback requirement to include both favoriting and unfavoriting.                                                           | ambiguous             | rewritten |
| WebTestBench_0089 | The template content (plain text or Markdown) renders correctly during preview and use, without any formatting errors.                                                      | content       | FAIL             | Custom content cannot guarantee correct formatting.                | Overbroad and incorrectly labelled FAIL: no system can guarantee arbitrary author content is semantically correct. Valid built-in and well-formed custom content render in the declared mode.     | Scoped the case to valid content and changed it to PASS; input-driven formatting corruption remains covered by test 17.                  | ambiguous             | rewritten |
| WebTestBench_0089 | Once a new template is submitted, the template library will update and display it immediately, and the list status will remain synchronized.                                | content       | PASS             |                                                                    | Valid, though 'list status' is vague and the behavior overlaps partly with test 6.                                                                                                                | Specified immediate card and visible-count updates without reloading.                                                                    | compound or duplicate | rewritten |
| WebTestBench_0097 | Human resources professionals can search for competency models by job title or keywords, and the system can accurately return a list of matching models.                    | functionality | PASS             |                                                                    | Valid, but "accurately" is underspecified and the case omits the implemented department filter and empty-result state.                                                                            | Defined case-insensitive matching across job title, keywords, and description; added department filtering and a clear no-results oracle. | ambiguous             | rewritten |
| WebTestBench_0097 | Users can click on any model in the search results to enter the model's details page.                                                                                       | functionality | PASS             |                                                                    | Valid core behavior, although the app opens an in-page detail editor rather than navigating to a separate page.                                                                                   | Reworded the destination as the corresponding detail editor and specified matching title, summary metrics, and graph content.            | ambiguous             | rewritten |
| WebTestBench_0097 | Users can directly adjust the hierarchy of capabilities by dragging and dropping nodes on the front end, and the model structure remains correct after layout changes.      | functionality | PASS             |                                                                    | Misleading terminology: dragging changes only the graph layout, not a competency hierarchy or dependency structure. The layout interaction itself works.                                          | Scoped the case to rearranging competency nodes while the existing relationships remain attached and unchanged.                          | ambiguous             | rewritten |
| WebTestBench_0097 | Users can modify the weight value of any edge, and the graph will be refreshed immediately after the modification, visually reflecting the weight change.                   | functionality | PASS             |                                                                    | The gold PASS is wrong. Edge weights are displayed as labels and stroke widths, but the app exposes no way to edit a relationship weight; the available weight sliders edit competencies instead. | Retained the relationship-weight requirement, changed it to FAIL, and made the expected graph and metric update explicit.                | incorrect verdict     | rewritten |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                                                                                          | Class         | Original verdict | Original reported bug           | Audit finding                                                                                                                                                                                                                                                                                  | Correction made                                                                                                                                                   | Classification    | Outcome         |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------|
| WebTestBench_0097 | Users can temporarily enable or disable certain capability nodes in the graph. When disabled, the node and its related edges will be visually grayed out or hidden, and will no longer participate in subsequent calculations. | functionality | FAIL             | This function is not available. | The feature is available, so the supplied bug is false, but its behavior is broken. Metrics change while the switch and node remain visually enabled, connected edges remain active-looking, a second click cannot re-enable the node, and the critical-path calculation can still include it. | Kept FAIL and replaced the fabricated missing-feature claim with the reproducible state-synchronization, edge-visibility, repeat-toggle, and calculation defects. | ambiguous         | rewritten       |
| WebTestBench_0097 | The system provides a "General Job Competency Model Template Library," which users can browse and preview by job type.                                                                                                         | functionality | PASS             |                                 | Mostly valid, but there is no separate preview or job-type browser. The library lists five role templates with department, competency count, connection count, and keywords.                                                                                                                   | Aligned the case with browsing and comparing the available template summaries by job role.                                                                        | ambiguous         | rewritten       |
| WebTestBench_0097 | Users can retrieve any template from the template library, which will be loaded as a new model into the editing interface.                                                                                                     | functionality | PASS             |                                 | The gold PASS is wrong. Applying a template changes the title and metrics, but React Flow retains the previous model's nodes and edges, leaving the editor internally inconsistent.                                                                                                            | Required the title, metrics, graph, and scenario inputs to all represent the applied template; changed to FAIL with the stale-graph defect.                       | incorrect verdict | verdict changed |
| WebTestBench_0097 | Users can save the edited model as a custom template, name it, and add a description. After saving, they can retrieve it from "My Templates" at any time.                                                                      | functionality | FAIL             | My model does not exist         | Valid requirement and correct FAIL label, but the bug description is vague. "Save Current" only displays a success toast; there is no name/description form, custom-template storage, or My Templates collection.                                                                              | Specified creation, naming, description, retrieval, and retained edits; kept FAIL with the missing workflow and persistence behavior.                             | ambiguous         | rewritten       |
| WebTestBench_0097 | The system supports scenario-based simulation modes such as "recruitment," "training," and "promotion," and users can switch between different scenarios.                                                                      | functionality | PASS             |                                 | Valid, but it omits the observable change in scenario-specific content and adjustment values.                                                                                                                                                                                                  | Specified all three supported scenarios and required their descriptions and category adjustments to update when switching.                                        | ambiguous         | rewritten       |
| WebTestBench_0097 | In simulation mode, users can adjust the model structure, and the system can simulate and demonstrate the quantitative or qualitative impact of this adjustment on key indicators in the scenario.                             | functionality | PASS             |                                 | Overstated and vague: simulation adjusts competency levels and weights by category, not graph structure, and the observable outputs are the summary metrics and alerts.                                                                                                                        | Scoped the case to category level/weight adjustments, quantitative metric and alert updates, and reset behavior.                                                  | ambiguous         | rewritten       |
| WebTestBench_0097 | When a user leaves the page and returns, the system should be able to restore the previously edited model state, or provide an explicit save/load mechanism.                                                                   | functionality | PASS             |                                 | The gold PASS is wrong. The Save action only displays a toast; leaving and reopening the model restores seed data, and a reload also discards changes.                                                                                                                                         | Required an actual save/load or automatic restoration round trip; changed to FAIL with the non-persistent save behavior.                                          | incorrect verdict | verdict changed |
| WebTestBench_0097 | The system can prevent invalid operations that may lead to calculation errors, such as adding dependency edges to disabled capabilities.                                                                                       | constraint    | PASS             |                                 | Irrelevant to the implemented editor: users cannot add dependency edges at all, so the example operation cannot be exercised and a PASS is meaningless.                                                                                                                                        | Replaced it with the related, testable invariant that competency levels and weights remain within their supported ranges after direct edits and simulations.      | irrelevant        | rewritten       |
| WebTestBench_0097 | Each time the graph structure is modified, the system can recalculate and display the updated "capability coverage" percentage in real time.                                                                                   | interaction   | PASS             |                                 | Valid recalculation intent, but competency level, weight, and enabled-state edits are attribute changes rather than graph-structure changes. Node dragging does not affect coverage.                                                                                                           | Specified which edits should affect coverage and required the percentage and progress indicator to update without reload.                                         | ambiguous         | rewritten       |
| WebTestBench_0097 | All graph structure editing operations are smooth and responsive, with no noticeable lag or delay when the model complexity is moderate.                                                                                       | interaction   | PASS             |                                 | Subjective and non-reproducible: neither "noticeable lag" nor "moderate" has a threshold, and most claimed structural edits are not implemented.                                                                                                                                               | Removed from the corrected suite; functional update behavior is covered by tests 3, 4, 5, 13, and 16.                                                             | ambiguous         | removed         |

Continued on next page

Table E.1 – continued

| Application       | Original test case ID                                                                                                                                                                                                         | Class   | Original verdict | Original reported bug | Audit finding                                                                                                                                                                                                                                                                                     | Correction made                                                                                                                                   | Classification    | Outcome         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------|
| WebTestBench_0007 | On the details page, the system presents the competency model in a clear, interactive, directed weighted graph structure, where nodes represent competencies and edges represent dependencies or reinforcement relationships. | content | PASS             |                       | Valid and directly testable on an initially opened model.                                                                                                                                                                                                                                         | Clarified the observable node labels, directed connections, relationship weights, relationship-type distinction, and graph navigation.            | ambiguous         | rewritten       |
| WebTestBench_0007 | After each structural change, the system can recalculate and highlight the "critical path" in the model in real time.                                                                                                         | content | PASS             |                       | The gold PASS is wrong. The implementation derives a set from the three highest-weight enabled competencies plus some inbound dependencies rather than a graph path; it can include disabled dependencies, and React Flow does not receive updated node or edge highlighting after model changes. | Defined recalculation after relevant weight or enabled-state changes and required both the summary and graph highlight to agree; changed to FAIL. | incorrect verdict | verdict changed |
| WebTestBench_0007 | After each structural change, the system can analyze in real time and provide a "risk capability gap" warning.                                                                                                                | content | PASS             |                       | Valid intent but imprecise: gap status is driven by an enabled competency's level and weight, not arbitrary structural changes.                                                                                                                                                                   | Specified a concrete risk-gap trigger and required the warning to appear and clear as the condition is introduced and resolved.                   | ambiguous         | rewritten       |
| WebTestBench_0007 | When a potential problem is detected, the system can immediately provide a clear text or icon prompt in a prominent position on the interface.                                                                                | content | FAIL             | No prompt found       | The gold FAIL and bug are wrong. Lowering an enabled high-weight competency below the threshold immediately shows a named Competency Gap Alert near the summary metrics, with an alert icon and affected competency name.                                                                         | Kept the warning requirement, grounded it in detected competency gaps, changed to PASS, and cleared the false bug.                                | incorrect verdict | verdict changed |

## Appendix F

# Corrected Test Cases

This appendix lists the 270 test cases on which every run in Chapter 6 was scored, as they were supplied to both arms. For each application, Table [F.1](#) gives each test case’s identifier, wording, WebTestBench class, ground-truth verdict and, for failing cases, the defect established by hand against the running application (Section 5.2).

Table F.1: Full list of corrected test cases used in the final experiments.

| Application       | ID | Test case                                                                                                                                                                                                                                                                                    | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                                                                           |
|-------------------|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0001 | 1  | An administrator can create and edit products in any supported category, including their name, price, rating, brief description, full review, image, retailer link, tags, specifications, and featured status; saved changes are immediately reflected in product browsing and detail views. | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 2  | Marking or unmarking a product as featured updates the homepage featured collection, and visitors can browse the complete set of featured products.                                                                                                                                          | functionality | FAIL    | The homepage renders only the first three featured products, so a fourth marked product is omitted. The destination intended to expose the complete featured set uses ?featured=true, but the products page ignores that parameter and displays all products. |
| WebTestBench_0001 | 3  | Keyword search matches product names, brief descriptions, and tags case-insensitively, and displays only matching products with an accurate result count.                                                                                                                                    | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 4  | A tag newly assigned to an existing product becomes available as a tag-filter choice and selecting it returns the matching product.                                                                                                                                                          | functionality | FAIL    | Edited tags are stored on the product and work in keyword search, but the filter choices come from the static initial-product tag list, so newly assigned tags never appear as selectable filters.                                                            |
| WebTestBench_0001 | 5  | When creating a product, an administrator can assign one or more new tag values, and the saved tags appear on the product and participate in keyword search.                                                                                                                                 | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 6  | An administrator can add, rename, or remove tags on an existing product, and the saved tag set is reflected on the product detail page and in keyword search.                                                                                                                                | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 7  | An administrator can assign a product to exactly one supported category, and the product appears under the selected category after saving.                                                                                                                                                   | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 8  | Visitors can filter the complete product list by Tech Gadgets, Home Goods, or Fitness, and the visible products and result count update to match the selected category.                                                                                                                      | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 9  | Visitors can browse the complete product list, where every product exposes its name, category, price, rating, brief review, image, and a purchase action.                                                                                                                                    | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 10 | Visitors can sort products by newest, top rating, ascending price, or descending price, and can filter them by a price range or minimum rating.                                                                                                                                              | functionality | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 11 | Every product belongs to exactly one of the three supported categories: Tech Gadgets, Home Goods, or Fitness.                                                                                                                                                                                | constraint    | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 12 | Every displayed result satisfies all active search, category, price, and rating criteria; when multiple tags are selected, each result has at least one selected tag.                                                                                                                        | constraint    | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 13 | Changing a search, filter, or sort criterion updates the product set, order, and result count without requiring a manual page reload.                                                                                                                                                        | interaction   | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 14 | Selecting a product's image or name opens the detail page for that same product.                                                                                                                                                                                                             | interaction   | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 15 | Each product's purchase action opens the corresponding product page on an external retailer website.                                                                                                                                                                                         | interaction   | FAIL    | All seeded affiliate links point to example.com documentation placeholders, such as https://example.com/sony-xm5, rather than actual retailer product pages.                                                                                                  |
| WebTestBench_0001 | 16 | For every complete product record, the detail page displays the matching name, category, rating, price, brief description, full review, tags, key specifications, image, and retailer action.                                                                                                | content       | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 17 | Every seeded product uses relevant category and tag values, and those values remain consistent between browsing and detail views.                                                                                                                                                            | content       | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 18 | A product's name, category, price, rating, and brief description remain consistent across homepage, category, list, and detail views, while the detail view presents its complete review and specifications without truncation.                                                              | content       | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0001 | 19 | Product additions and edits remain available after the page is reloaded or revisited in a later session.                                                                                                                                                                                     | functionality | FAIL    | Products are held only in React component state. Reloading the page recreates the initial product list and discards every admin addition or edit.                                                                                                             |
| WebTestBench_0001 | 20 | A product cannot be published until it has the content needed for a usable recommendation, including a name, supported category, non-negative price, rating from 0 to 5, brief description, full review, image, valid external retailer link, and at least one complete specification.       | constraint    | FAIL    | The form validates only name and brief description. It accepts a product with a blank image, retailer link, review, and specifications and a zero price, producing an incomplete detail page with an empty purchase destination.                              |
| WebTestBench_0001 | 21 | Whenever category filtering is offered, choosing a different category updates both the active category and the displayed products.                                                                                                                                                           | interaction   | FAIL    | On a category page, the category selector offers all categories but its change handler immediately overwrites every selection with the route's original category, so choosing another category has no effect.                                                 |
| WebTestBench_0001 | 22 | An invalid product or category identifier shows a clear not-found state and does not display unrelated product data.                                                                                                                                                                         | constraint    | PASS    |                                                                                                                                                                                                                                                               |
| WebTestBench_0003 | 1  | Employers can publish a job listing with a title, industry, employment type, location, valid salary range, description, and required skills; the new listing appears in the employer dashboard and the job-seeker listing.                                                                   | functionality | PASS    |                                                                                                                                                                                                                                                               |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                                               | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                           |
|-------------------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0003 | 2  | Employers can delete one of their posted jobs only after confirming the action; the job and its applications are then removed from the employer dashboard and the job-seeker listing.                                                                                                   | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 3  | The employer dashboard lists all active jobs posted by the current employer and shows accurate active-job and total-application counts.                                                                                                                                                 | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 4  | The employer dashboard lets the employer open each posted job's application list and view the applications associated with that job.                                                                                                                                                    | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 5  | Job seekers can browse the complete list of available jobs see an accurate result count and open any listing to view its details.                                                                                                                                                       | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 6  | Job seekers can filter listings by industry, location, and employment type individually or in combination; clearing the criteria restores the full list.                                                                                                                                | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 7  | Keyword search matches job titles, company names, and required skills without case sensitivity and immediately updates the visible results and count.                                                                                                                                   | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 8  | On a job details page, a job seeker can submit an application with a name, syntactically valid email address, optional phone number, and message and receives a clear success confirmation.                                                                                             | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 9  | A job application cannot be submitted when the required name email or message is missing or when the email syntax is invalid; the invalid field is identified to the user.                                                                                                              | constraint    | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 10 | A job cannot be published until its title, industry, employment type, location, description, salary range, and at least one required skill are provided; the salary values must be non-negative with the minimum no greater than the maximum, and invalid data produces clear feedback. | constraint    | FAIL    | The form publishes jobs with no industry, employment type, location, or skills and accepts a minimum salary greater than the maximum, producing incomplete or inverted listings without a validation message. |
| WebTestBench_0003 | 11 | A job seeker cannot submit another application for the same job using an email address already used for that job; the existing application remains unchanged.                                                                                                                           | constraint    | FAIL    | The same name and email can be submitted repeatedly for one job and every duplicate is added to that job's application list.                                                                                  |
| WebTestBench_0003 | 13 | A successfully submitted application produces visible confirmation and appears exactly once under the matching job in the employer dashboard with the submitted information.                                                                                                            | interaction   | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 14 | Changing search or filter criteria immediately refreshes the matching jobs and result count including the no-results state; clearing all criteria restores the full list.                                                                                                               | interaction   | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 15 | Every job card displays the job title, company, industry, employment type, location, salary range, posting date, and required skills.                                                                                                                                                   | content       | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 16 | A job details page displays information matching the selected listing: title, company, industry, employment type, location, salary range, posting date, full description, and required skills.                                                                                          | content       | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 17 | For each application, the employer can view the applicant's name, email address, optional phone number, message, and submission date as a distinct record.                                                                                                                              | content       | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0003 | 18 | Employers can edit the details of a published job and the updated information is reflected in both employer and job-seeker views.                                                                                                                                                       | functionality | FAIL    | Published jobs have no edit action or edit route and the job state exposes no update operation.                                                                                                               |
| WebTestBench_0003 | 19 | Published jobs, submitted applications, and job deletions remain in effect after the page is reloaded.                                                                                                                                                                                  | interaction   | FAIL    | All changes exist only in React component memory; reloading restores the original seeded jobs and applications and reverses any in-session changes.                                                           |
| WebTestBench_0003 | 20 | Newly published jobs and newly submitted applications display an accurate submission date relative to the user's current date.                                                                                                                                                          | content       | FAIL    | Records created today are displayed as "Yesterday" because the relative-date calculation rounds any positive same-day elapsed interval up to one full day.                                                    |
| WebTestBench_0004 | 1  | Couples can create their locally saved wedding dashboard by entering both partner names, a future wedding date, a positive budget, and a non-negative whole-number guest count; the saved values are shown on the dashboard.                                                            | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 2  | Couples can reopen their wedding details, change any saved partner name, date, budget, or guest count, save the changes, and see the updated values on the dashboard.                                                                                                                   | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 3  | Users can filter venues by venue type and price range and vendors by service type and price range, individually or in combination; clearing the criteria restores the full corresponding list.                                                                                          | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 4  | Keyword search matches relevant venue and vendor information, including names, descriptions, locations, and vendor specialties, without case sensitivity and updates the visible results and count.                                                                                     | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 5  | Users can add or remove any venue or vendor from their favorites and view the current favorite venues and vendors together in the saved-favorites area.                                                                                                                                 | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 6  | Users can browse planning checklists by category, open a checklist, and view every task with its explanatory guidance.                                                                                                                                                                  | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 7  | Users can save or unsave a whole planning checklist and the dashboard updates its saved-checklist count and preview accordingly.                                                                                                                                                        | functionality | PASS    |                                                                                                                                                                                                               |
| WebTestBench_0004 | 8  | The dashboard accepts only a positive numeric wedding budget; zero, negative, empty, or otherwise invalid values are rejected without replacing the previously saved budget.                                                                                                            | constraint    | FAIL    | The dashboard saves zero or negative budgets because wedding details are stored without validation.                                                                                                           |
| WebTestBench_0004 | 9  | The dashboard accepts only a non-negative whole-number guest count; negative or fractional values are rejected without replacing the previously saved count.                                                                                                                            | constraint    | FAIL    | The dashboard saves negative and fractional guest counts because wedding details are stored without validation.                                                                                               |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                       | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                                                         |
|-------------------|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0004 | 10 | Favorite venues, favorite vendors, and saved checklists remain available after navigating away and after reloading the application.                                                                                             | constraint    | PASS    | The date field and save action have no future-date validation, so a non-future date can be saved.<br>Venue "View Details" and vendor "View Profile" actions have no handlers, and the application defines no listing-detail routes.         |
| WebTestBench_0004 | 11 | A saved wedding date must be a valid future calendar date; current or past dates are rejected without replacing the previously saved date.                                                                                      | constraint    | FAIL    |                                                                                                                                                                                                                                             |
| WebTestBench_0004 | 12 | Users can open any venue or vendor listing to view its complete details.                                                                                                                                                        | interaction   | FAIL    |                                                                                                                                                                                                                                             |
| WebTestBench_0004 | 13 | Users can mark and unmark checklist tasks; the checklist percentage updates immediately, aggregate dashboard progress updates accordingly, and completion state remains after reload.                                           | interaction   | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0004 | 14 | Visible venue or vendor results and their count update immediately as search or filter criteria change, without a separate submit action; clearing all criteria restores the full list.                                         | interaction   | PASS    | Cancel only closes the editor; it does not reset the draft, so cancelled values reappear when the editor is reopened.                                                                                                                       |
| WebTestBench_0004 | 15 | After valid wedding details are saved, the dashboard displays the couple's names, formatted wedding date, budget, guest count, planning progress, favorite counts or previews, and saved-checklist counts or previews.          | content       | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0004 | 16 | Each venue and vendor listing presents its name, location, category or service type, price range, rating, and summary; venues also show guest capacity and vendors show offered specialties.                                    | content       | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0004 | 17 | The checklist collection covers planning, ceremony, reception, attire, and honeymoon topics, with at least one checklist per category and a title and explanatory description for every task.                                   | content       | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0004 | 18 | When search and filter criteria are combined, every visible result satisfies all active criteria, the displayed result count is accurate, a clear no-match state appears for zero results, and clearing restores the full list. | content       | PASS    | Work and education date fields have no past-date bounds or cross-field validation. In August 2026, the runtime accepted a work entry starting in December 2035 and ending in January 2030 and displayed it in the preview without feedback. |
| WebTestBench_0004 | 19 | If a user changes wedding details and selects Cancel, the saved dashboard values remain unchanged and reopening the editor shows the last saved values rather than the cancelled draft.                                         | interaction   | FAIL    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 1  | Users can create a new resume with blank contact, summary, experience, education, and skills content and begin editing it immediately.                                                                                          | functionality | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 2  | Users can choose among the Modern, Classic, and Minimal resume styles; changing the style updates the preview without losing the resume's content.                                                                              | functionality | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 3  | Users can enter and edit contact details and a professional summary, add, edit, remove, and reorder work and education entries, and add, change, or remove skills.                                                              | functionality | PASS    | Resume state is kept only in React component memory. Reloading discards all in-session changes and restores the original three seeded resumes.                                                                                              |
| WebTestBench_0005 | 4  | Users can change the order of resume sections and hide or show individual sections, and the preview reflects the resulting layout.                                                                                              | functionality | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 5  | Users can view the active resume in a preview, hide the preview, and show it again without losing editor content or the selected template.                                                                                      | functionality | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 6  | Users can duplicate an existing resume, give the copy a different title, and edit the copy without changing the source resume.                                                                                                  | functionality | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 7  | Users can create and manage multiple distinct resume variations for different applications during the current session.                                                                                                          | functionality | PASS    | Changes to resume fields and the selected template appear in the visible preview immediately, without requiring a manual save or refresh.                                                                                                   |
| WebTestBench_0005 | 8  | Users can search saved resumes by title without case sensitivity and order the results by creation date from newest to oldest or oldest to newest; clearing the search restores all resumes.                                    | functionality | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 9  | For each past work or education entry, the end month must not precede the start month and neither date may be in the future; invalid periods are rejected with clear feedback.                                                  | constraint    | FAIL    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 10 | Experience and education dates use a consistent month/year format, and marking a work position as current clears and disables its end date.                                                                                     | constraint    | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 11 | Created, edited, duplicated, renamed, and deleted resume versions remain unchanged after the application is reloaded.                                                                                                           | constraint    | FAIL    | Changing a section's order or visibility updates the preview immediately and preserves the new layout while navigating within the current session.                                                                                          |
| WebTestBench_0005 | 12 | Changes to resume fields and the selected template appear in the visible preview immediately, without requiring a manual save or refresh.                                                                                       | interaction   | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 13 | Changing a section's order or visibility updates the preview immediately and preserves the new layout while navigating within the current session.                                                                              | interaction   | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 14 | Resume edits are retained automatically when the user moves between editor areas, returns to the dashboard, and reopens the resume during the current session; the dashboard reflects that the resume was updated.              | interaction   | PASS    |                                                                                                                                                                                                                                             |
| WebTestBench_0005 | 15 | For the selected style, the preview renders every visible section that has content and keeps the complete resume readable through scrolling when it exceeds the available preview area.                                         | content       | PASS    | Contact details, summary, work experience, education, and skills entered for a resume remain present when switching editor areas, returning to the dashboard, and reopening that resume during the current session.                         |
| WebTestBench_0005 | 16 | Contact details, summary, work experience, education, and skills entered for a resume remain present when switching editor areas, returning to the dashboard, and reopening that resume during the current session.             | content       | PASS    |                                                                                                                                                                                                                                             |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                            | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                                 |
|-------------------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0005 | 17 | After a resume is duplicated, editing any content in the copy does not alter the corresponding content in the original, and editing the original does not alter the copy.                            | content       | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0005 | 18 | After a resume is renamed, the same non-empty title is shown in its editor and on its dashboard card.                                                                                                | content       | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0005 | 19 | Users can narrow saved resumes by an exact creation date or a creation-date range and clear the date criterion to restore all matching titles.                                                       | functionality | FAIL    | The dashboard exposes title search and creation-date sorting only; it has no creation-date search or filter control. Searching for a displayed date produces no results.                                            |
| WebTestBench_0005 | 20 | Every section heading that the editor allows the user to customize is shown with that customized text in each resume style where the section is visible.                                             | content       | FAIL    | The contact-section title is never rendered by any template, and the Minimal template renders summary content without its editable summary heading, so those exposed heading changes have no effect in the preview. |
| WebTestBench_0005 | 21 | A resume cannot be renamed to an empty or whitespace-only title; the current title remains and the user receives clear validation feedback.                                                          | constraint    | FAIL    | The dashboard rename dialog accepts an empty value and saves a blank resume title with no validation message.                                                                                                       |
| WebTestBench_0006 | 1  | Users can create financial transactions with a type, date, description, amount, and category, and may also record a reference, notes, and tax-related status.                                        | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 2  | Users can view a summary of their financial status, including total balance, current-month income and expenses, net profit or loss, financial trends, spending by category, and recent transactions. | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 3  | Users can search transactions by keywords contained in their descriptions or references.                                                                                                             | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 4  | Users can review an annual tax summary containing taxable income, deductible expenses, net taxable income, category breakdowns, and the corresponding tax-related transactions.                      | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 5  | Users can filter financial transactions by income or expense type, category, and date range.                                                                                                         | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 6  | A valid newly created transaction is retained and shown with the date, description, category, reference, amount, and tax-related status supplied by the user.                                        | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 7  | Users can update or delete existing financial transactions, and the resulting records reflect those changes.                                                                                         | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 8  | Users can create, edit, and delete income or expense categories, including the category name, type, and color.                                                                                       | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 9  | The categories available for a transaction are compatible with its income or expense type, and the assigned category is retained with the transaction.                                               | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 11 | Users can mark or unmark a transaction as tax-related when creating or updating it.                                                                                                                  | functionality | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 12 | When search, type, category, and date criteria are combined, every resulting transaction satisfies all active criteria; removing the criteria restores the full set of transactions.                 | constraint    | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 13 | Financial transactions without a description or amount cannot be created, and the missing required information is identified to the user.                                                            | constraint    | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 15 | A transaction date range whose start date is later than its end date must be rejected with an explanation of the invalid range.                                                                      | constraint    | FAIL    | The app accepts an inverted date range without feedback and can present a misleading empty result.                                                                                                                  |
| WebTestBench_0006 | 16 | Within the current session, adding, updating, or deleting a current-month transaction immediately updates the corresponding financial summaries, reports, and recent activity.                       | interaction   | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 17 | Marking or unmarking a transaction as tax-related updates its recorded tax status and its inclusion in the corresponding annual tax summary.                                                         | interaction   | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 18 | The transaction count and total amount for each category update when categorized transactions are created or changed.                                                                                | interaction   | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 19 | Users can print the tax summary for a selected year without an application error.                                                                                                                    | interaction   | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 20 | Users can view each transaction's date, description, category, reference, amount, and tax-related status, together with income and expense totals for the current result set.                        | content       | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 21 | Current-month income, expenses, net profit or loss, recent activity, and monthly report totals accurately reflect the underlying transactions.                                                       | content       | PASS    |                                                                                                                                                                                                                     |
| WebTestBench_0006 | 22 | When a financial chart represents a non-zero amount, its axis labels must use enough precision to distinguish that amount from zero, including for values below \$500.                               | content       | FAIL    | Non-zero values below \$500 are plotted, but every axis label is displayed as '\$0k', making the chart misleading.                                                                                                  |
| WebTestBench_0006 | 23 | Financial transactions without an assigned category cannot be created, and a missing category is identified to the user.                                                                             | constraint    | FAIL    | The app accepts transactions without a category; they appear without a category in transaction records and as 'Unknown' in tax summaries.                                                                           |
| WebTestBench_0006 | 24 | Deleting a category assigned to existing transactions must be blocked or require reassignment so no transaction is left with a blank or Unknown category.                                            | constraint    | FAIL    | The app deletes assigned categories without checking or reassigning their transactions, leaving those transactions uncategorized and reports showing 'Unknown'.                                                     |
| WebTestBench_0010 | 1  | Users can browse restaurants and view each restaurant's name, cuisine, average price per diner, rating, location, description, and notable dining features.                                          | functionality | FAIL    | The app shows symbolic price tiers such as '\$\$\$\$' but provides no average price per diner.                                                                                                                      |
| WebTestBench_0010 | 2  | Users can search restaurants by name, cuisine, or neighborhood and can combine the search with cuisine and price-range filters; every displayed restaurant satisfies all active criteria.            | functionality | PASS    |                                                                                                                                                                                                                     |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                           | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                                                                     |
|-------------------|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0010 | 3  | Users can create a reservation by selecting a future date, an available time, and a party size, providing the required contact information, and submitting it; a successful booking is confirmed and appears among upcoming reservations with the supplied details. | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 4  | Users can view their upcoming reservations and cancel one after confirming the action; the cancelled reservation is removed from upcoming reservations and retained with a cancelled status in reservation history.                                                 | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 5  | Users can submit a one-to-five-star rating, visit date, name, and written review for a restaurant, and the new review is retained and displayed only with that restaurant.                                                                                          | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 6  | During reservation, users can apply a supported coupon, see the offered discount, remove it if desired, and have the applied coupon recorded with the resulting reservation.                                                                                        | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 7  | Past dates are unavailable for new reservations, while a valid future date can be combined with one of the restaurant's available reservation times.                                                                                                                | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 8  | Users can change the date, time, and party size of an upcoming reservation, and the reservation list retains and displays all submitted changes.                                                                                                                    | functionality | FAIL    | The modification flow can change time and party size but provides no way to change the reservation date.<br>Review submission is not linked to reservations or user identity, so anyone can review any restaurant repeatedly without a completed visit. |
| WebTestBench_0010 | 10 | A review can be submitted only for a restaurant covered by the user's completed reservation, and each completed reservation can authorize at most one review.                                                                                                       | constraint    | FAIL    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 11 | A coupon is accepted only while all of its stated eligibility conditions are met, including first-reservation, day, and party-size rules, and no reservation can have more than one coupon applied.                                                                 | constraint    | FAIL    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 13 | A reservation cannot be created without a date, time, guest name, valid email address, and phone number, and the missing or invalid information is identified to the user.                                                                                          | constraint    | PASS    | Coupon eligibility is only checked for minimum party size at application time: WEEKEND20 applies on weekdays, WELCOME10 is not limited to a first reservation, and GROUP15 remains applied after reducing the party below six.                          |
| WebTestBench_0010 | 14 | For an upcoming reservation, users can open its modification choices and save supported changes, while cancellation requires an explicit confirmation before taking effect.                                                                                         | interaction   | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 15 | Creating, modifying, or canceling a reservation produces immediate visible confirmation, and the reservation views reflect the new state in the same session without a manual reload.                                                                               | interaction   | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0010 | 16 | Users can distinguish upcoming reservations from past or cancelled reservations and can view each reservation's restaurant, date, time, party size, status, location, and applied coupon when present.                                                              | content       | PASS    | The list shows The Golden Fork as 4.8 with 312 reviews, while its detail page immediately shows 4.5 with 2 reviews; locally submitted reviews affect only the detail summary, leaving the list stale.                                                   |
| WebTestBench_0010 | 17 | A restaurant's displayed rating and review count remain consistent between the restaurant list and its details, and submitting a review updates every displayed summary derived from those reviews.                                                                 | content       | FAIL    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 1  | A visitor can register a unique username and email with a password of at least six characters, is signed in as the new user, and can sign out and sign back in with the same credentials during the current session; duplicate usernames or emails are rejected.    | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 2  | A registered user can edit their bio and comma-separated interests, save the changes, and see the saved bio and interests on their own public profile.                                                                                                              | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 3  | A logged-in user can publish a discussion with a category, title, and body; the published discussion displays those values and appears in its selected category.                                                                                                    | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 4  | A logged-in user can add a non-empty reply to a discussion, and the reply appears without a manual reload while the discussion's displayed reply total increases by one.                                                                                            | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 5  | A logged-in user can add or remove their single upvote on another user's discussion or reply, and the displayed count and selected state update immediately.                                                                                                        | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 6  | Submitting a keyword search returns only discussions whose title or body contains the keyword, case-insensitively, and an unmatched keyword produces an empty result with clear feedback.                                                                           | functionality | FAIL    | The visible search field accepts a query, but submitting it leaves the unfiltered discussion list on screen.                                                                                                                                            |
| WebTestBench_0019 | 7  | Selecting a discussion category shows only discussions assigned to that category, and selecting a different category changes the result set to that category.                                                                                                       | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 8  | The Most Active discussion list is ordered by reply count from highest to lowest and updates its order when reply activity changes the ranking.                                                                                                                     | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 10 | From a discussion or reply, users can open the author's public profile and view the username, bio, interests, join date, reputation, and authored discussions.                                                                                                      | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 13 | Within the same loaded session, a user's saved profile details, authored discussions, replies, and vote state remain intact after logging out and logging back in.                                                                                                  | functionality | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 14 | A discussion cannot be published without a category, non-blank title, and non-blank body; the missing information is identified and no discussion is created.                                                                                                       | constraint    | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 16 | The Trending discussion list is ordered by upvote count from highest to lowest and updates its order when voting changes the ranking.                                                                                                                               | content       | PASS    |                                                                                                                                                                                                                                                         |
| WebTestBench_0019 | 17 | An account can be signed in only with its registered password, and an incorrect password is rejected without creating a logged-in session.                                                                                                                          | constraint    | FAIL    | Login looks up only the email address and ignores the supplied password, so any non-empty password signs in to an existing account.                                                                                                                     |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                         | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                                                        |
|-------------------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0019 | 18 | Registered accounts, signed-in status, profile changes, authored discussions, replies, and vote state survive a page reload and remain available when the user returns.                                                                                           | functionality | FAIL    | All users, authentication, discussions, replies, profile edits, and votes are held only in React memory and reset to seed data on reload.                                                                                                  |
| WebTestBench_0019 | 19 | Every discussion's displayed reply total matches the number of replies listed on its detail view, and adding a reply keeps both totals consistent.                                                                                                                | content       | FAIL    | Seeded discussions report large reply totals that do not match their reply lists; for example, one discussion shows 67 replies but lists only one, then changes to 68 versus two after a new reply.                                        |
| WebTestBench_0019 | 20 | Trending and most-active views that describe recent popularity use a stated recent time window and rank discussions only by interactions within that window.                                                                                                      | content       | FAIL    | The app has no recent-activity window and ranks all seeded discussions by all-time upvotes or reply counts, even when every displayed discussion is more than two years old.                                                               |
| WebTestBench_0019 | 21 | Category and community counts accurately reflect the discussions, replies, and members available in the forum and update when those records are created.                                                                                                          | content       | FAIL    | Category and community counts are hard-coded and contradict the available data; for example, Technology is labelled with 243 discussions while its category page contains two, and creating content does not update the counters.          |
| WebTestBench_0019 | 22 | An unauthenticated user who attempts to start a discussion is taken to a usable sign-in screen and can return to discussion creation after authenticating.                                                                                                        | constraint    | FAIL    | Opening the new-discussion route while signed out renders only an empty notifications region and neither redirects to nor displays the sign-in screen.                                                                                     |
| WebTestBench_0023 | 1  | Sellers can create a listing by attaching one or more of their own product photos and supplying a title, description, non-negative price, category, condition, location, and upcycled status; the created listing retains and displays the submitted information. | functionality | FAIL    | The listing form has no photo upload or URL input and can only attach up to four fixed stock images.                                                                                                                                       |
| WebTestBench_0023 | 2  | Sellers can edit the title, description, photos, price, category, condition, location, and upcycled status of an existing listing they own, and the saved listing displays the changes.                                                                           | functionality | FAIL    | Owned listings can be viewed, marked sold or available, or deleted, but there is no way to edit their details.                                                                                                                             |
| WebTestBench_0023 | 3  | Sellers can mark an owned listing as sold or available and can delete it after confirmation; their listing view immediately reflects the resulting status or removal.                                                                                             | functionality | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 4  | A listing marked sold or deleted is no longer offered to buyers in browse results and cannot be newly added to a cart or used to start a purchase request.                                                                                                        | functionality | PASS    | The price control exposes only the minimum-price thumb, so buyers cannot set a maximum price. In addition, newly created listings have no computed distance and are treated as zero miles away, causing them to pass every distance limit. |
| WebTestBench_0023 | 5  | A user can register with a name, unique valid email address, password of at least six characters, and location; successful registration is confirmed and leaves the new user signed in.                                                                           | functionality | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 6  | Buyers can combine category, minimum and maximum price, condition, distance, and upcycled-only filters and sort the available listings by newest, price, or distance; every result satisfies all active criteria.                                                 | functionality | FAIL    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 7  | Buyers can search available listings case-insensitively by words in the title, description, or location; every result matches the query, and clearing it restores the unsearched result set.                                                                      | functionality | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 8  | A signed-in buyer can add an available listing they do not own to the cart; the item is retained once with its title, photo, location, and price, and the cart total is correct.                                                                                  | functionality | PASS    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 9  | A signed-in buyer can submit a non-empty purchase-request message for an available listing they do not own and receives visible confirmation that it was submitted.                                                                                               | functionality | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 10 | A login attempt using an email that is not registered is rejected and does not create an authenticated session.                                                                                                                                                   | constraint    | PASS    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 11 | Registration rejects an email already used by an account, including a case-only variation, without replacing the existing account.                                                                                                                                | constraint    | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 12 | A registered account can be authenticated only with its correct password; an incorrect password is rejected without creating a signed-in session.                                                                                                                 | constraint    | FAIL    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 13 | Seller-management actions are available only for listings owned by the signed-in seller and are not exposed when that user views another seller's listing.                                                                                                        | constraint    | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 14 | A listing cannot be created without at least one photo, a title, description, valid price, category, condition, and location, and the missing required information is identified to the seller.                                                                   | constraint    | PASS    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 15 | A listing price must be a finite number greater than or equal to zero; negative or non-numeric prices are rejected without creating the listing.                                                                                                                  | constraint    | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 16 | Attempting to register an existing email produces immediate, visible duplicate-account feedback and keeps the registration form available for correction.                                                                                                         | interaction   | PASS    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 17 | Search results and the available-item count update as the buyer types or clears a query, without requiring a separate search submission.                                                                                                                          | interaction   | PASS    |                                                                                                                                                                                                                                            |
| WebTestBench_0023 | 18 | Changing any available filter or sort choice immediately updates the displayed listings and result count, and clearing all filters restores the unfiltered available listings.                                                                                    | interaction   | PASS    | Login ignores the password and authenticates any registered email with any non-empty password; registration does not retain passwords for later verification.                                                                              |
| WebTestBench_0023 | 19 | Adding a listing to the cart produces immediate visible confirmation and updates the cart count, line item, and total without a manual reload.                                                                                                                    | interaction   | PASS    |                                                                                                                                                                                                                                            |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                 | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                   |
|-------------------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0023 | 20 | Each listing's details present its photos, title, description, price, category, condition, location, seller, listing date, availability, and upcycled status consistently with the listing data.                                                          | content       | PASS    |                                                                                                                                                                                       |
| WebTestBench_0023 | 21 | Choosing a category from the category-browsing page opens browse results containing only available listings in that category and shows a matching result count.                                                                                           | functionality | FAIL    | Category choices navigate with a category query parameter, but the browse page never reads it and displays all available listings.                                                    |
| WebTestBench_0023 | 22 | After a buyer submits a purchase request, the relevant seller can view the requested item and buyer message, and a cart request is reported as sent only after a request has been recorded for each cart item.                                            | functionality | FAIL    | There is no seller-facing purchase-request view. The cart action shows success and clears the cart without recording any purchase requests.                                           |
| WebTestBench_0023 | 23 | Creating or managing listings, adding items to or viewing the cart, and submitting purchase requests require a signed-in account; unauthenticated attempts are blocked with a clear sign-in prompt or redirect.                                           | constraint    | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 1  | Users can sort the tasks in the current view by priority, with high-priority tasks before medium-priority tasks and medium-priority tasks before low-priority tasks.                                                                                      | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 2  | Users can sort the tasks in the current view by deadline, with the earliest deadlines first and tasks without deadlines after dated tasks.                                                                                                                | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 3  | When a task is blocked by an incomplete prerequisite, users can still change its description and priority, but cannot change its title or deadline.                                                                                                       | functionality | FAIL    | Blocked tasks have no usable edit action, so their description and priority cannot be changed.                                                                                        |
| WebTestBench_0045 | 4  | Users can create a task with a non-empty title; the new task is retained in the active list and the active-task count increases.                                                                                                                          | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 5  | When creating a task, users can provide a title, optional description, optional deadline, priority, and zero or more prerequisites, and the created task retains the submitted details.                                                                   | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 6  | Users can update an unblocked task's title, description, deadline, priority, and prerequisites, and can delete a task; saved changes are retained and a deleted task is removed from the task views.                                                      | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 7  | Users can mark an eligible active task as completed; it leaves the active view, appears in the completed view, and the active and completed counts update.                                                                                                | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 8  | Users can switch between active and completed task views; each view shows only tasks in the corresponding state and displays the corresponding count.                                                                                                     | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 9  | Users can assign one or more existing tasks as prerequisites of another task, and the saved task retains and displays the resulting dependency state.                                                                                                     | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 10 | A task with any incomplete prerequisite is visibly distinguished as blocked, and its completion action remains unavailable until all prerequisites are complete.                                                                                          | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 11 | A blocked task displays a prominent warning that names its single incomplete prerequisite or reports the number of incomplete prerequisites when there are several.                                                                                       | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 12 | A task cannot be marked completed while any prerequisite is incomplete; after all prerequisites are completed, it becomes eligible for completion.                                                                                                        | constraint    | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 13 | Users cannot create a direct or indirect circular dependency between tasks; any task whose selection would create a cycle is unavailable as a prerequisite.                                                                                               | constraint    | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 14 | A new task cannot be created from an empty or whitespace-only title, and attempting to do so leaves the task list unchanged.                                                                                                                              | constraint    | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 17 | Completing a task does not delete it: it remains available in the completed view until the user reopens or deletes it.                                                                                                                                    | constraint    | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 18 | Hovering over a visual connection between dependent tasks displays the related dependency chain and the completion state of its tasks.                                                                                                                    | interaction   | FAIL    | Hovering the dashed dependency connection does not display the dependency-chain pop-up.                                                                                               |
| WebTestBench_0045 | 20 | Hovering over a blocked task's prerequisite warning displays the dependency chain and identifies which prerequisite tasks are still pending.                                                                                                              | interaction   | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 21 | Each task displays its title and priority, displays its description and deadline when supplied, exposes any blocking prerequisite state, and visibly distinguishes completed tasks from active tasks.                                                     | content       | PASS    |                                                                                                                                                                                       |
| WebTestBench_0045 | 22 | Reopening a prerequisite of a completed dependent task cannot leave that dependent task simultaneously completed and blocked; the system either prevents the prerequisite from being reopened or returns affected dependents to a valid incomplete state. | constraint    | FAIL    | A prerequisite can be reopened after its dependent is completed, leaving the dependent in the completed view with an incomplete-prerequisite warning and disabled completion control. |
| WebTestBench_0054 | 1  | The document list identifies each document's module; choosing a module shows only documents from that module, and choosing all modules restores the complete list.                                                                                        | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0054 | 2  | Selecting a document displays its title, description, version and update date, content sections, and related documents.                                                                                                                                   | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0054 | 3  | From an expanded content section or the related-documents list, choosing a valid document reference makes the referenced document the visible document.                                                                                                   | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0054 | 4  | Collapsing an expanded content section hides that section's body and expanded reference links while leaving the section available to reopen.                                                                                                              | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0054 | 5  | Expanding a collapsed content section restores that section's body and any reference links it contains.                                                                                                                                                   | functionality | PASS    |                                                                                                                                                                                       |
| WebTestBench_0054 | 6  | Users can select a knowledge-graph node and receive a visible selection indication, and can change the graph zoom and reset the graph view with visible zoom feedback.                                                                                    | functionality | PASS    |                                                                                                                                                                                       |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                   | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                  |
|-------------------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0054 | 7  | The knowledge graph represents every available document across all modules and their related-document connections, and its view controls allow the complete graph to be brought into view.                                                                  | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 8  | The application provides Documents, Knowledge Graph, and Favorites views, and users can switch among all three.                                                                                                                                             | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 9  | Switching among Documents, Knowledge Graph, and Favorites preserves the selected document, each section's expanded or collapsed state, and the graph's zoom and pan state.                                                                                  | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 10 | Favorites are retained after reload, their navigation count updates immediately, their markers survive document filtering and sorting, and the Favorites view continues to expose every favorited document regardless of a module filter used in Documents. | functionality | FAIL    | A module filter selected in Documents remains active but hidden in Favorites, so favorited documents from other modules disappear from the Favorites list even though the count still includes them. |
| WebTestBench_0054 | 12 | A document never lists itself as either a content-section reference or a related document.                                                                                                                                                                  | constraint    | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 13 | The knowledge graph contains one node for every available document, associates each node with the document's module, and draws connections for the available related-document relationships.                                                                | constraint    | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 15 | Each content section has a clear collapsed or expanded indicator, and the indicator's state agrees with whether the section body and expanded links are visible.                                                                                            | interaction   | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 16 | Choosing a document node in the knowledge graph opens that document in the document view without requiring a separate tab change, while preserving the graph view for later return.                                                                         | interaction   | FAIL    | Choosing a graph node only highlights it and records the selection; the application remains on the graph until the user manually switches to Documents.                                              |
| WebTestBench_0054 | 17 | Choosing a reference link shown for a collapsed section leaves the current document open and displays a prompt to expand the section; after expansion, choosing the same link opens its target document.                                                    | interaction   | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0054 | 18 | When search or module criteria match no documents, the document list displays a clear no-results message, and clearing those criteria restores matching documents.                                                                                          | interaction   | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 1  | Users can view the complete list of heritage items, together with an accurate visible result count.                                                                                                                                                         | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 2  | Selecting a loaded hotspot in the panorama opens a usable information overlay for the matching heritage item, including its title, image, category, region, brief description, and an action to view full details.                                          | functionality | FAIL    | The hotspot overlay is displaced down and right, leaving most content and actions outside the viewport. Framar Motion's inline transform overrides the centering transform classes.                  |
| WebTestBench_0063 | 3  | A heritage detail page presents the matching title, full introduction, category, region, inclusion year when available, and a gallery containing multiple images.                                                                                           | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 4  | Users can search the heritage list, filter it by any supported category or region, and sort it by name, year, or region; the displayed items and result count update accordingly.                                                                           | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 5  | After visiting a heritage detail page and returning to the panorama, the previous camera orientation and zoom level are restored.                                                                                                                           | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 7  | Every displayed heritage item belongs to one supported category: Traditional Crafts, Performing Arts, Folk Activities, or Traditional Art.                                                                                                                  | constraint    | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 8  | A valid project detail link opens the matching heritage item; an invalid project identifier shows a clear not-found state and a way back to the homepage without displaying unrelated project details.                                                      | constraint    | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 9  | Every heritage card displays the item's title, brief description, region or province, and category.                                                                                                                                                         | constraint    | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 10 | Category-browsing navigation from a heritage detail page returns the user to the project list on the homepage.                                                                                                                                              | interaction   | FAIL    | The detail-page category link only appends \#categories to the current /heritage/:id URL, where no categories section exists, instead of navigating to /\#categories.                                |
| WebTestBench_0063 | 11 | Images presented for a heritage item are relevant to and consistent with the item's name and description.                                                                                                                                                   | content       | FAIL    | Several generic Unsplash stock photos are unrelated to their associated heritage items; for example, the Suzhou Embroidery gallery includes unrelated imagery.                                       |
| WebTestBench_0063 | 12 | Users can rotate the panorama through a full 360 degrees and continue to see and interact with hotspots as the view wraps around; zooming does not cause hotspots to be permanently lost.                                                                   | interaction   | FAIL    | The camera yaw is never normalized. After a 360-degree drag, hotspot visibility compares raw angles, so all hotspots disappear until the user rotates all the way back.                              |
| WebTestBench_0063 | 13 | The detail-page image gallery lets users select each available image and updates the main image area or its explicit failure state to match the selected image.                                                                                             | interaction   | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0063 | 14 | Search, filter, and sort selections remain applied after a user opens a heritage detail page and returns to the project list.                                                                                                                               | interaction   | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0074 | 1  | Users can create an entry with values for configured fields; the new entry appears immediately and remains present after the page is reloaded.                                                                                                              | functionality | FAIL    | Entries and other database changes exist only in in-memory component state, so reloading restores the seed data.                                                                                     |
| WebTestBench_0074 | 2  | Users can add fields and choose text, number, date, or checkbox as each field's type; the new field appears in entry forms and both data views.                                                                                                             | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0074 | 3  | Users can enter and save a text string, number, calendar date, and checkbox status for an entry, and the saved values are retained and shown in both data views.                                                                                            | functionality | PASS    |                                                                                                                                                                                                      |
| WebTestBench_0074 | 4  | Users can switch between table and card views without changing the current entries or their field values.                                                                                                                                                   | functionality | PASS    |                                                                                                                                                                                                      |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                  | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                                                                                                   |
|-------------------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0074 | 5  | Users can sort the current view by any configured field in ascending or descending order; table and card views retain independent sort selections while the user switches between them.                    | functionality | PASS    | The app provides free-text search but has no filter control or field-specific filtering logic.                                                                                                                                                                                        |
| WebTestBench_0074 | 6  | Users can define field-specific filter criteria, view only matching entries, and clear the filters to restore the unfiltered data.                                                                         | functionality | FAIL    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 7  | When no field sort is active, users can manually reorder entries in table and card views; each view retains its own manual order while the user switches between them.                                     | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 8  | A number field accepts valid numeric values and prevents a non-numeric value from being saved as that field's value.                                                                                       | constraint    | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 9  | A date field accepts valid calendar dates and prevents an invalid date value from being saved as that field's value.                                                                                       | constraint    | PASS    | Filtering is not implemented, so criteria cannot be applied or cleared.<br>Card view promotes the first text field to an unlabeled title and omits it completely when its value is empty, so neither the field nor an empty-value indicator is shown.                                 |
| WebTestBench_0074 | 11 | Sorting changes only the order of entries; it preserves the entry count and every saved field value.                                                                                                       | constraint    | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 12 | Filtering includes exactly the entries that satisfy the active criteria; clearing the filters restores all entries without modifying or deleting data.                                                     | constraint    | FAIL    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 13 | Every entry and each configured field value are displayed consistently in both table and card views, including a clear representation for an empty value.                                                  | constraint    | FAIL    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 14 | Field-type choices are clearly labeled during configuration, and entry forms provide type-appropriate controls for text, number, date, and checkbox fields.                                                | constraint    | PASS    | No data-import action or import-processing logic exists, despite the brief's requirement that field configuration precede import.<br>Search compares raw values, so displayed checkbox labels such as "Yes" and "No" and formatted date text cannot be found by those visible values. |
| WebTestBench_0074 | 15 | Checkbox values have clearly distinguishable checked and unchecked states in entry forms and both data views.                                                                                              | constraint    | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 17 | Quick search returns entries whose text values contain the query case-insensitively, updates the visible result count, and restores all entries when cleared.                                              | interaction   | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 18 | Selecting a sort field immediately reorders the visible entries, identifies the active field and direction, and leaves the app usable.                                                                     | interaction   | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0074 | 19 | Users can import data after defining a field schema, and the app rejects imported values that do not match the configured field types without corrupting existing data.                                    | functionality | FAIL    | Search uses case-insensitive substring matching only; entering 'Resme' returns zero results instead of matching 'Professional Resume'.                                                                                                                                                |
| WebTestBench_0074 | 20 | Quick search matches the field values shown to users, including formatted dates and checkbox states, rather than only internal raw values.                                                                 | constraint    | FAIL    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 1  | Users can browse the template library and see the available templates with their authors and descriptions.                                                                                                 | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 2  | Selecting a template opens its customizable form and template details.                                                                                                                                     | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 3  | Users can enter values for every field defined by the selected template.                                                                                                                                   | functionality | PASS    | An invalid email such as 'not-an-email' is accepted, and built-in date-like text fields accept values such as 'not a date'; preview still becomes available.                                                                                                                          |
| WebTestBench_0089 | 4  | After all required fields are valid, users can preview the generated result with their values substituted into the template.                                                                               | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 5  | Users can submit a custom plain-text or Markdown template with a title, description, author, category, scenario, content, and at least one defined field.                                                  | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 6  | After a custom template is successfully submitted, it appears in the current browser's template library and can be selected and used.                                                                      | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 7  | Each template card displays its title, author, description, category, scenario, usage count, and update date.                                                                                              | functionality | PASS    | Custom templates cannot be submitted without a nonblank title, description, author, content, and at least one defined field.                                                                                                                                                          |
| WebTestBench_0089 | 8  | For a built-in Markdown template, filling all required fields produces a preview with every declared placeholder replaced and Markdown headings, emphasis, separators, and line breaks rendered correctly. | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 9  | Users can filter templates by category and scenario and sort results by popularity, update recency, creation age, or title; the visible result count and order update accordingly.                         | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 10 | Search tolerates a small typo in a template keyword and still returns or suggests the intended matching template.                                                                                          | functionality | FAIL    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 11 | Users can add or remove a template from favorites and open a favorites-only list.                                                                                                                          | functionality | PASS    | Custom templates cannot be submitted without a nonblank title, description, author, content, and at least one defined field.                                                                                                                                                          |
| WebTestBench_0089 | 12 | Within the favorites-only view, users can search and filter their saved templates.                                                                                                                         | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 13 | Users can copy the generated result to the clipboard in the selected template's declared format, with visible success feedback.                                                                            | functionality | PASS    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 14 | Fields with a declared format, such as email or date, reject invalid values or keep preview unavailable until the value is valid.                                                                          | constraint    | FAIL    |                                                                                                                                                                                                                                                                                       |
| WebTestBench_0089 | 16 | Custom templates cannot be submitted without a nonblank title, description, author, content, and at least one defined field.                                                                               | constraint    | PASS    |                                                                                                                                                                                                                                                                                       |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                 | Class         | Verdict | Defect (FAIL cases)                                                                                                                                                                                               |
|-------------------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0089 | 17 | Values entered into template fields reject or neutralize special characters that would alter generated Markdown structure or formatting.                                                                                                                  | constraint    | FAIL    | Validation blocks only a limited character set and permits Markdown metacharacters such as \#, \*, and \_. Entering '\# Unexpected Heading' in a textarea renders an unintended level-one heading in the preview. |
| WebTestBench_0089 | 19 | The preview remains unavailable until every required field of the selected template has a nonblank, valid value.                                                                                                                                          | constraint    | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 20 | Typing at least two matching characters shows real-time suggestions; choosing a suggestion fills the search and narrows the library to matching templates.                                                                                                | interaction   | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 21 | Search queries, category and scenario filters, favorites-only mode, and sort choices update the visible results and count without a reload.                                                                                                               | interaction   | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 22 | When required selected-template fields are empty, the UI lists the missing fields and keeps preview unavailable.                                                                                                                                          | interaction   | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 24 | Favoriting or unfavoriting a template provides immediate visual feedback and updates the favorites count and list.                                                                                                                                        | interaction   | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 25 | Valid plain-text templates preview as preserved text, and valid Markdown templates render expected Markdown after field substitution.                                                                                                                     | content       | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 26 | Submitting a valid custom template immediately updates the library and visible result count without a reload.                                                                                                                                             | content       | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 27 | A custom template can be submitted only when every declared dynamic field maps to a placeholder in its content and every content placeholder maps to a declared field; generated previews contain no unresolved placeholders.                             | constraint    | FAIL    |                                                                                                                                                                                                                   |
| WebTestBench_0089 | 28 | Favorites and custom templates remain available after reloading the app in the same browser. Every field defined for a custom template remains independently fillable; duplicate labels are rejected or disambiguated before submission.                  | functionality | PASS    | Duplicate labels are accepted and normalized to the same field identifier. Two fields labelled 'Name' both become {{name}}, producing duplicate keys and shared form state rather than independent values.        |
| WebTestBench_0089 | 29 |                                                                                                                                                                                                                                                           | constraint    | FAIL    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 1  | Searching by a case-insensitive job-title, keyword, or description fragment returns exactly the matching competency models; an optional department filter further narrows the results, and no matches produce a clear empty state.                        | functionality | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 2  | Selecting a model from the search results opens the corresponding detail editor with the matching job title, summary metrics, and competency graph.                                                                                                       | functionality | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 3  | Users can drag competency nodes to rearrange the graph layout while existing directed relationships remain attached to the correct competencies and their meaning does not change.                                                                        | functionality | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 4  | Users can change the weight of a competency relationship, after which the displayed relationship weight, its visual emphasis, and any dependent analysis update immediately.                                                                              | functionality | FAIL    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 5  | Users can disable and later re-enable a competency; the node and its relationships clearly show the disabled state, disabled competencies are excluded from coverage, gap, and critical-path calculations, and repeated toggles act on the current state. | functionality | FAIL    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 6  | Users can browse the template library and compare each available job-role template by its role, department, competency count, relationship count, and representative keywords.                                                                            | functionality | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 7  | Applying a template replaces the edited model consistently, so the title, metrics, competency graph, relationships, and scenario-adjustment categories all represent the selected template.                                                               | functionality | FAIL    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 8  | Users can save the current edited model as a custom template with a name and description, then retrieve the same configuration from a My Templates collection after leaving and returning.                                                                | functionality | FAIL    | Save Current only shows a success toast. There is no naming or description workflow, custom-template state or storage, or My Templates collection.                                                                |
| WebTestBench_0097 | 9  | Users can switch among Recruitment, Training, and Promotion simulation scenarios, and the scenario description and category adjustments update to match the selected scenario.                                                                            | functionality | PASS    | Save only reports success through a toast. Reopening the model or reloading reconstructs it from static seed data and loses all edits.                                                                            |
| WebTestBench_0097 | 10 | Users can choose category-level competency and weight adjustments for a scenario, run the simulation to update coverage, average level, critical-path and risk-gap outputs, and reset the model to its original state.                                    | functionality | PASS    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 11 | After editing a model, users can save and later restore the same levels, weights, and enabled states after leaving and reopening the model or reloading the app.                                                                                          | functionality | FAIL    |                                                                                                                                                                                                                   |
| WebTestBench_0097 | 12 | Direct edits and scenario simulations keep every competency level within 1 to 5 and every competency weight within the supported 10% to 100% range.                                                                                                       | constraint    | PASS    |                                                                                                                                                                                                                   |

Continued on next page

Table F.1 – continued

| Application       | ID | Test case                                                                                                                                                                                                                                                                       | Class       | Verdict | Defect (FAIL cases)                                                                                                                                                                                                                                                              |
|-------------------|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebTestBench_0097 | 13 | Changing an enabled competency's level or weight, or changing which competencies are enabled, immediately updates the displayed coverage percentage and progress indicator without a reload.                                                                                    | interaction | PASS    |                                                                                                                                                                                                                                                                                  |
| WebTestBench_0097 | 15 | The detail editor presents the current model as an interactive directed weighted graph: nodes identify competencies, arrowed connections identify dependency or reinforcement relationships, relationship weights are visible, and users can pan, zoom, and rearrange the view. | content     | PASS    |                                                                                                                                                                                                                                                                                  |
| WebTestBench_0097 | 16 | After a relevant competency-weight or enabled-state change, the critical-path summary and graph highlighting recalculate immediately, form a valid path through enabled competencies, and agree with each other.                                                                | content     | FAIL    | The calculation selects high-weight competencies and some inbound dependencies rather than computing a graph path, can include a disabled dependency, and the graph's stale React Flow state prevents updated node and edge highlighting from being rendered.                    |
| WebTestBench_0097 | 17 | Lowering an enabled high-weight competency below the recommended level immediately creates a risk-gap warning naming that competency; resolving the level or disabling the competency clears it.                                                                                | content     | PASS    |                                                                                                                                                                                                                                                                                  |
| WebTestBench_0097 | 18 | When one or more risk competency gaps are detected, the app prominently displays a text-and-icon alert near the summary metrics that names the affected competencies and suggests corrective action.                                                                            | content     | PASS    |                                                                                                                                                                                                                                                                                  |
| WebTestBench_0097 | 19 | The initially selected simulation scenario is fully initialized: its default adjustments are visible and running it without first switching tabs applies those adjustments and changes the corresponding model outputs.                                                         | interaction | FAIL    | The Recruitment tab is selected initially, but adjustments starts as an empty object and defaults are populated only by a tab-change handler. Running the initial scenario makes no model change while still showing a misleading Simulation Applied success toast.              |
| WebTestBench_0097 | 20 | After any competency level, weight, or enabled-state edit, the graph control displays the current value or state, its Gap and Critical indicators agree with the summary analysis, and a subsequent edit starts from that current state.                                        | interaction | FAIL    | CompetencyGraph initializes React Flow node state once and never synchronizes it with changed model props. Metrics and alerts update, but node labels, sliders, switches, and Gap/Critical badges remain stale; later edits can therefore overwrite newer state with old values. |

## Appendix G

# Original and Corrected Checklist Comparison

Table G.1 compares the baseline agent on Claude Sonnet 5 across the fourteen applications, run ten times per application on WebTestBench’s original checklists and ten times on the corrected checklists of Appendix F. Values are means over the ten runs; effective tokens follow the definition of Section 5.4. Across applications, the paired difference is significant at  $p < 0.001$  for all three efficacy metrics. Effective token use rose by 61% on the corrected checklists, since these contain more test cases and the agent completed more of them.

| Application | Precision orig. | Precision corr. | Recall orig. | Recall corr. | F1 orig.    | F1 corr.    | Eff. tokens orig. (M) | Eff. tokens corr. (M) |
|-------------|-----------------|-----------------|--------------|--------------|-------------|-------------|-----------------------|-----------------------|
| 0001        | 0.10            | 0.75            | 0.13         | 0.60         | 0.11        | 0.66        | 1.35                  | 2.01                  |
| 0003        | 0.62            | 1.00            | 0.67         | 1.00         | 0.64        | 1.00        | 0.66                  | 1.12                  |
| 0004        | 1.00            | 1.00            | 0.57         | 0.96         | 0.73        | 0.98        | 0.69                  | 1.42                  |
| 0005        | 1.00            | 0.98            | 0.47         | 0.84         | 0.63        | 0.90        | 0.81                  | 1.99                  |
| 0006        | 0.83            | 0.78            | 0.37         | 1.00         | 0.50        | 0.87        | 1.21                  | 2.09                  |
| 0010        | 0.63            | 0.98            | 0.38         | 0.78         | 0.47        | 0.86        | 0.82                  | 1.13                  |
| 0019        | 0.63            | 0.89            | 0.54         | 0.83         | 0.58        | 0.85        | 1.30                  | 2.63                  |
| 0023        | 0.50            | 0.72            | 0.54         | 0.63         | 0.50        | 0.66        | 2.19                  | 3.04                  |
| 0045        | 1.00            | 1.00            | 0.50         | 0.87         | 0.63        | 0.92        | 0.75                  | 1.12                  |
| 0054        | 0.00            | 0.95            | 0.00         | 0.90         | 0.00        | 0.92        | 1.18                  | 1.61                  |
| 0063        | 0.00            | 0.60            | 0.00         | 0.15         | 0.00        | 0.24        | 0.64                  | 1.39                  |
| 0074        | 0.75            | 1.00            | 0.60         | 0.85         | 0.67        | 0.92        | 0.85                  | 1.29                  |
| 0089        | 0.92            | 0.97            | 0.71         | 0.82         | 0.80        | 0.87        | 1.12                  | 1.43                  |
| 0097        | 0.30            | 0.64            | 0.77         | 0.81         | 0.43        | 0.71        | 1.91                  | 2.68                  |
| <b>Mean</b> | <b>0.59</b>     | <b>0.88</b>     | <b>0.45</b>  | <b>0.79</b>  | <b>0.48</b> | <b>0.81</b> | <b>1.11</b>           | <b>1.78</b>           |

Table G.1: Original and corrected checklist metrics by application.

## Appendix H

# Supplementary Statistical Tests

Table H.1 gives, for each comparison in Table 6.1, the pooled change reported there and the mean of the fourteen per-application effects with its 95% confidence interval. Alongside each are two p-values: that of the paired t-test used in Chapter 6, and that of an exact two-sided Wilcoxon signed-rank test on the same fourteen effects, with zero differences excluded and midranks for ties. The column  $n$  gives the number of non-zero differences retained by the Wilcoxon test.

Efficacy effects are absolute differences; token, latency and turn effects are relative changes. Cost is a positive multiple of effective tokens and has the same test results. The two tests agree on every comparison except two, both borderline under the t-test: precision on Haiku 4.5 and turns on Opus 5. No conclusion in Chapter 6 depends on either.

| Model     | Metric           | Pooled $\Delta$ | Mean per-app $\Delta \pm$ 95% CI | t-test $p$ | Wilcoxon $p$ ( $n$ ) |
|-----------|------------------|-----------------|----------------------------------|------------|----------------------|
| Haiku 4.5 | Precision        | +0.13           | +0.13 $\pm$ 0.14                 | 0.06       | 0.04 (13)            |
|           | Recall           | +0.09           | +0.09 $\pm$ 0.12                 | 0.12       | 0.08 (12)            |
|           | F1               | +0.10           | +0.10 $\pm$ 0.12                 | 0.10       | 0.09 (13)            |
|           | Effective tokens | +1.7%           | +1.4% $\pm$ 6.9%                 | 0.68       | 0.71 (14)            |
|           | Latency          | +30.4%          | +32.0% $\pm$ 10.1%               | < 0.001    | < 0.001 (14)         |
|           | Turns            | +29.6%          | +32.0% $\pm$ 11.8%               | < 0.001    | < 0.001 (14)         |
| Sonnet 5  | Precision        | +0.03           | +0.03 $\pm$ 0.06                 | 0.36       | 0.34 (12)            |
|           | Recall           | +0.03           | +0.03 $\pm$ 0.04                 | 0.18       | 0.23 (12)            |
|           | F1               | +0.02           | +0.02 $\pm$ 0.05                 | 0.26       | 0.49 (14)            |
|           | Effective tokens | -15.9%          | -16.5% $\pm$ 7.7%                | < 0.001    | < 0.001 (14)         |
|           | Latency          | +6.2%           | +7.3% $\pm$ 9.4%                 | 0.12       | 0.22 (14)            |
|           | Turns            | +4.5%           | +4.6% $\pm$ 7.2%                 | 0.18       | 0.16 (14)            |
| Opus 5    | Precision        | -0.01           | -0.01 $\pm$ 0.02                 | 0.25       | 0.16 (9)             |
|           | Recall           | +0.02           | +0.02 $\pm$ 0.02                 | 0.03       | 0.02 (7)             |
|           | F1               | +0.01           | +0.01 $\pm$ 0.02                 | 0.43       | 0.37 (12)            |
|           | Effective tokens | -12.6%          | -11.6% $\pm$ 3.4%                | < 0.001    | < 0.001 (14)         |
|           | Latency          | -2.2%           | -2.3% $\pm$ 4.4%                 | 0.28       | 0.27 (14)            |
|           | Turns            | -2.9%           | -2.4% $\pm$ 2.7%                 | 0.07       | 0.04 (14)            |

Table H.1: Pooled and mean per-application effects of hints, with paired t-test and Wilcoxon signed-rank p-values.

## Appendix I

# Wall-Clock Time Decomposition

Table I.1 decomposes the wall-clock time of a run, reported as latency in Section 6.3, into the three components the harness records from its timestamps. Model time is the time spent waiting on model generation; tool time is the time spent executing tool calls, including browser actions and snapshots; the remainder is time attributable to neither, which the harness spends on session start-up, context compaction and result recording between one tool result and the next model call. Model generation accounts for 70–77% of a run on every model and arm, tool execution for 8–16%, and the remainder for 9–19%.

| Model     | Arm      | Model (s) | Tool (s) | Other (s) | Total (s) |
|-----------|----------|-----------|----------|-----------|-----------|
| Haiku 4.5 | Baseline | 258       | 28       | 84        | 370       |
|           | Hinted   | 326       | 47       | 111       | 483       |
|           | $\Delta$ | +26%*     | +64%*    | +31%      | +30%*     |
| Sonnet 5  | Baseline | 408       | 51       | 104       | 563       |
|           | Hinted   | 462       | 66       | 70        | 598       |
|           | $\Delta$ | +13%      | +29%     | −33%      | +6%       |
| Opus 5    | Baseline | 460       | 98       | 55        | 612       |
|           | Hinted   | 449       | 84       | 66        | 599       |
|           | $\Delta$ | −2%       | −14%*    | +21%*     | −2%       |

Table I.1: Wall-clock time per run by component, model and arm.

Seconds per run, means over fourteen applications  $\times$  ten runs. Model is time spent in model generation, Tool in tool execution, and Other is the remaining harness and orchestration time.  $\Delta$  is the pooled change from baseline to hinted. \*  $p < 0.05$ , two-tailed paired t-test on per-application means.

## Appendix J

# Application-Level Predictors of Token Savings

Table J.1 gives, for each model, the Pearson correlation across the fourteen applications between the change in effective tokens from baseline to hinted (negative values are savings) and four candidate predictors: the mean accessibility-snapshot size in the baseline arm, the per-snapshot reduction achieved by the semantic snapshot, the hinted agent’s rate of fallback on the accessibility snapshot, and the observation content consumed per baseline run. A positive coefficient means that larger values of the predictor go with smaller savings. The p-values are two-tailed, with  $n = 14$ .

| Predictor                                   | Haiku 4.5 $r$ ( $p$ ) | Sonnet 5 $r$ ( $p$ ) | Opus 5 $r$ ( $p$ ) |
|---------------------------------------------|-----------------------|----------------------|--------------------|
| Mean accessibility-snapshot size (baseline) | 0.45 (0.11)           | 0.01 (0.98)          | −0.06 (0.84)       |
| Per-snapshot size reduction                 | 0.15 (0.61)           | −0.03 (0.93)         | 0.06 (0.85)        |
| Fallback rate (hinted)                      | −0.24 (0.41)          | 0.50 (0.07)          | 0.18 (0.54)        |
| Observation content per baseline run        | 0.51 (0.06)           | 0.03 (0.91)          | −0.61 (0.02)       |

Table J.1: Correlations between application-level predictors and effective-token change.

## Appendix K

# Effects by Application and Category

Table K.1 gives the per-application effect of hints on effective tokens and on F1 for each model, grouped by the WebTestBench category of the application. With two applications per category, no statistical comparison between categories is possible, and the table is descriptive. It shows that the two applications in a category are typically as far apart as applications in different categories: on Sonnet 5, the two Presentation applications saved 44% and 2%, the two Search applications 5% and 3%, and the two Workflow applications 25% and 32%; on Haiku 4.5, the two Data Management applications moved by +0.72 and  $-0.06$  in F1. Within the range this dataset covers, the effect of the representation does not follow application category, and the variation between applications is better explained by the fallback rate and observation content examined in Section 6.4 and Appendix J.

| Category               | App  | Name             | Effective tokens ( $\Delta\%$ ) |        |      | F1 ( $\Delta$ ) |        |       |
|------------------------|------|------------------|---------------------------------|--------|------|-----------------|--------|-------|
|                        |      |                  | Haiku                           | Sonnet | Opus | Haiku           | Sonnet | Opus  |
| Presentation           | 0054 | TechSpecs        | +3                              | -44    | -16  | +0.05           | -0.02  | +0.02 |
|                        | 0063 | Chinese Heritage | -13                             | -2     | -2   | 0.00            | +0.17  | +0.07 |
| Search                 | 0001 | CuratedPicks     | +20                             | -5     | -17  | +0.14           | +0.16  | +0.01 |
|                        | 0003 | JobBoard         | -9                              | -3     | -14  | -0.02           | -0.01  | -0.04 |
| Tool                   | 0005 | ResumeBuilder    | +18                             | -7     | -12  | +0.01           | +0.06  | 0.00  |
|                        | 0097 | CompetencyHub    | +5                              | -12    | -10  | +0.26           | +0.10  | +0.02 |
| Commerce               | 0010 | TableSpot        | -8                              | -12    | -12  | +0.21           | -0.08  | -0.03 |
|                        | 0023 | ReVive           | 0                               | -2     | -15  | +0.15           | +0.08  | 0.00  |
| Data Management        | 0006 | LedgerFlow       | +17                             | -14    | -12  | +0.72           | -0.02  | +0.01 |
|                        | 0074 | FlexBase         | +11                             | -13    | +1   | -0.06           | -0.01  | +0.01 |
| Workflow               | 0004 | Ever After       | -6                              | -25    | -22  | -0.05           | -0.04  | 0.00  |
|                        | 0045 | My Tasks         | -20                             | -32    | -8   | +0.04           | +0.02  | +0.02 |
| User-Generated Content | 0019 | ThreadHive       | -1                              | -34    | -9   | +0.09           | 0.00   | -0.02 |
|                        | 0089 | Template Library | +2                              | -24    | -16  | -0.14           | -0.08  | +0.03 |

Table K.1: Effect of hints on tokens and F1 by application and category.

Per-application effect of hints, grouped by WebTestBench category (two applications each). Token change is the relative change in effective tokens per run, hinted versus baseline; F1 change is absolute. Each cell is the difference of two ten-run means.